# คู่มือทบทวนและเฉลยข้อสอบฉบับสมบูรณ์: ครอบคลุมทั้ง LECTURE และ LAB

## วิชา Digital Circuit Design with Verilog HDL (Week 2 ถึง Week 8 + Special Labs 9 & 11)

### บทนำ

เอกสารฉบับนี้จัดทำขึ้นเพื่อเป็น คู่มือการเรียนการสอนและเตรียมสอบฉบับสมบูรณ์ที่สุด (Master Comprehensive Study Guide)
โดยรวบรวมและวิเคราะห์เนื้อหาเชิงทฤษฎีจาก สไลด์บรรยาย (Lecture) ควบคู่ไปกับ เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ (Lab Sheet Exercises Solution)
ตั้งแต่ Week 2 ถึง Week 8 ไว้อย่างครบถ้วน 100%

ทุกแบบฝึกหัดมีรายละเอียดโจทย์ การเติมคำในช่องว่าง การค้นหาและแก้ไขบักในโค้ด Verilog การจับคู่ตัวดำเนินการ ตารางความจริง ตารางการทดลอง และโค้ด Verilog HDL ฉบับสมบูรณ์ พร้อมการอธิบายหลักการทำงานและระบุแหล่งที่มาอย่างเคร่งครัด

## [สารบัญ] สารบัญ


1. [Week 2: Quartus II Design Flow & DE0 Setup (Lec & Practice Lab 2)](#week-2)
2. [Week 3: Foundations & Verilog Language Basics (Lec & Lab 3)](#week-3)
3. [Week 4: Combinational Logic Design (Lec & Lab 4)](#week-4)
4. [Week 5: Sequential Logic Design (Lec & Lab 5)](#week-5)
5. [Week 6: Counters & Shift Registers (Lec & Lab 6)](#week-6)
6. [Week 7: Finite State Machine (FSM) Design (Lec & Lab 7)](#week-7)
7. [Week 8: Testbench Design & Functional Verification (Lec & Lab 8)](#week-8)
8. [Lab 9 (Special): การออกแบบวงจร AES-128 Encryption ด้วย Generative AI](#lab-9)
9. [Lab 11 (Special): ตัวนับขึ้น/ลงบน 7-Segment ด้วย RV32I SoC](#lab-11)
10. [ตารางสรุปภาพรวมความเชื่อมโยงตลอดหลักสูตร (Week 2 - Week 8)](#summary-table)


## [WEEK] Week 2: Quartus II Design Flow & DE0 Setup

### [LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

## [WEEK] Week 2 - Lecture: Software Installation & First DE0 Test

แหล่งอ้างอิงหลัก: ไฟล์ [Week2_software installation.pdf](file:///d:/vhdl/lec/Week2_software%20installation.pdf) (สไลด์ที่ 1-25)

### 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามที่เอกสารระบุ)

อ้างอิงจากสไลด์ที่ 1, 16, 17 และ 25:

- หัวข้อหลัก: Software Installation & First DE0 Test (EDA, Quartus II, ModelSim-Altera, USB-Blaster, Hello DE0)

- วัตถุประสงค์การเรียนรู้:

1. เข้าใจความหมายและความจำเป็นของเครื่องมือ EDA (Electronic Design Automation) และสถาปัตยกรรมพื้นฐานของ FPGA ([สไลด์ที่ 2-4](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L2))

2. รู้จักสถาปัตยกรรมภายในของชิป Cyclone และบอร์ดทดลอง ([สไลด์ที่ 5-15](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L5))

3. ติดตั้งโปรแกรม Quartus II 13.0sp1 Web Edition และ ModelSim-Altera Starter Edition 10.1d ([สไลด์ที่ 18-19](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L18))

4. ติดตั้งไดรเวอร์ USB-Blaster JTAG Interface ([สไลด์ที่ 20](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L20))

5. สร้างโปรเจกต์แรก (Hello DE0), กำหนดขา (Pin Assignment), สังเคราะห์วงจร (Compile) และดาวน์โหลดโปรแกรมลงบอร์ด FPGA เพื่อทดสอบวงจรเลียนแบบสวิตช์สู่ LED (SW[9:0] → LEDG[9:0]) ([สไลด์ที่ 21-23](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L21))

6. รู้วิธีการใช้ Gemini AI เข้ามาช่วยในการสร้างโค้ด Verilog, ทำความเข้าใจแนวคิด และแก้ไขข้อผิดพลาด (Debug) ([สไลด์ที 24](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L24))

---

## 2. แนวคิดหลัก (เรียงลำดับจากพื้นฐานไปสู่ระดับลึก)

### 2.1 EDA (Electronic Design Automation)

- นิยาม ([สไลด์ที 2](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L2)): ซอฟต์แวร์ที่ช่วยทำงานอัตโนมัติในการออกแบบ (design), จำลองการทำงาน (simulation) และตรวจสอบความถูกต้อง (verification) ของวงจรอิเล็กทรอนิกส์ เพื่อรองรับการออกแบบวงจรที่มีระดับล้านเกต

- ลำดับขั้นตอนการทำงาน (EDA Flow):

$$\text{Write Verilog HDL} \longrightarrow \text{Simulate (ModelSim)} \longrightarrow \text{Synthesize (Quartus)} \longrightarrow \text{Place \& Route} \longrightarrow \text{Program FPGA Board}$$

- เครื่องมือที่ใช้ในรายวิชา:

Quartus II: เครื่องมือหลักสำหรับ synthesis, fitting (place & route) และโปรแกรมลงบอร์ด FPGA

ModelSim-Altera: สำหรับทำ functional simulation ก่อนต่อฮาร์ดแวร์จริง

### 2.2 FPGA และการเปรียบเทียบกับ CPU และ ASIC

- นิยาม FPGA ([สไลด์ที 3](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L3)): Field-Programmable Gate Array คือชิปวงจรรวมที่สามารถปรับแต่งโครงสร้างลอจิกใหม่ได้หลังจากกระบวนการผลิต (Reconfigurable Hardware)

- คุณลักษณะสำคัญของ FPGA:

Reconfigurable Hardware: โปรแกรมซ้ำได้หลายพันครั้ง เหมาะแก่การทำต้นแบบ (prototyping)

Parallel Execution: ฮาร์ดแวร์ทำงานทุกส่วนพร้อมกันจริง ประมวลผลได้หลายคำสั่งภายใน 1 clock cycle

Low Latency: การตอบสนองระดับนาโนวินาที (Deterministic nanosecond-level response) ปราศจาก overhead ของระบบปฏิบัติการ (OS)

องค์ประกอบภายใน: สร้างขึ้นจาก Look-Up Tables (LUTs), registers/flip-flops, memory blocks และฮาร์ดแวร์ตัวคูณ DSP

- ตารางเปรียบเทียบสถาปัตยกรรม ([สไลด์ที 4](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L4)):

| หัวข้อการเปรียบเทียบ | ASIC | FPGA | CPU |
|---|---|---|---|
| Architecture | Fixed custom silicon | Reconfigurable logic fabric | Sequential, fixed cores |
| Performance | Highest | High - parallel hardware | Moderate |
| Flexibility | None after fab | Hardware reprogrammable | Software only |
| Development time | Months-years | Days-weeks | Fast |
| Unit cost | Very low (เมื่อผลิตจำนวนมาก) | Medium | Low (mass production) |
| Power | Lowest | Lower than CPU for same task | Moderate |
| Best for | Mass production chips | Prototyping, real-time DSP | General computing |

- แบรนด์และตระกูล FPGA ในท้องตลาด ([สไลด์ที 5](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L5)):

Intel / Altera: ตระกูล Cyclone, Arria, Stratix, Agilex (ใช้ Quartus Prime / Quartus II) โดยชิปบนบอร์ดทดลองคือ Cyclone III EP3C16F484C6

AMD / Xilinx: ตระกูล Spartan, Artix, Kintex, Virtex, Versal (ใช้ Vivado, ISE, Vitis)

Microchip / Microsemi / Lattice: ตระกูล PolarFire, iCE40, ECP5, IGLOO2 (เน้น Low power, security-focused)

### 2.3 สถาปัตยกรรมฮาร์ดแวร์ภายในตระกูล Cyclone

- ข้อมูลฮาร์ดแวร์บอร์ดทดลอง ([สไลด์ที 6](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L6)):

FPGA: Cyclone III EP3C16F484C6

Logic Elements (LEs): 15,408 LEs

หน่วยความจำฝังตัว (Embedded RAM): 504 Kbits

Phase-Locked Loops (PLLs): 4 ชุด

Clock Oscillator: 50 MHz (CLOCK_50)

สวิตช์: 10 x Slide Switch (SW[9:0])

LED: 10 x Green LED (LEDG[9:0])

การแสดงผล: 4 x 7-segment display (HEX0-HEX3)

ปุ่มกด: 3 x Push Button (KEY[2:0])

การเชื่อมต่อภายนอก: 2 x 40-pin expansion header (GPIO)

- บล็อกการทำงานหลัก (Overall Block Diagram - [สไลด์ที 7](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L7)):

1. Logic Array Blocks (LABs): โครงข่ายแบบ 2 มิติ แต่ละ LAB ประกอบด้วย 10 Logic Elements (LEs)

2. M4K Embedded RAM Blocks: หน่วยความจำ True dual-port ขนาด 4,608 บิตต่อบล็อก ทำงานได้ความเร็วสูงสุด 250 MHz ([สไลด์ที 12](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L12)) นำไปใช้ทำ ROM (โหลดด้วยไฟล์ .mif), FIFO Buffer (มีวงจรควบคุม full/empty ภายใน) หรือ Shift Register

3. Phase-Locked Loops (PLLs) และ Global Clock Network: มี PLL 2 ชุด (PLL1, PLL2) รองรับการคูณ, หาร และเลื่อนเฟสสัญญาณนาฬิกา (สูตร $m/(n \times \text{post-scale})$, ปรับเฟสต่ำสุดได้ 125 ps) และมี Global Clock Line 8 เส้นที่ค่า skew ต่ำสำหรับกระจายสัญญาณนาฬิกาทั่วทั้งชิป ([สไลด์ที 13](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L13))

4. I/O Elements (IOEs): อยู่บริเวณขอบชิป เป็นบัฟเฟอร์ 2 ทิศทางที่มี 3 รีจิสเตอร์ภายใน (Input Register, Output Register, Output Enable (OE) Register) รองรับมาตรฐานแรงดันไฟ เช่น LVTTL, LVCMOS (1.5-3.3 V), PCI 3.3 V, LVDS, SSTL ([สไลด์ที 14](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L14))

### 2.4 โครงสร้างเจาะลึกของ Logic Element (LE)

อ้างอิงจากสไลด์ที่ 8-11:

- ส่วนประกอบของ LE (หน่วยลอจิกที่เล็กที่สุด):

4-Input LUT (Look-Up Table): สร้างฟังก์ชันบูลีนใดๆ ที่มีตัวแปรขาเข้าไม่เกิน 4 ตัวแปร

Carry Chain: วงจรสายทดความเร็วสูงสำหรับงานคำนวณคณิตศาสตร์

Programmable Register: ฟลิปฟลอปแบบ D/T/JK/SR ทำงานสัมพันธ์กับสัญญาณนาฬิกาของ LAB

3 ทางออก (Outputs): ขับสัญญาณ local, row/column และ direct-link โดย LUT และ Register สามารถขับสัญญาณแยกอิสระจากกันได้ (Register Packing)

- โหมดการทำงานของ LE:

1. Normal Mode ([สไลด์ที 10](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L10)): สำหรับวงจรแบบ Combinational ทั่วไป (เช่น gates, multiplexers, decoders) โดยสัญญาณจาก LUT สามารถบายพาส D flip-flop (Register Bypass) ออกไปได้ทันทีโดยไม่ต้องผ่านสัญญาณนาฬิกา

2. Dynamic Arithmetic Mode ([สไลด์ที 11](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L11)): สำหรับวงจรบวก/ลบ และวงจรนับ (Counters/Adders) โดย LE จะเปลี่ยนเป็น 2-Input LUT จำนวน 4 ตัว ร่วมกับ Carry-Select Chain เพื่อคำนวณกรณีทดเข้าเป็น 0 และ 1 พร้อมกันล่วงหน้า และมีสัญญาณ addnsub ประจำ LAB สำหรับสลับการบวก/ลบด้วย Two's complement

### 2.5 การเชื่อมโยงคำสั่ง Verilog ไปเป็นโครงสร้างฮาร์ดแวร์ (Hardware Mapping)

ตารางสรุปการแปลงภาษา Verilog ไปสู่อุปกรณ์จริง ([สไลด์ที 15](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L15)):

| คำสั่ง Verilog (Verilog Construct) | ฮาร์ดแวร์ที่แปลงได้ (Maps to Hardwa | ตำแหน่งจริงบนชิป (Physical Location |
|---|---|---|
| `assign Y = A & B & C;` | LUT (Normal Mode) | ภายใน Logic Element (LE) |
| `always @(posedge clk)` | Flip-Flop Register | รีจิสเตอร์ภายใน LE |
| วงจรบวก/นับ เช่น `4-bit counter / a | Carry Chain + Arithmetic Mode | LE หลายตัวภายใน LAB |
| `reg [7:0] mem [255:0]` | M4K RAM Block | คอลัมน์ M4K RAM |
| `input` / `output` ports | I/O Element (IOE) | ขอบรอบนอกของชิป (Periphery) |
| `CLOCK_50` | Global Clock Network | เส้นสัญญาณนาฬิกาส่วนกลาง 8 เส้น |

## 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. กระบวนการออกแบบดิจิทัล: ต้องอาศัยเครื่องมือ EDA ในการแปลงจากโค้ดภาษาฮาร์ดแวร์ (Verilog HDL) ไปเป็นวงจรเกตลอจิก (Synthesis) จัดวางตำแหน่งและลากสาย (Place & Route / Fitter) จนได้ไฟล์ Bitstream (.sof/.pof) สำหรับโปรแกรมชิป

2. โครงสร้างพื้นฐานของ FPGA: ประกอบด้วยหน่วยพื้นฐานคือ LE (ภายในมี 4-input LUT และ 1 Flip-Flop) รวมกันเป็น LAB, มี M4K RAM, PLL และ IOE ควบคุมการเข้าออกสัญญาณ

3. การทำงานของ Verilog: ไม่ใช่การเขียนโปรแกรมเรียงลำดับแบบ CPU แต่เป็นการจำลองและการอธิบายการเชื่อมต่อโครงข่ายฮาร์ดแวร์จริง เช่น การใช้ assign เทียบเท่ากับการลากสายสัญญาณตรงเข้า LUT ในโหมด Normal

### 4. จุดที่เอกสารไม่ได้ระบุ / ข้อขัดแย้งในเอกสาร Lecture

- ข้อขัดแย้งเกี่ยวกับรุ่นบอร์ดและเบอร์ชิป FPGA ในเอกสาร:

ใน [สไลด์ที 5 และ 23](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L5) ระบุว่าบอร์ด DE0 ใช้ชิป Cyclone III EP3C16F484C6

แต่ใน [สไลด์ที 6](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L6) หัวข้อระบุชื่อว่า Altera DE1 Development Board แต่ตารางรายละเอียดด้านล่างกลับระบุสเปกของ DE0 (เช่น ชิป Cyclone III, 10 Green LEDs)

ใน [สไลด์ที 7 และ 12](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L7) นำรูปบล็อกไดอะแกรมของชิปรุ่นเก่าคือ Cyclone EP1C12 มาเป็นภาพตัวอย่างประกอบ

ใน [สไลด์ที 21](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L21) ขั้นตอนสร้างโปรเจกต์ระบุว่า: Family: Cyclone II, Device: EP2C20f4847n (matches the DE1 board chip) ซึ่งขัดแย้งกับสไลด์อื่นที่ระบุว่าเป็นบอร์ด DE0 (Cyclone III)

- จุดที่เอกสารไม่ได้ระบุ: เอกสารไม่ได้แสดงการเขียน Testbench สำหรับวงจร hello_de0 (มีเพียงการทดสอบ syntax เบื้องต้นของ ModelSim บนสไลด์ที 19 บรรทัดเดียว)

---

## [LAB] Week 2 - Lab: Practice Lab Sheet 2 & Hello DE0

แหล่งอ้างอิงหลัก:

1. [Practice Lab Sheet 2.pdf](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf) (หน้า 1-4)
2. [Week2_software installation.pdf](file:///d:/vhdl/lec/Week2_software%20installation.pdf) (สไลด์ที 17-23)

---

### 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร)

อ้างอิงจาก Lab Sheet 2 และสไลด์ปฏิบัติการ:

- วัตถุประสงค์:

1. ติดตั้งเครื่องมือ Quartus II, ModelSim-Altera และไดรเวอร์ USB-Blaster บนระบบปฏิบัติการ Windows/Linux ([สไลด์ที 18-20](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L18))

2. ดำเนินการสร้างโปรเจกต์แรก hello_de0 เพื่อเชื่อมต่อสัญญาณสวิตช์ SW[9:0] ตรงไปยังหลอดไฟ LEDG[9:0] บนบอร์ดจริง ([สไลด์ที 21-23](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L21))

3. ออกแบบวงจรลอจิกกดปุ่ม KEY[3:0] แสดงผลออก 7-segment display (HEX0) ตามโจทย์ใน Lab Sheet 2 ข้อที 1 ([หน้า 1-2](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf#L1)):

กด KEY[0] $\rightarrow$ HEX0 แสดง '0'

กด KEY[1] $\rightarrow$ HEX0 แสดง '1'

กด KEY[2] $\rightarrow$ HEX0 แสดง '2'

กด KEY[3] $\rightarrow$ HEX0 แสดง '3'

ไม่กดปุ่มใด $\rightarrow$ HEX0 ดับทั้งหมด

หมายเหตุ: ปุ่ม KEY บนบอร์ด DE0 เป็น active-low (กดปุ่ม = 0, ปล่อยปุ่ม = 1)

4. ออกแบบวงจรนับเลขตามโจทย์ใน Lab Sheet 2 ข้อที 2 ([หน้า 4](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf#L4)):

กด KEY[0] ค้างไว้ $\rightarrow$ HEX0 นับเพิ่ม $0, 1, 2, \dots, 9$ (เพิ่มทุก 1 วินาที) และเมื่อถึง 9 ให้ค้างไว้

ปล่อย KEY[0] $\rightarrow$ HEX0 นับถอยหลัง $9, 8, \dots, 0$ (ลดทุก 1 วินาที) และเมื่อถึง 0 ให้ค้างไว้

ขณะปล่อย KEY[0] แต่ตัวเลขยังนับไม่ถึง 0 ให้นับขึ้นต่อ

---

### 2. การเชื่อมโยงกับ Lecture

- Continuous Assignment (assign) $\leftrightarrow$ Combinational LE Normal Mode: โค้ด assign LEDG = SW; เชื่อมโยงกับทฤษฎีในเลกเชอร์ ([สไลด์ที 10 และ 15](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L10)) โดย Quartus จะต่อสัญญาณจาก Input IOE ตรงไปยัง Output IOE หรือผ่าน LUT ในโหมดปกติโดยไม่มีการผ่านฟลิปฟลอป

- I/O Elements (IOEs) $\leftrightarrow$ Pin Assignment: การทำข้อกำหนดขาพินทางกายภาพ (Assignments $\rightarrow$ Pin Planner) น่าพอร์ตเสมือนในโค้ด Verilog (SW, LEDG) ไปผูกกับขาจริงของชิป Cyclone เช่น PIN_J6 และ PIN_J1 ([สไลด์ที 14 และ 23](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L14))

- Active-Low Push Buttons & IOE: ปุ่มกด KEY และหน้าจอ HEX0 ทีเป็น active-low
  สัมพันธ์กับโครงสร้างอินพุต/เอาต์พุตของบอร์ดทดลองทีอธิบายไว้ในสเปกฮาร์ดแวร์ของเลกเชอร์ ([สไลด์ที 6](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L6))

---

## 3. อธิบายโค้ด Verilog ทีละส่วน

**โค้ดที 1: `hello_de0.v` (คัดลอกจาก [สไลด์ที 22](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L22))**

```
module hello_de0 (
input
[9:0] SW,
output
[9:0] LEDG
);
assign
LEDG = SW;
// SW mirrors LEDG
endmodule
```

- การทำงานทีละส่วน:

module hello_de0 (...) ... endmodule: กำหนดขอบเขตของวงจรฮาร์ดแวร์ (โมดูล)

input [9:0] SW: กำหนดพอร์ตขาเข้าขนาด 10 บิต (เวกเตอร์บิต 9 ถึง 0) สำหรับรับสัญญาณจากสวิตช์เลือน 10 ตัวบนบอร์ด

output [9:0] LEDG: กำหนดพอร์ตขาออกขนาด 10 บิต สำหรับขับหลอดไฟ LED สีเขียว 10 ดวงบนบอร์ด

assign LEDG = SW;: คำสัง Continuous assignment ทำหน้าทีเปรียบเสมือนสายสัญญาณต่อตรงทางฮาร์ดแวร์ เมือสวิตช์เปลียนสถานะ หลอดไฟ LED จะเปลียนตามทันที

- การจัดประเภทวงจร: เป็นวงจรแบบ Combinational Logic ล้วน (ไม่มี clock, ไม่มี flip-flop, สัญญาณบายพาสรีจิสเตอร์ตามสไลด์ที 10)

**โค้ดที 2: โค้ดทดสอบ ModelSim เบืองต้น (คัดลอกจาก [สไลด์ที 19](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L19))**

```
module test; initial $display("OK"); endmodule
```

- โค้ดโมดูลอย่างง่ายสำหรับพิมพ์ข้อความ "OK" ออกหน้าต่างคอนโซลของ ModelSim เพือทดสอบว่าซอฟต์แวร์จำลองวงจรทำงานได้ถูกต้อง

**หมายเหตุเรืองโค้ดใน Lab Sheet 2 (ข้อ 1 และ 2):**

- ในเอกสารไม่ได้ให้โค้ด Verilog สำเร็จรูปมา: เอกสาร [Practice Lab Sheet 2.pdf](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf) กำหนดให้ผู้เรียนนำข้อความ Prompt ไปสังให้ Gemini AI สร้างโค้ดขึนมาเอง และเว้นว่างโค้ดไว้ จึงไม่มีโค้ด Verilog ของ 7-Segment หรือวงจรนับเลขปรากฏในเอกสารต้นฉบับ

---

## 4. ขันตอนการทำ Lab และผลลัพธ์ทีคาดหวัง (เฉพาะทีเอกสารระบุ)

อ้างอิงจากขันตอนในสไลด์ที 21-23:

1. New Project Wizard: เลือกเมนู File → New Project Wizard ตังชือโปรเจกต์และ Top-level entity เป็น hello_de0

2. Select Device: เลือกตระกูล Cyclone II ชิป EP2C20f4847n (ตามขันตอนในสไลด์ 21)

3. Create Verilog File: สร้างไฟล์ใหม่ hello_de0.v เขียนโค้ดตามสไลด์ที 22

4. Pin Assignment: เข้าเมนู Assignments → Pin Planner กำหนดขาพินตามตาราง ([สไลด์ที 23](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L23)):

SW[0] $\rightarrow$ PIN_J6, SW[1] $\rightarrow$ PIN_K6, SW[2] $\rightarrow$ PIN_K5, SW[3] $\rightarrow$ PIN_H5 (สวิตช์ 4-9 ดูจากคู่มือ DE0)

LEDG[0] $\rightarrow$ PIN_J1, LEDG[1] $\rightarrow$ PIN_J2, LEDG[2] $\rightarrow$ PIN_J3, LEDG[3] $\rightarrow$ PIN_J4 (LED 4-9 ดูจากคู่มือ DE0)

5. Compile: กด Processing → Start Compilation ใช้เวลารอประมาณ 2-5 นาที ตรวจสอบให้ผ่านโดยมีข้อผิดพลาดเป็น 0 (0 errors)

6. Program DE0: เข้าเมนู Tools → Programmer เลือกไฟล์บิตสตรีม .sof กดปุ่ม Start จนสถานะขึน Programming Done!

- ผลลัพธ์ทีคาดหวัง: เมือเลือนสวิตช์ตัวใด หลอด LEDG ตำแหน่งตรงกันจะสว่างทันที (SW mirrors LEDG)

---

### 5. ข้อควรระวัง / ข้อผิดพลาดที่พบบ่อย (เฉพาะที่เอกสารกล่าวถึง)

- USB-Blaster ตรวจจับไม่พบ ([สไลด์ที่ 20](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L20)): ในหน้าต่าง Tools → Programmer → Hardware Setup หากไม่ปรากฏรายการ "USB-Blaster" ให้รีสตาร์ตเครื่องคอมพิวเตอร์และตรวจสอบการลงไดรเวอร์ผ่าน Device Manager (ชี้ไปที่โฟลเดอร์ C:\altera\13.0sp1\quartus\drivers\usb-blaster)

- สิทธิ์การใช้งานบนระบบปฏิบัติการ Linux ([สไลด์ที่ 20](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L20)): ไดรเวอร์ของ Linux รวมอยู่ใน Quartus II แล้ว แต่ต้องตั้งสิทธิ์ของพอร์ต USB ด้วยคำสั่ง sudo chmod a+rw /dev/bus/usb// หรือเพิ่มผู้ใช้เข้ากลุ่ม plugdev

- ข้อจำกัดของ ModelSim Starter Edition ([สไลด์ที่ 19](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L19)): มีการจำกัดความเร็วในการจำลองวงจร (simulation speed limit) แต่เพียงพอสำหรับการเรียนในรายวิชานี้

### 6. สรุป Lab และตารางโยง Lecture ↔ Lab

| แนวคิดจาก Lecture | อยู่ในส่วนใดของโค้ด / ขั้นตอนปฏิบัติ |
|---|---|
| EDA Tool Flow (Synthesize & Program | ขั้นตอนการ Compile โปรเจกต์ `hello_ |
| Continuous Assignment (`assign`) | บรรทัด `assign LEDG = SW;` ในไฟล์ ` |
| I/O Element (IOE) & Pin Mapping | การตั้งค่าขาพินในหน้าต่าง Pin Plann |
| Active-Low Logic | ข้อกำหนดของสวิตช์ปุ่มกด `KEY` และหล |

### 7. ส่วนที่เอกสารไม่ได้ระบุ / จุดที่เว้นว่างในเอกสาร

1. โค้ดและเฉลยของ Practice Lab Sheet 2:

ตารางค่าตัวเลขฐานสองและฐาน 16 ของ HEX[6:0] ในหน้า 2 เป็นช่องตารางว่างสำหรับผู้เรียนกรอก

คำอธิบายเหตุผลว่าทำไมต้องใส่ ~ หน้า KEY ในหน้า 2 เป็นเส้นประว่าง

รูปวงจร RTL ของการสังเคราะห์ในหน้า 3 เป็นพื้นที่ว่าง

โค้ด Verilog และ Prompt ที่ใช้สำหรับวงจรนับเลขในหน้า 4 เป็นเส้นประว่าง

ทั้งหมดนี้ไม่มีการระบุข้อมูลหรือคำตอบไว้ในเอกสารต้นฉบับ

2. ความไม่สอดคล้องของจำนวนสวิตช์ปุ่มกด (KEY):

เลกเชอร์ [สไลด์ที่ 6](file:///d:/vhdl/lec/Week2_software%20installation.pdf#L6) ระบุว่าบอร์ดมี 3 x Push Button (KEY[2:0])

แต่ในโจทย์ Practice Lab Sheet 2 [หน้าที่ 1](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf#L1) กลับระบุให้กดปุ่ม KEY0-3 และใช้พอร์ต KEY[3:0] (มี 4 ปุ่มกด)

ต้องการให้อาจารย์อธิบายต่อใน Week 3 (Lecture & Lab 3) เลยหรือไม่ครับ?

## [LAB] ส่วนที่ 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 2

# [LAB] เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ: Practice Lab Sheet 2

เอกสารอ้างอิง: [Practice Lab Sheet 2.pdf](file:///d:/vhdl/lab/Practice%20Lab%20Sheet%202.pdf) และ [Week2_software installation.pdf](file:///d:/vhdl/lec/Week2_software%20installation.pdf)

ไฟล์ที่ทำเสร็จแล้ว: [Practice_Lab_Sheet_2_Answered.pdf](file:///d:/vhdl/lab/Practice_Lab_Sheet_2_Answered.pdf)

## ■ ตอนที่ 1: วงจรกดปุ่ม KEY0-3 แสดงตัวเลขบน 7-Segment

### 1.1 โจทย์และพฤติกรรมวงจร (ตามเอกสาร)

สร้างวงจรที่มีพฤติกรรมดังนี้:

- กด KEY[0] $\rightarrow$ HEX0 แสดง '0'
- กด KEY[1] $\rightarrow$ HEX0 แสดง '1'
- กด KEY[2] $\rightarrow$ HEX0 แสดง '2'
- กด KEY[3] $\rightarrow$ HEX0 แสดง '3'
- ไม่กดปุ่มใด $\rightarrow$ HEX0 ดับทั้งหมด

- หมายเหตุ: ปุ่ม KEY บนบอร์ด DE0 เป็นแบบ Active-Low (กดปุ่ม = logic 0, ปล่อยปุ่ม/ไม่กด = logic 1)

---

### 1.2 โค้ด Verilog ฉบับสมบูรณ์ (คำตอบที่สร้างขึ้นสำหรับ Lab นี้)

```
//=======================================================================
// Course: 03603473-65 Digital System Design on FPGAs with Verilog
// Lab Sheet 2: Practice 1 - Push Buttons KEY[3:0] to 7-Segment HEX0[6:0]
// Target Board: Altera DE0 (Cyclone III EP3C16F484C6)
//=======================================================================
module lab2_key_7seg (
    input  [3:0] KEY,      // Push button inputs (Active-Low: Pressed = 0, Released = 1)
    output reg [6:0] HEX0  // 7-segment display (Active-Low: ON = 0, OFF = 1)
                           // Bit Mapping: HEX0[6:0] = {g, f, e, d, c, b, a}
);

    // Continuous Combinational Logic
    // □□□□□□□□□□□□□□□□□□□□□□ Active-Low (□□=0) □□□□□□□□□□ Bitwise NOT (~KEY)
    // □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ Active-High □□□□□□□□□□□□ (□□=1)
    always @(*) begin
        case (~KEY)
            4'b0001: HEX0 = 7'b1000000; // □□ KEY[0] -> □□□□□□□□ '0' (0x40): □□□□□ a,b,c,d,e,f
            4'b0010: HEX0 = 7'b1111001; // □□ KEY[1] -> □□□□□□□□ '1' (0x79): □□□□□ b,c □□□□, □□
            4'b0100: HEX0 = 7'b0100100; // □□ KEY[2] -> □□□□□□□□ '2' (0x24): □□□□□ a,b,d,e,g □
            4'b1000: HEX0 = 7'b0110000; // □□ KEY[3] -> □□□□□□□□ '3' (0x30): □□□□□ a,b,c,d,g □
            default: HEX0 = 7'b1111111; // □□□□□□□□□□□□□□ □□□□□□□□□□□□□□□□□□ -> □□□□□□□□□□□□ (0x7F)
        endcase
    end

endmodule
```

---

### 1.3 ตอบคำถามข้อที่ 1 (หน้า 2): ตารางตัวเลขฐานสองและฐาน 16 ของ `HEX[6:0]`

โครงสร้างบิต 7-segment บนบอร์ดเป็นแบบ Active-Low Common Anode (หลอดติดเมื่อบิตเป็น 0, หลอดดับเมื่อบิตเป็น 1) เรียงลำดับบิตจากสูงสุดไปต่ำสุดคือ HEX0[6:0] = {g, f, e, d, c, b, a}:

| ตัวเลขที่แสดง | ฐาน 2 (`HEX[6:0]` gfedcba) | ฐาน 16 (`HEX[6:0]`) | คำอธิบายหลอดที่ติด (Active = 0) |
|---|---|---|---|
| 0 (กด `KEY[0]`) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | แถบ a |
| 1 (กด `KEY[1]`) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | แถบ b |
| 2 (กด `KEY[2]`) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | แถบ a |
| 3 (กด `KEY[3]`) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | แถบ a |
| ดับ (ไม่กดปุ่ม) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | ดับทุ |

---

### 1.4 ตอบคำถามข้อที่ 2 (หน้า 2): เหตุผลที่ต้องใส่ `~` หน้า KEY ในโค้ด Verilog

คำตอบ:

1. ลักษณะทางกายภาพของวงจรปุ่มกด (Active-Low Push Button):<br>

ปุ่มกด KEY บนบอร์ดทดลอง Altera DE0 มีวงจรดึงแรงดันขึ้น (Pull-up Resistor) อยู่ภายใน เมื่อสวิตช์เปิดวงจร (ไม่ได้กดปุ่ม) ขาพินของ FPGA จะอ่านค่าแรงดันไฟฟ้าได้ระดับสูง นั่นคือสถานะลอจิกเป็น 1 (1'b1) แต่เมื่อผู้ใช้งานออกแรงกดปุ่ม หน้าสัมผัสของสวิตช์จะลัดวงจรต่อลงกราวด์ (GND) ทำให้ขาพินอ่านค่าแรงดันได้ระดับต่ำ นั่นคือสถานะลอจิกตกเป็น 0 (1'b0)

2. บทบาทของตัวดำเนินการ Bitwise NOT (<code>~</code>):<br>

การใส่เครื่องหมาย <code>~</code> หน้าตัวแปร <code>KEY</code> (เขียนเป็น <code>~KEY</code>) คือการสั่ง Invert สัญญาณทุกบิตทางลอจิก ส่งผลให้:

<ul>

<li>เมื่อกดปุ่ม (รับค่า 0) สัญญาณจะถูกกลับสถานะกลายเป็น 1 (Active-High ภายในวงจร)</li>

<li>เมื่อปล่อยปุ่ม (รับค่า 1) สัญญาณจะถูกกลับสถานะกลายเป็น 0</li>

</ul>

ช่วยให้การเขียนโค้ดตรวจสอบเงื่อนไขในคำสั่ง <code>case</code> หรือ <code>if-else</code> สอดคล้องกับการคิดตามธรรมชาติ (Intuitive Logic) เช่น กำหนดให้ <code>4'b0001</code> สื่อถึงการกดปุ่ม <code>KEY[0]</code> และ <code>4'b0010</code> สื่อถึงการกดปุ่ม <code>KEY[1]</code> ทำให้อ่านโค้ดเข้าใจได้ง่าย ไม่สับสน และลดโอกาสเกิดข้อผิดพลาดในการเขียนเงื่อนไขตรวจสอบปุ่มกด

### 1.5 ตอบคำถามข้อที่ 3 (หน้า 3): วงจร RTL ที่ได้หลังจากสังเคราะห์วงจรเสร็จสิ้น

```
flowchart LR
    subgraph INPUTS["Input IOE"]
        KEY["KEY[3:0]<br/>(Active-Low Pins)"]
    end

    subgraph INVERTER["Bitwise Inverter"]
        INV["4x NOT Gates<br/>(~KEY)"]
    end

    subgraph DECODER["Combinational Logic (LE Normal Mode)"]
        DEC["7-Segment Decoder<br/>(4-to-7 Priority Case Mux)<br/>4'b0001 -> 7'h40<br/>4'b00
    end

    subgraph OUTPUTS["Output IOE"]
        HEX["HEX0[6:0]<br/>(Active-Low Display)"]
    end

    KEY -->|"4-bit wire"| INV
    INV -->|"4-bit wire (~KEY)"| DEC
    DEC -->|"7-bit bus"| HEX

    style INPUTS fill:#eef2ff,stroke:#4338ca,stroke-width:1px
    style INVERTER fill:#e0f2fe,stroke:#0369a1,stroke-width:1px
    style DECODER fill:#f0fdf4,stroke:#15803d,stroke-width:2px
    style OUTPUTS fill:#fef2f2,stroke:#b91c1c,stroke-width:1px
```

คำอธิบายโครงสร้างวงจร RTL:

1. Input Buffer & Inverters: สัญญาณปุ่มกด 4 บิต <code>KEY[3:0]</code> เข้าสู่วงจร Inverter 4 ชุด เพื่อกลับระดับสัญญาณเป็น Active-High (<code>~KEY</code>)<br>

2. Combinational Decoder Logic: ถูกสังเคราะห์ลงใน 4-Input LUT (Normal Mode) ภายใน Logic Element (LE) ของ Cyclone FPGA ทำหน้าที่เลือกส่งรหัส 7 บิตตามตารางถอดรหัส<br>

3. Output Buffer: ส่งสัญญาณบัส 7 บิต <code>HEX0[6:0]</code> ผ่าน I/O Element (IOE) ขับหลอด 7-Segment ชนิด Common Anode บนบอร์ดทดลองโดยตรง

## ■ ตอนที่ 2: วงจรนับเลข (0-9 Up/Down Counter บน 7-Segment)

### 2.1 โจทย์และพฤติกรรมวงจร (ตามเอกสาร หน้า 4)

- กด KEY[0] ค้างไว้ $\rightarrow$ HEX0 นับเพิ่ม $0, 1, 2, \dots, 9$ (เพิ่มทุก 1 วินาที)
- เมื่อนับถึง 9 แล้ว $\rightarrow$ ค้างที่ 9 ไม่นับเพิ่ม (Saturate at 9)
- ปล่อย KEY[0] $\rightarrow$ HEX0 นับถอยหลัง $9, 8, \dots, 0$ (ลดทุก 1 วินาที)
- เมื่อนับถึง 0 แล้ว $\rightarrow$ ค้างที่ 0 ไม่นับลด (Saturate at 0)
- หมายเหตุข้อความในเอกสาร: "ขณะปล่อย KEY[0] แต่ตัวเลขยังนับไม่ถึง 0 $\rightarrow$ ให้นับขึ้นต่อ"

### 2.2 Prompt สำหรับ Gemini ทีใช้ (เขียนลงในเอกสาร หน้า 4)

ข้อความ Prompt:

```
"□□□□□□□□□□□□ Altera DE0 (Cyclone III EP3C16F484C6) □□□□□□□□□□□□□□□□ Quartus II 13.0sp1 □□□□□□□□
□□ Verilog module □□□□□□□□□□□□□□□□□□□□□□□□□□□□ 0-9 □□□□□□□□□ 7-segment display (HEX0) □□□ Active-L
1. Input:
   - CLOCK_50: □□□□□□□□□□□□□□ 50 MHz □□□□□□□□□
   - KEY[0]: □□□□□□□□□□ Active-Low (□□□□□□□ = 0, □□□□□ = 1)
2. Output:
   - HEX0[6:0]: □□□□□□□□□□ 7-segment 1 □□□□□ □□□ Active-Low Common Anode (gfedcba)
3. □□□□□□□□□□□□□□□□□□□□:
   - □□□□□□□□□□□□□□□□□□□□□□□□ (Clock Divider) □□□ 50 MHz □□□□□□□□□□□□□□□□□□□ tick □□□ 1 □□□□□□□ (1
   - □□□□□□□□□□□□□ KEY[0] □□□□□□□□: □□□□□□□□□□□□□□□□ 1 □□□□□ 1 □□□□□□□ (0, 1, 2, ..., 9) □□□□□□□□□□□□
   - □□□□□□□□□□□□□□□□ KEY[0]: □□□□□□□□□□□□□□□□□□□□□ 1 □□□□□ 1 □□□□□□□ (9, 8, ..., 0) □□□□□□□□□□□□□□□□
   - □□□□□□□□□□□□□□□ (7-segment decoder) □□□□□□□□□□□□□□□□□□ 0-9 □□□□□□□□□□□□□□□□□□□□□□□□ HEX0 □□□ Ac
□□□□□□□ Verilog □□□□□□□□□□□□□□□□□□□□□□□□ (Synthesizable) □□□□□□ inline comment □□□□□□□□□□□□□□□□□□□□□□□□
```

### 2.3 โค้ด Verilog ฉบับสมบูรณ์สำหรับวงจรนับเลข

```
//================================================================================
// Course: 03603473-65 Digital System Design on FPGAs with Verilog
// Lab Sheet 2: Practice 2 - 1-Second Up/Down Saturating Counter on HEX0
// Target Board: Altera DE0 (Cyclone III EP3C16F484C6)
//================================================================================
module lab2_counter_7seg (
    input        CLOCK_50,  // On-board 50 MHz oscillator
    input  [0:0] KEY,       // Push button KEY[0] (Active-Low: Pressed=0, Released=1)
    output reg [6:0] HEX0   // 7-segment display (Active-Low: gfedcba)
);

    //--------------------------------------------------------------------------
    // 1. Clock Divider: □□□□□□□□□□□□□□□□□□ 1 □□□□□□□ (1 Hz tick)
    // □□□□□□: 50,000,000 cycles / 1 Hz = 50,000,000 cycles
    // □□□□□□□□□□□□□□□□□□□□ 26 □□□□ (2^26 = 67,108,864 > 50,000,000)
    //--------------------------------------------------------------------------
    reg [25:0] clk_div = 26'd0;
    reg tick_1s = 1'b0;

    always @(posedge CLOCK_50) begin
        if (clk_div == 26'd49_999_999) begin
            clk_div <= 26'd0;
            tick_1s <= 1'b1;  // □□□□□□□□□□□□□□□□□□□□□□ 1 □□□ clock □□□ 50MHz
        end else begin
            clk_div <= clk_div + 1'b1;
            tick_1s <= 1'b0;
        end
    end

    //--------------------------------------------------------------------------
    // 2. Up/Down Saturating Counter (0 □□□ 9)
    // □□□□□□□ sequential logic □□□□□□□□□□□□□□□□□□□□□□□□□□□□
    //--------------------------------------------------------------------------
    reg [3:0] count = 4'd0;
    wire key0_pressed = ~KEY[0]; // Active-High: □□□□ 1 □□□□□□□□□□□□ KEY[0]

    always @(posedge CLOCK_50) begin
        if (tick_1s) begin
            if (key0_pressed) begin
                // □□□□□□□□□□□□□□□□□□□□□□□□□□□□: □□□□□□□□□□□□ 1 □□□□□□ 9 □□□□□□□□□□□□□□□
                if (count < 4'd9)
                    count <= count + 1'b1;
                else
                    count <= 4'd9;
```

```
            end else begin
                // ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯: ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯ 1 ▯▯▯▯▯▯ 0 ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯
                if (count > 4'd0)
                    count <= count - 1'b1;
                else
                    count <= 4'd0;
            end
        end
    end

    //------------------------------------------------------------------------
    // 3. 7-Segment Decoder (Active-Low Common Anode: gfedcba)
    // ▯▯▯▯▯ Combinational logic ▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯▯ 0-9 ▯▯▯▯▯▯▯▯▯▯▯▯ 7 ▯▯
    //------------------------------------------------------------------------
    always @(*) begin
        case (count)
            4'd0: HEX0 = 7'b1000000; // 0x40
            4'd1: HEX0 = 7'b1111001; // 0x79
            4'd2: HEX0 = 7'b0100100; // 0x24
            4'd3: HEX0 = 7'b0110000; // 0x30
            4'd4: HEX0 = 7'b0011001; // 0x19
            4'd5: HEX0 = 7'b0010010; // 0x12
            4'd6: HEX0 = 7'b0000010; // 0x02
            4'd7: HEX0 = 7'b1111000; // 0x78
            4'd8: HEX0 = 7'b0000000; // 0x00
            4'd9: HEX0 = 7'b0010000; // 0x10
            default: HEX0 = 7'b1111111; // ▯▯▯▯▯▯▯▯▯▯▯▯
        endcase
    end

endmodule
```

### 2.4 ข้อสังเกตความขัดแย้งในเอกสาร (ตามกฎเหล็กข้อ 5)

- ในโจทย์ตอนที 2 มีข้อความบรรทัดหนึงระบุว่า:

"ปล่อย KEY[0] $\rightarrow$ HEX0 นับถอยหลัง 9, 8, ... 0 (ลดทุก 1 วินาที) เมือนับถึง 0 แล้ว $\rightarrow$ ค้างที 0 ไม่นับลด"

แต่บรรทัดถัดมาระบุว่า: "ขณะปล่อย KEY[0] แต่ตัวเลขยังนับไม่ถึง 0 $\rightarrow$ ให้นับขึนต่อ"

- จุดทีขัดแย้ง: ข้อความสองบรรทัดนีมีความขัดแย้งกันโดยตรง เพราะหากปล่อยปุ่มแล้วให้นับถอยหลัง ย่อมไม่สามารถนับขึนต่อในขณะทีปล่อยปุ่มได้ โค้ดด้านบนจึงออกแบบตามตรรกะทีเป็นไปได้หลัก (กด = นับขึนถึง 9, ปล่อย = นับถอยหลังลงมาถึง 0)

## ☐ สรุปไฟล์ทีสร้างขึน

- ไฟล์เอกสารทีกรอกคำตอบแล้ว: [Practice_Lab_Sheet_2_Answered.pdf](file:///d:/vhdl/lab/Practice_Lab_Sheet_2_Answered.pdf)

ต้องการให้อาจารย์ไปต่อที Week 3 (Lecture & Lab 3) เลยหรือไม่ครับ? โดยในสัปดาห์ถัดไปจะจัดทำคำตอบของไฟล์ Lab เป็นสีอีนตามรูปแบบนีเช่นเดียวกันครับ

# [WEEK] Week 3: Foundations & Verilog Language Basics

## [LECTURE] ส่วนที 1: สรุปเนือหา LECTURE

# [WEEK] Week 3 - Lecture: Foundations & Language Basics

แหล่งอ้างอิงหลัก: ไฟล์ [Week3_foundations and language basics.pdf](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf) (สไลด์ที 1-33)

## 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามที่เอกสารระบุ)

อ้างอิงจากสไลด์ที่ 1, 2 และ 33:

- หัวข้อหลัก: Verilog HDL Foundations & Language Basics (History, Introduction, Specification, Module, Data Types & Operators)

- วัตถุประสงค์การเรียนรู้:

1. เข้าใจประวัติความเป็นมาของภาษา Verilog, เส้นทางสู่มาตรฐาน IEEE และการเปรียบเทียบระหว่าง Verilog กับ VHDL ([สไลด์ที่ 3-5](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L3))

2. เข้าใจธรรมชาติของ HDL ที่แตกต่างจากการเขียนโปรแกรมซอฟต์แวร์, ระดับชั้นของการออกแบบ (Abstraction Levels) และรูปแบบการเขียนวงจร 3 สไตล์ (Behavioral, Dataflow, Structural) ([สไลด์ที่ 6-11](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L6))

3. เรียนรู้ขั้นตอนการแปลงข้อกำหนดวงจร (Specification) ไปสู่วงจรจริงผ่านกรณีศึกษา Arbiter ([สไลด์ที่ 12-16](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L12))

4. เข้าใจโครงสร้างของ Module, การเชื่อมต่อพอร์ตแบบลำดับขั้น (Hierarchical Module Instantiation) ทั้งแบบ Named Mapping และ Positional Mapping ([สไลด์ที่ 17-19](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L17))

5. ใช้งานชนิดข้อมูลพื้นฐาน โดยเฉพาะความแตกต่างระหว่าง wire กับ reg, การกำหนดสัญญาณ Scalar/Vector, ลอจิก 4 สถานะ (4-state logic) และการแทนค่าตัวเลขคงที่ (Number Literals) ([สไลด์ที่ 20-26](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L20))

6. เข้าใจและประยุกต์ใช้ตัวดำเนินการทางคณิตศาสตร์, ลอจิก, ระดับบิต (Bitwise vs Logical), การต่อบัส (Concatenation) และลำดับความสำคัญของตัวดำเนินการ (Operator Precedence) ([สไลด์ที่ 27-32](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L27))

---

## 2. แนวคิดหลัก (เรียงลำดับจากพื้นฐานไปสู่ระดับลึก)

### 2.1 ประวัติความเป็นมาและมาตรฐาน Verilog

- จุดเริ่มต้น ([สไลด์ที่ 3](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L3)): พัฒนาขึ้นในปี 1984 โดย Phil Moorby ที่บริษัท Gateway Design Automation สำหรับโปรแกรมจำลองวงจร ต่อมาในปี 1989 บริษัท Cadence ซื้อกิจการไป และเปิดซอร์สให้เป็นสาธารณะ (Public domain) ในปี 1990 เพื่อแข่งขันกับ VHDL

- เส้นทางสู่มาตรฐานสากล ([สไลด์ที่ 4](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L4)):

1995 $\rightarrow$ IEEE 1364-1995 (Verilog-95): ก้าวขึ้นเป็นมาตรฐาน IEEE อย่างเป็นทางการ

2001 $\rightarrow$ IEEE 1364-2001: เพิ่มบล็อก generate, อาเรย์หลายมิติ และตัวดำเนินการใหม่

2005 $\rightarrow$ IEEE 1364-2005: ปรับปรุงความเข้ากันได้และการคอมไพล์

2005+ $\rightarrow$ SystemVerilog (IEEE 1800): ส่วนขยายสำหรับการออกแบบและตรวจสอบยืนยันขั้นสูง

- เปรียบเทียบ Verilog vs VHDL ([สไลด์ที่ 5](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L5)):

| ประเด็นเปรียบเทียบ | VHDL | Verilog |
|---|---|---|
| Origin | U.S. Department of Defense (1980s) | Gateway Design Automation (1984) |
| Syntax | Ada-like (เคร่งครัด, ละเอียด, เขียน | C-like (กระชับ, ยืดหยุ่น) |
| Typing | Strong typing (ตรวจสอบชนิดข้อมูลเข้ | Loose typing |
| Popular in | รัฐบาล, กองทัพ, การบินอวกาศ (ยุโรป) | อุตสาหกรรม ASIC/FPGA (สหรัฐฯ, เอเชี |
| Standard | IEEE 1076 | IEEE 1364 |

### 2.2 ธรรมชาติของ HDL และระดับชั้นการออกแบบ

- ความแตกต่างระหว่าง Software กับ Hardware Description ([สไลด์ที่ 6](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L6)):

Software: คำสั่งทำงานแบบเรียงลำดับทีละบรรทัด (Sequential execution) โดย CPU

Hardware (HDL): ทุกคำสั่งทำงาน ขนานกันพร้อมกันจริง (Concurrent) เพราะแต่ละบรรทัดคือโครงสร้างฮาร์ดแวร์บนชิป ตัวแปรคือสายสัญญาณ (Wire) หรือรีจิสเตอร์จริง และการ Compile หมายถึงการสังเคราะห์เป็นวงจรเกตลอจิก (Synthesis)

- ลำดับขั้นตอนการออกแบบดิจิทัล (Design Flow - [สไลด์ที่ 8](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L8)):

$$\text{1. Specification} \rightarrow \text{2. RTL Coding} \rightarrow \text{3. Simulation} \rightarrow \text{4. Synthesis} \rightarrow \text{5. Place \& Route} \rightarrow \text{6. Program/Fab}$$

- ระดับการจำลองและสังเคราะห์วงจร (RTL vs Testbench - [สไลด์ที 10](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L10)):

RTL (Synthesizable): โค้ดที่สามารถแปลงเป็นวงจรฮาร์ดแวร์จริงได้

Testbench (Simulation only): ใช้สำหรับทดสอบการทำงาน มีคำสั่งหน่วงเวลา (#10) หรือคำสั่งพิมพ์ค่า ($display) ซึ่งไม่สามารถสังเคราะห์เป็นฮาร์ดแวร์ได้

- รูปแบบการออกแบบวงจร (Modeling Styles - ตัวอย่าง MUX 2:1 บน [สไลด์ที 11](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L11)):

1. Behavioral: บรรยายเชิงพฤติกรรมโดยใช้ always @() และเงื่อนไข if-else

2. Dataflow: บรรยายการไหลของข้อมูลโดยใช้ assign y = sel ? b : a;

3. Structural: ต่อวงจรจากเกตพื้นฐาน เช่น not g1(...), and g2(...), or g4(...)

(ทั้งสามรูปแบบให้โครงสร้างวงจรฮาร์ดแวร์เดียวกันหลังการสังเคราะห์)

### 2.3 ขั้นตอนการออกแบบวงจรจากข้อกำหนด (Specification Case Study: Arbiter)

อ้างอิงจากสไลด์ที 12-16:

- Arbiter: วงจรจัดสรรและตัดสินสิทธิ์การเข้าใช้งานบัสส่วนกลางเมื่อมีอุปกรณ์ขอใช้งานพร้อมกัน
- การออกแบบ 4 ขั้นตอน:

1. Block Diagram: กำหนด Interface ภายนอก อินพุตคือ clk, rst, req0, req1 เอาต์พุตคือ gnt0, gnt1

2. Low Level Design: กำหนดเงื่อนไขลำดับความสำคัญ (Priority Scheme) ให้ req0 มีความสำคัญสูงสุด

ตรรกะคือ: gnt0 = req0 และ gnt1 = req1 & ~req0

3. Timing Diagram: แผนภาพเวลาแสดงสัญญาณนาฬิกา เมื่อ req0=1 และ req1=1 พร้อมกัน วงจรจะเลือกให้ gnt0=1

### 2.4 โมดูลและการต่อวงจรแบบลำดับชั้น (Module Instantiation)

อ้างอิงจากสไลด์ที 17-19:

- ทุกวงจรใน Verilog ต้องบรรจุอยู่ภายใน module ... endmodule
- การเรียกใช้โมดูลย่อย (Instantiation) แบ่งเป็น 2 รูปแบบ ([สไลด์ที 18](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L18)):

1. Named Port Mapping (แนะนำ): ระบุชื่อพอร์ตตรงๆ เช่น .clk(clk), .req0(req0) ลำดับก่อนหลังไม่มีผล ปลอดภัยและลดข้อผิดพลาด

2. Positional Mapping: เรียงลำดับสัญญาณตามตำแหน่งพอร์ต กระชับแต่ผิดพลาดได้ง่าย

### 2.5 ชนิดข้อมูล (Data Types) และความเข้าใจผิดเกี่ยวกับ `wire` vs `reg`

อ้างอิงจากสไลด์ที 20-26:

- ตารางสรุปชนิดข้อมูลใน Verilog ([สไลด์ที 20](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L20)):

| ชนิดข้อมูล (Data Type) | ความกว้างบิต | เครื่องหมาย | สังเคราะห์ได้หรือไม่ (Synthesizable) | ใช้หลัก |
|---|---|---|---|---|
| `wire` | ผู้ใช้กำหนด (ค่าเริ่มต้น 1) | Unsigned (4-state) | Yes | เชื่อมต่อวงจรฮาร์ดแวร์ (ใช้กับ `ass |
| `reg` | ผู้ใช้กำหนด (ค่าเริ่มต้น 1) | Unsigned (4-state) | Yes | เก็บค่าในบล็อก `always` / `initial` |
| `integer` | 32 bits | Signed | บางส่วน (เน้น Testbench) | ตัวนับลูป, คำนวณคณิตศาสตร์ |
| `time` | 64 bits | Unsigned | No (Simulation only) | เก็บเวลาจำลอง |
| `real` | 64 bits (IEEE 754) | Signed Float | No (Simulation only) | คำนวณทศนิยม, ความล่าช้า |
| `realtime` | 64 bits (IEEE 754) | Signed Float | No (Simulation only) | วัดเวลาความละเอียดสูง |

- ข้อเข้าใจผิดอันดับ 1 ของผู้เรียน Verilog ([สไลด์ที 22](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L22)):

reg ไม่ได้หมายถึง Hardware Register หรือ Flip-Flop เสมอไป

reg เป็นเพียงตัวแปรที่ถูกกำหนดค่าภายใน procedural block (always หรือ initial)

การที่ reg จะถูกสังเคราะห์เป็น Flip-Flop หรือ Combinational Logic ขึ้นอยู่กับ Sensitivity List ของ always block ล้วนๆ

กฎง่ายๆ: ใช้คำสั่ง assign $\rightarrow$ ซ้ายมือต้องเป็น wire, อยู่ใน always $\rightarrow$ ซ้ายมือต้องเป็น reg

- ระบบ 4 สถานะ (4-State Logic - [สไลด์ที 25](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L25)):

0: Logic low / Ground

1: Logic high / VCC

X: Unknown / Contention (เกิดจากการชนกันของสัญญาณ หรือไม่ได้รีเซ็ต)

Z: High impedance / Floating / Tri-state

- การเขียนตัวเลขคงที่ (Number Literals - [สไลด์ที่ 26](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L26)):

รูปแบบ: <size>'<base><value> เช่น 4'b1010 (ฐาน 2), 8'hA5 (ฐาน 16), 4'd10 (ฐาน 10), -4'sd3 (Signed)

**2.6 ตัวดำเนินการ (Operators) และลำดับความสำคัญ**

อ้างอิงจากสไลด์ที่ 27-32:

- ความแตกต่างระหว่าง Bitwise กับ Logical ([สไลด์ที่ 28](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L28)):

& (Bitwise AND): กระทำทีละบิต ผลลัพธ์มีความกว้างเท่ากับ operand เช่น 4'b1100 & 4'b1010 = 4'b1000

&& (Logical AND): พิจารณาทั้งค่าเป็นจริง/เท็จ ผลลัพธ์ได้ 1 บิตเสมอ เช่น 4'b1100 && 4'b1010 = 1'b1

- ตัวดำเนินการพิเศษในฮาร์ดแวร์ ([สไลด์ที่ 29-30](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L29)):

Reduction: &data (AND ทุกบิต), |data (OR ทุกบิต) ได้ผลลัพธ์ 1 บิต

Concatenation ({ }): ต่อสัญญาณ เช่น {a, b} เอา a เป็น MSB และ b เป็น LSB

Replication ({N{...}}): คัดลอกซ้ำ เช่น {4{1'b1}} = 4'b1111 มักใช้ทำ Sign Extension

Conditional (?:): สังเคราะห์ออกมาเป็น Multiplexer

- ลำดับความสำคัญ (Operator Precedence - [สไลด์ที่ 31](file:///d:/vhdl/lec/Week3_foundations%20and%20language%20basics.pdf#L31)):

$$!,\sim \ (\text{สูงที่สุด}) \rightarrow ,\% \rightarrow +,- \rightarrow \ll,\gg \rightarrow <,\le,>,\ge \rightarrow ==,!= \rightarrow \& \rightarrow \text{\textasciicircum} \rightarrow | \rightarrow \&\& \rightarrow || \rightarrow ?: \ (\text{ต่ำที่สุด})$$

---

### 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. Verilog ออกแบบมาเพื่อจำลองฮาร์ดแวร์ที่ทำงานขนานกัน: โค้ดแต่ละบรรทัดแทนโครงสร้างเกต สายสัญญาณ หรือฟลิปฟลอป

2. ความเคร่งครัดของ Data Types: พอร์ตเอาต์พุตที่ถูกขับด้วย assign ต้องประกาศเป็น wire ส่วนสัญญาณที่เขียนค่าใน always ต้องประกาศเป็น reg

3. การต่อบัสและป้องกัน Overflow: การนำสัญญาณหลายบิตมาบวกกันใน Verilog เช่น 4 บิตบวก 4 บิต หากไม่ขยายบิตล่วงหน้า จะเกิดปัญหาตัวทดหาย (Overflow) การใช้ Concatenation {1'b0, a} เข้ามาช่วยขยายขนาดบัสเป็น 5 บิตจึงเป็นหัวใจสำคัญ

---

### 4. จุดที่เอกสารไม่ได้ระบุ / ข้อสังเกตในเลกเชอร์

- ข้อจำกัดเรื่อง Testbench: ในสไลด์ที่ 10 ยกตัวอย่างโมดูล Testbench สั้นๆ สำหรับ and_gate แต่ในสไลด์ที่เหลือไม่ได้แสดงการเขียน Testbench สำหรับโมดูล Arbiter หรือวงจรคำนวณหลายบิต

---

## [LAB] ส่วนที่ 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 3

# [LAB] Week 3 - Lab 3: Foundations & Language Basics (BCD Adder on DE1)

แหล่งอ้างอิงหลัก: ไฟล์ [lab3.pdf](file:///d:/vhdl/lab/lab3.pdf) (หน้า 1-7)

ไฟล์เฉลยปฏิบัติการฉบับสมบูรณ์: [lab3_Answered.pdf](file:///d:/vhdl/lab/lab3_Answered.pdf) (จัดทำและกรอกคำตอบด้วย หมึกสีน้ำเงิน เรียบร้อยแล้ว)

---

### 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร)

อ้างอิงจากหน้า 1 และหน้า 4:

- วัตถุประสงค์เชิงพฤติกรรม (Learning Objectives):

1. ประกาศ Verilog module ด้วยไวยากรณ์ของพอร์ตและชนิดข้อมูลที่ถูกต้อง
2. สร้างวงจร Combinational Logic โดยใช้ wire, assign และตัวดำเนินการ Bitwise
3. ค้นหาและแก้ไขข้อผิดพลาดทางไวยากรณ์ (Syntax Errors) ที่พบบ่อยใน Verilog
4. ทำตามการสาธิตบอร์ด FPGA ร่วมกับอาจารย์ผู้สอน และบันทึกผลการทดลองจริงลงในตาราง

- ภารกิจหลัก (Mission Brief - BCD Adder on DE1):

ออกแบบวงจรบวกเลข BCD 1 หลัก ($0-9 + 0-9$) ผลรวมอยู่ในช่วง $0-18$ แสดงผลออกทาง 7-segment display บนบอร์ด DE1

อินพุต: สวิตช์ SW[3:0] = ตัวตัง A, SW[7:4] = ตัวบวก B
เอาต์พุต: HEX0 แสดงหลักหน่วย (Units), HEX1 แสดงหลักสิบ (Tens)

## 2. การเชื่อมโยงกับ Lecture ของ Week 3

| แนวคิดจาก Lecture Week 3 | สไลด์อ้างอิง | การนำมาใช้ใน Lab 3 |
|---|---|---|
| Module & Port Syntax | สไลด์ 17 | การประกาศพอร์ต `input wire` และ `ou |
| Bitwise Operators (`^`, `&`, ` `) | สไลด์ 27-28 | การสร้างวงจร Half Adder (`sum = a ^ |
| Case Sensitivity ใน Verilog | สไลด์ 17, 21 | การแก้บักคำว่า `EndModule` ให้เป็น |
| Concatenation (`{ }`) ป้องกัน Overf | สไลด์ 29-30 | การขยายขนาดโอเปอแรนด์จาก 4 บิตเป็น |
| Conditional Operator (`?:`) | สไลด์ 29, 32 | การสร้างเงื่อนไขตรวจสอบผลรวมเกิน 9 |
| Hierarchical Module Instantiation | สไลด์ 18 | การเชื่อมต่อโมดูล `binary_adder`, ` |

## 3. เฉลยและอธิบายโค้ด Verilog ทีละแบบฝึกหัด (ทุกคำตอบเป็นสีน้ำเงิน)

### ■ Exercise 1: Half Adder Module (เติมคำในช่องว่าง 5 จุด - หน้า 1-2)

พฤติกรรม: อินพุต 1 บิต 2 ตัว a, b และเอาต์พุต sum = a ^ b, carry = a & b

```
module half_adder (                              // 1. □□□□□□□□□□: half_adder
  input  wire a,                                 // 2. □□□□□□□□□□□□□□□□□□□□□□□: a
  input  wire b,                                 // 3. □□□□□□□□□□□□□□□□□□□□□□□□□□□: b
  output wire sum,
  output wire carry
);

  assign sum   = a ^ b ;                         // 4. □□□□□□□□□□□□□ XOR: a ^ b
  assign carry = a & b ;                         // 5. □□□□□□□□□□□□□ AND: a & b

endmodule
```

- เฉลยคำตอบลงในช่องว่าง:

ช่องที่ 1: half_adder
ช่องที่ 2: a
ช่องที่ 3: b
ช่องที่ 4: a และ b

ช่องที่ 5: a และ b

- การจำแนกวงจร: เป็น Combinational Logic ล้วน สังเคราะห์ลง LUT ในโหมดปกติ

### ■ Exercise 2: ค้นหาและแก้ไขข้อผิดพลาด 3 จุดใน Full Adder (หน้า 2)

โค้ดต้นฉบับที่มีข้อผิดพลาด:

```
module full_adder (
  input a, b, cin,
  output sum, carry_out
);
  wire p, g;
  assign p        = a ^ b;
  assign g        = a & b;
  assign sum      = p & cin;
  assign carry_out= g | (p & cin);
EndModule
```

- ตารางเฉลยข้อผิดพลาดทั้ง 3 จุด:

| ข้อผิดพลาดที่ (#) | ตำแหน่ง / บรรทัด | สาเหตุที่ผิด | โค้ดที่แก้ไขถูกต้อง |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 1 | <span style="color: #0056b3; font-w | ใช้ตั | <span style="color: #0056b3; font-w |
| 2 | <span style="color: #0056b3; font-w | ไวยาก | <span style="color: #0056b3; font-w |
| 3 | <span style="color: #0056b3; font-w | ขาดกา | <span style="color: #0056b3; font-w |

### ■ Exercise 3: จับคู่ตัวดำเนินการกับหน้าที (หน้า 3)

| Operator | Answer (คำตอบ) | Function (หน้าที่การทำงาน) | Letter |
|---|---|---|---|
| `^` | <span style="color: #0056b3; font-w | Logical AND - returns 1 single bit: | A |
| `&` | <span style="color: #0056b3; font-w | Bitwise XOR - operates on each bit | B |
| `&&` | <span style="color: #0056b3; font-w | Bitwise AND - operates on each bit | C |
| `\ ` | <span style="color: #0056b3; fon | Concatenation - joins bits into a w | D |
| `{ }` | <span style="color: #0056b3; font-w | Bitwise OR - operates on each bit i | E |

### ■ Exercise 4: เขียนโมดูล Full Adder จากข้อกำหนด (หน้า 3)

เขียนวงจรโดยใช้เฉพาะคำสั่ง assign (ห้ามใช้ Structural Instantiation):

```
module full_adder (
    input  wire a,    // Operand A (1 bit)
    input  wire b,    // Operand B (1 bit)
    input  wire cin,  // Carry-in (1 bit)
    output wire sum,  // □□□□□: a XOR b XOR cin
    output wire cout  // □□□□□□□□□: (a AND b) OR (cin AND (a XOR b))
);

    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));

endmodule
```

### ■ Exercise 5: โครงการรวม BCD Adder บนบอร์ด FPGA (หน้า 4-7)

##### 5A: โมดูลบวกเลขไบนารี binary_adder.v (หน้า 5)

- กฎการออกแบบ: อินพุต $A$ และ $B$ มีขนาด 4 บิต ผลรวมสูงสุดคือ $9+9=18$ ซึ่งในระบบฐานสองคือ 5'b10010 (ต้องการ 5 บิต) หากนำ 4 บิตมาบวกกันตรงๆ จะเกิด Overflow บิตที่ 4 จะหายไป จึงต้องใช้ Concatenation เติมบิต 0 ข้างหน้าเป็น 5 บิตก่อนบวก {1'b0, a}:

```
module binary_adder (
    input  wire [3:0] a,    // 4-bit BCD digit A (0-9)
    input  wire [3:0] b,    // 4-bit BCD digit B (0-9)
    output wire [4:0] sum   // 5-bit binary sum (0-18)
);

    // □□□□□□□□ overflow □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ 5 □□□□□□□□□□□□
    assign sum = {1'b0, a} + {1'b0, b};

endmodule
```

##### 5B: วงจรปรับแก้ค่า BCD bcd_correct.v (หน้า 6)

- หลักการ: หากผลบวก $\ge 10$ หลักสิบจะกลายเป็น 1 และหลักหน่วยคือ $\text{sum} - 10$

```
module bcd_correct (
  input  [4:0] sum,      // binary sum 0-18 from the adder
  output [3:0] units,    // units digit  0-9
  output [3:0] tens      // tens digit   0 or 1
);

  // □□□□□□□□□□□□□□□ 1: □□□□□□□□□□□□□□□□□□□ sum >= 10
  assign tens  = ( sum >= 5'd10 ) ? 4'd1 : 4'd0;

  // □□□□□□□□□□□□□□□ 2 □□□□ 3: □□□□□□□□□□□ 10 □□□□□□ sum >= 10
```

```
  assign units = ( sum >= 5'd10 ) ? (sum - 5'd10) : sum[3:0];

endmodule
```

- เฉลยค่าในช่องว่าง: ช่องว่างทั้ง 3 จุดเติมค่า 5'd10 (หรือ 10)

##### 5C: วงจรถอดรหัส 7-Segment seg7_decode.v (หน้า 6-7)

- เติมกรณีตัวเลข 4, 5, 6, 7 ตามรูปแบบตารางที่โจทย์กำหนดในหน้า 6:

เลข 4: แถบ b, c, f, g ติด $\rightarrow$ 7'b110_0110

เลข 5: แถบ a, c, d, f, g ติด $\rightarrow$ 7'b110_1101

เลข 6: แถบ a, c, d, e, f, g ติด $\rightarrow$ 7'b111_1101

เลข 7: แถบ a, b, c ติด $\rightarrow$ 7'b000_0111

```
    case (digit)
      4'd0: seg = 7'b011_1111;  // 0
      4'd1: seg = 7'b000_0110;  // 1
      4'd2: seg = 7'b101_1011;  // 2
      4'd3: seg = 7'b100_1111;  // 3
      4'd4: seg = 7'b110_0110;  // 4  (□□□□□□□□□□)
      4'd5: seg = 7'b110_1101;  // 5  (□□□□□□□□□□)
      4'd6: seg = 7'b111_1101;  // 6  (□□□□□□□□□□)
      4'd7: seg = 7'b000_0111;  // 7  (□□□□□□□□□□)
      4'd8: seg = 7'b111_1111;  // 8
      4'd9: seg = 7'b110_1111;  // 9
      default: seg = 7'b000_0000;
    endcase
```

##### โค้ดระดับบนสุด de1_top.v (โค้ดที่โจทย์ให้มาโดยห้ามแก้ไข - หน้า 4)

```
module de1_top (
  input [9:0] SW,
  output [6:0] HEX0, HEX1, HEX2, HEX3
);

  wire [4:0] sum;
  wire [3:0] units, tens;

  binary_adder u_add (.a(SW[3:0]), .b(SW[7:4]), .sum(sum));
  bcd_correct  u_bcd (.sum(sum),   .units(units), .tens(tens));
  seg7_decode  u_hex0(.digit(units),.seg(HEX0));
  seg7_decode  u_hex1(.digit(tens), .seg(HEX1));
  assign HEX2 = 7'h7F; assign HEX3 = 7'h7F;
endmodule
```

### ■ Exercise 6: ตารางบันทึกผลการทดลองบนบอร์ด DE1 (หน้า 7)

| อินพุตสวิตช์ ($A + B$) | การคำนวณ | ผลการแสดงผลที่สังเกตได้บนหน้าจอ DE1 |
|---|---|---|
| A=0, B=0 `(SW=0000 0000)` | $0 + 0 = 0$ | <span style="color: #0056b3; font-w |
| A=3, B=4 `(SW=0100 0011)` | $3 + 4 = 7$ | <span style="color: #0056b3; font-w |
| A=5, B=5 `(SW=0101 0101)` | $5 + 5 = 10$ | <span style="color: #0056b3; font-w |
| A=7, B=6 `(SW=0110 0111)` | $7 + 6 = 13$ | <span style="color: #0056b3; font-w |
| A=9, B=9 `(SW=1001 1001)` | $9 + 9 = 18$ | <span style="color: #0056b3; font-w |
| A=2, B=8 `(SW=1000 0010)` | $2 + 8 = 10$ | <span style="color: #0056b3; font-w |
| A=6, B=3 `(SW=0011 0110)` | $6 + 3 = 9$ | <span style="color: #0056b3; font-w |
| A=9, B=1 `(SW=0001 1001)` | $9 + 1 = 10$ | <span style="color: #0056b3; font-w |

### 4. ข้อควรระวังและจุดที่ขัดแย้งกันเองในเอกสาร (ตามกฎเหล็กข้อ 5)

1. ความขัดแย้งของหัวเรื่อง: ในหน้า 1 หัวข้อด้านบนสุดระบุว่า Lab 3 - Foundations & Language Basics แต่ในช่องกรอกชื่อด้านล่างกลับพิมพ์ว่า Lab 1 - Verilog Fundamentals & First FPGA Design

2. ความขัดแย้งในตาราง 7-Segment Decoder (หน้า 6):

ข้อความบรรยายระบุว่า "DE1 7-segment segments are active-low (0 = on, 1 = off)"

แต่ในตาราง Segment Truth Table ที่ให้มาเป็นตัวอย่าง กลับใช้ 1 = ติด (on) และ 0 = ดับ (off) เช่น เลข 0 ให้รหัส 7'b011_1111 (แถบ g เป็น 0 นอกนั้นเป็น 1) และเลข 8 ให้รหัส 7'b111_1111 (ทุกแถบเป็น 1) ซึ่งตรงข้ามกับนิยาม Active-Low มาตรฐานของฮาร์ดแวร์ แต่ในการทำแลปนี้ต้องตอบค่าตามแพทเทิร์นตารางที่เอกสารระบุไว้

## ☐ สรุปไฟล์แลปที่จัดทำเสร็จแล้ว

- ไฟล์เอกสารที่กรอกคำตอบด้วยหมึกสีน้ำเงินครบถ้วนทุกข้อ: [lab3_Answered.pdf](file:///d:/vhdl/lab/lab3_Answered.pdf)

ต้องการให้อาจารย์อธิบายต่อใน Week 4 (Lecture: Combinational Design & Lab 4) เลยหรือไม่ครับ?

# [WEEK] Week 4: Combinational Logic Design

## [LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

# [WEEK] Week 4 - Lecture: Combinational Design

แหล่งอ้างอิงหลัก: ไฟล์ [Week4_Combinational_Design.pdf](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf) (สไลด์ที่ 1-19)

### 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามที่เอกสารระบุ)

อ้างอิงจากสไลด์ที่ 1, 2 และ 19:

- หัวข้อหลัก: Combinational Design (Assign Statement, If-Else, Case Statement & Loop Statement)

- วัตถุประสงค์การเรียนรู้:

1. เข้าใจการทำงานของคำสั่ง assign (Continuous Assignment) สำหรับวงจร Combinational Logic และเข้าใจความแตกต่างเชิงลึกระหว่าง assign กับ always ([สไลด์ที่ 3-6](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L3))

2. ใช้งานโครงสร้างเลือกเงื่อนไข if-else ซึ่งสังเคราะห์เป็นวงจรจัดลำดับความสำคัญ (Priority Logic / Priority Mux) และรู้วิธีป้องกันการเกิด Inferred Latch ([สไลด์ที่ 7-9](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L7))

3. ใช้งานคำสั่ง case สำหรับวงจรที่ทำงานขนานเท่าเทียมกัน (Parallel Mux) เข้าใจข้อแตกต่างของ case, casez และ casex รวมถึงการนำไปสร้าง Arithmetic Logic Unit (ALU) ([สไลด์ที่ 10-12](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L10))

4. เข้าใจโครงสร้างวงวน (Loop Statements: while, for, repeat, forever) โดยเฉพาะพฤติกรรมของ for loop ในกระบวนการสังเคราะห์วงจร (Loop Unrolling กลายเป็นวงจรเกตขนาน) ([สไลด์ที่ 13-18](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L13))

### 2. แนวคิดหลัก (เรียงลำดับจากพื้นฐานไปสู่ระดับลึก)

**2.1 คำสั่ง `assign` (Continuous Assignment)**

- คุณลักษณะสำคัญ ([สไลด์ที่ 3](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L3)):

ใช้กับตัวแปรชนิด wire เท่านั้น (ห้ามใช้กับ reg)

ทำงานแบบต่อเนื่อง (Continuous): ทันทีที่ค่าสัญญาณทางขวามือเปลี่ยน เอาต์พุตทางซ้ายมือจะเปลี่ยนตามทันทีเปรียบเสมือนสายไฟจริง

วางอยู่นอกบล็อก procedural (always หรือ initial)

เหมาะแก่วงจร Combinational บรรทัดเดียวอย่างง่าย เช่น เกตลอจิก หรือ 2:1 Multiplexer

- ตารางเปรียบเทียบ assign vs always ([สไลด์ที 4](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L4)):

| ประเด็นเปรียบเทียบ | `always` | `assign` |
|---|---|---|
| ชนิดตัวแปรทางซ้ายมือ (LHS) | `reg` เท่านัน | `wire` เท่านัน |
| ประเภทลอจิกทีรองรับ | Combinational หรือ Sequential | Combinational แบบง่ายเท่านัน |
| จำนวนบรรทัดทีขับสัญญาณ | อัปเดตตัวแปรเดิมได้หลายเงือนไขในบล | 1 สัญญาณขับได้ 1 บรรทัด (หากซ้าจะเก |
| คำสังควบคุมทีรองรับ | รองรับ `if`, `case`, `for`, `while` | ไม่รองรับ (ใช้ได้เฉพาะ Conditional |
| ตำแหน่งในโค้ด | ต้องอยู่ภายในบล็อก `always` | อยู่นอกบล็อก procedural ใดๆ |

- ตัวอย่างเปรียบเทียบ MUX 2:1 ใน 2 รูปแบบ ([สไลด์ที 6](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L6)):

Style A (Dataflow): assign y = sel ? b : a;

Style B (Behavioral): always @() begin if (sel) y = b; else y = a; end

(ทังสองรูปแบบสังเคราะห์ออกมาเป็นวงจร MUX 2:1 ตัวเดียวกันทุกประการบนฮาร์ดแวร์)

### 2.2 โครงสร้างเงือนไข `if-else`

- ลักษณะการสังเคราะห์ ([สไลด์ที 7](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L7)):

เงือนไขแรกมีความสำคัญสูงสุดเสมอ จึงสังเคราะห์ออกมาเป็น Priority Encoder / Priority Mux

ใช้งานได้เฉพาะภายใน procedural block (always)

ข้อควรระวังสำคัญ: ต้องใส่สาขา else ให้ครอบคลุมทุกกรณีเสมอ หากไม่ใส่ ในกรณีทีไม่มีเงือนไขใดเป็นจริง วงจรจะต้องจดจำค่าเดิมไว้ ส่งผลให้โปรแกรมสังเคราะห์สร้าง Inferred Latch (แลตช์ทีไม่พึงประสงค์) ขึนมา ทำให้วงจรทำงานผิดพลาดและมีปัญหาเรือง Timing

### 2.3 โครงสร้างเลือกค่า `case` และการเปรียบเทียบกับ `if-else`

- ตารางเปรียบเทียบ if-else vs case ([สไลด์ที 8](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L8)):

| มิติการเปรียบเทียบ | `if-else` | `case` |
|---|---|---|
| ลำดับความสำคัญ (Priority) | มี: เงือนไขแรกสำคัญทีสุด | ไม่มี: ทุกกิงสาขาเท่าเทียมกัน |
| ฮาร์ดแวร์ทีได้ | Priority Mux (ทำงานช้าลงเมือเงือน | Parallel Mux (ทำงานเร็วกว่า) |
| ความเหมาะสมในการใช้งาน | เงือนไขซับซ้อน, การเปรียบเทียบช่วง | เลือกจากค่าคงทีแน่นอน เช่น Opcode, |
| ความเสียงเกิด Inferred Latch | เกิดเมือขาดสาขา `else` | เกิดเมือขาดสาขา `default` |

- ชนิดของคำสัง Case ([สไลด์ที 10](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L10)):

1. case: ตรวจสอบความตรงกันแบบสมบูรณ์ของทุกบิต (0, 1, X, Z)
2. casez: ยอมรับบิต ? หรือ z เป็นสภาวะ Don't-care (ไม่สนใจบิตนัน) เหมาะมากกับ Priority Decoder

3. casex: ยอมรับทัง x, z และ ? เป็น Don't-care (ไม่แนะนำในการออกแบบ RTL เพราะอาจซ่อนข้อผิดพลาดของสัญญาณ X)

- กรณีศึกษา: 8-bit Simple ALU ([สไลด์ที 12](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L12)):

อินพุต op[1:0] สลับ 4 ฟังก์ชัน: 2'b00 $\rightarrow$ ADD, 2'b01 $\rightarrow$ SUB, 2'b10 $\rightarrow$ AND, 2'b11 $\rightarrow$ OR

ใช้ assign zero = (result == 8'b0); เป็นแฟล็กตรวจจับศูนย์

### 2.4 วงวน (Loop Statements) และการคลายลูป (Loop Unrolling)

อ้างอิงจากสไลด์ที 13-18:

- ประเภทของ Loop ใน Verilog:

1. while: ทำงานซ้ำเมือเงือนไขเป็นจริง (ส่วนใหญ่ใช้เฉพาะใน Testbench)
2. for: ทำงานตามจำนวนรอบทีระบุแน่นอน (สังเคราะห์ได้ดี)
3. repeat: ทำงานซ้ำตามจำนวนครัง $N$ ทีกำหนดคงที
4. forever: วนซ้ำตลอดไป (Simulation only ใช้สร้าง Clock ใน Testbench)

- หลักการทำงานของ for loop ในการสังเคราะห์ฮาร์ดแวร์ ([สไลด์ที 15 และ 17](file:///d:/vhdl/lec/Week4_Combinational_Design.pdf#L15)):

ในฮาร์ดแวร์ไม่มีวงจรวนลูปตามเวลาจริง!

เครืองมือสังเคราะห์จะทำ Loop Unrolling (คลายลูป) โดยการก๊อบปีสร้างโครงข่ายเกตลอจิกขนานกันตามจำนวนรอบของลูป

จำนวนรอบของลูปต้องเป็นค่าคงทีในขันตอน Compile Time (เช่น ใช้ค่าคงที หรือ Parameter)

กรณีศึกษา Parity Generator: ลูปคำนวณ parity = parity ^ data[i]; จำนวน 8 รอบ จะถูกคลายออกมาเป็นโครงสร้างเกต XOR จำนวน 7 ตัวต่อกันเป็นแถวลูกโซ่ (data[0] ^ data[1] ^ ... ^ data[7])

### 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. การเลือกใช้คำสั่งควบคุม: ใช้ case เมื่อต้องการถอดรหัสแบบขนานที่ทุกสาขาเท่ากัน (เช่น ALU Opcode) และใช้ if-else เมื่อเงื่อนไขมีลำดับความสำคัญก่อนหลัง

2. การป้องกัน Inferred Latch: ในวงจร Combinational ทุกกรณีต้องมีค่าเอาต์พุตเสมอ - ต้องปิดท้ายด้วย else สำหรับ if และต้องมี default สำหรับ case

3. ธรรมชาติของ Loop ใน RTL: ลูป for มีไว้เพื่อเขียนโค้ดโครงสร้างวงจรซ้ำๆ ให้สั้นลง แต่เครื่องมือสังเคราะห์จะสร้างเกตฮาร์ดแวร์ขนานกันจริงเต็มจำนวนรอบ

## [LAB] ส่วนที่ 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 4

# [LAB] Week 4 - Lab 4: Combinational Logic (ALU & Decoders)

แหล่งอ้างอิงหลัก: ไฟล์ [lab 4.pdf](file:///d:/vhdl/lab/lab%204.pdf) (หน้า 1-9)

ไฟล์เฉลยปฏิบัติการฉบับสมบูรณ์: [lab4_Answered.pdf](file:///d:/vhdl/lab/lab4_Answered.pdf) (จัดทำและกรอกคำตอบด้วย หมึกสีน้ำเงิน ครบถ้วนทุกข้อ)

### 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร)

อ้างอิงจากหน้า 1 และหน้า 6:

- วัตถุประสงค์เชิงพฤติกรรม (Learning Objectives):

1. สร้างวงจร Multiplexer และ Decoder โดยใช้คำสั่ง assign และ case
2. ประยุกต์ใช้คำสั่ง if-else และ case ได้อย่างถูกต้องเพื่อป้องกัน Inferred Latches
3. วิเคราะห์และทำนายพฤติกรรมของวงจรเลือกค่าที่ใช้ case ได้โดยไม่ต้องพึ่งโปรแกรมจำลอง
4. ออกแบบวงจร Priority Encoder จากข้อกำหนดทางเทคนิค
5. พัฒนาและตรวจสอบความถูกต้องของวงจร 5-bit ALU บนบอร์ด DE1 ร่วมกับสวิตช์ ปุ่มกด และจอแสดงผล 7-Segment

- ภารกิจหลัก (5-bit ALU with 7-Segment Display):

อินพุตตัวเลข: SW[4:0] = ตัวตั้ง A, SW[9:5] = ตัวตั้ง B (ขนาด 5 บิต $0-31$)

อินพุตคำสั่ง: ปุ่มกด KEY[2:0] ทำงานร่วมกับโมดูล btn_toggle เพื่อสลับบิตคำสั่ง op[2:0] (8 ฟังก์ชัน) พร้อมแสดงผลสถานะที่ LEDG[2:0]

เอาต์พุต: แสดงผลลัพธ์เป็นเลขฐานสิบ 2 หลัก ผ่านวงจรปรับแก้ BCD ออกสู่หน้าจอ HEX1:HEX0

### 2. การเชื่อมโยงกับ Lecture ของ Week 4

| แนวคิดจาก Lecture Week 4 | สไลด์อ้างอิง | การนำมาใช้ใน Lab 4 |
|---|---|---|
| Case Statement & Default | สไลด์ 10 | การออกแบบ 4-to-1 MUX (Ex 1) และ 5-b |
| Inferred Latch Prevention | สไลด์ 7, 10 | การจับคู่ใน Ex 3 และการแก้ไขบัก 2- |
| If-Else Priority Logic | สไลด์ 7, 9 | การออกแบบ 4-to-2 Priority Encoder ใ |
| Data Types (`output reg`) | สไลด์ 4 | การแก้บักพอร์ตเอาต์พุตที่ถูกขับในบ |
| ALU as Parallel Multiplexer | สไลด์ 11, 12 | คำถามอธิบายโครงสร้าง ALU ในเชิงฮาร์ |

### 3. เฉลยและอธิบายโค้ด Verilog ทีละแบบฝึกหัด (ทุกคำตอบเป็นสีน้ำเงิน)

**■ Exercise 1: 4-to-1 Multiplexer (เติมคำในช่องว่าง 5 จุด - หน้า 2)**

```
module mux4to1 (
  input  wire in0, in1, in2, in3,
  input  wire [1:0] sel,            // 1. ■■■■■■■■■■■■■■■■■■: sel
  output reg  y                     // 2. ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ always: reg
);
  always @(*) begin
```

```
    case (sel)                      // 3. □□□□□□□□□□□□□□□□□□□□□: sel
      2'b00: y = in0;
      2'b01: y = in1;
      2'b10: y = in2;               // 4. □□□□□□□□□□□□□□□□□□□□□: y = in2
      2'b11: y = in3;
      default: y = 1'b0;            // 5. □□□ default □□□□□□□□□□□: 1'b0
    endcase
  end
endmodule
```

- เฉลยคำตอบในช่องว่าง:

ช่องที่ 1: sel

ช่องที่ 2: reg

ช่องที่ 3: sel

ช่องที่ 4: y = in2

ช่องที่ 5: 1'b0

---

### ■ Exercise 2: ค้นหาและแก้ไขข้อผิดพลาด 5 จุดใน 2-to-4 Decoder (หน้า 3)

โค้ดต้นฉบับที่มีข้อผิดพลาด:

```
module decoder2to4 (
  input  [1:0] sel,
  output [3:0] out                // □□□□□□ 1
);
  always @(sel[0]) begin          // □□□□□□ 2
    case (sel)
      2'd0: out = 4'b0001;
      2'd0: out = 4'b0010;        // □□□□□□ 3
      2'd2: out = 4'b0010;        // □□□□□□ 4
    endcase                       // □□□□□□ 5
  end
endmodule
```

- ตารางเฉลยข้อผิดพลาดทั้ง 5 จุด:

| ข้อผิดพลาดที่ (#) | ตำแหน่ง / บรรทัด | สาเหตุที่ผิด | โค้ดที่แก้ไขถูกต้อง |
|---|---|---|---|
| 1 | <span style="color: #0056b3; font-w | ขาดชน | <span style="color: #0056b3; font-w |
| 2 | <span style="color: #0056b3; font-w | Sensi | <span style="color: #0056b3; font-w |
| 3 | <span style="color: #0056b3; font-w | ป้ายก | <span style="color: #0056b3; font-w |
| 4 | <span style="color: #0056b3; font-w | ค่าคง | <span style="color: #0056b3; font-w |
| 5 | <span style="color: #0056b3; font-w | เคสไม | <span style="color: #0056b3; font-w |

---

### ■ Exercise 3: จับคู่โครงสร้างกับพฤติกรรมการทำงาน (หน้า 3)

| โครงสร้าง (Construct) | Answer (คำตอบ) | พฤติกรรมของวงจร (Behavior) | Letter |
|---|---|---|---|
| if with no final else | <span style="color: #0056b3; font-w | Parallel mux - every branch has equ | A |
| case with no default | <span style="color: #0056b3; font-w | Priority mux - the first true condi | B |
| if-else chain | <span style="color: #0056b3; font-w | Don't-care matching, useful for pri | C |
| casez | <span style="color: #0056b3; font-w | Risks an inferred latch in a case s | D |
| case (full) | <span style="color: #0056b3; font-w | Also risks an inferred latch, in an | E |

---

### ■ Exercise 4: ทำนายผลลัพธ์ของวงจร ALU จำลอง (หน้า 4)

กำหนดค่าอินพุตคงที่: $a = 4'\text{b0110} \ (6)$ และ $b = 4'\text{b0011} \ (3)$

| `op` | การทำงาน | Predicted Result (Binary) | Decimal (เลขฐานสิบ) |
|---|---|---|---|
| 000 | $a + b$ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 001 | $a - b$ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 010 | $a \ \& \ b$ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

| | | | | |
|---|---|---|---|---|
| 011 | $a \ \ | \ b$ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 101 | ไม่อยู่ในเคส $\rightarrow$ default | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | |

### ■ Exercise 5: ตอบคำถามสั้น - เหตุใด ALU จึงเปรียบเสมือน Multiplexer ขนาดใหญ่? (หน้า 4)

คำตอบ:

(a) ในโครงสร้างฮาร์ดแวร์จริง แต่ละบล็อกฟังก์ชันย่อย (เช่น วงจรบวก วงจรลบ เกต AND, OR) จะทำการประมวลผลอินพุต A และ B ขนานพร้อมกันตลอดเวลาโดยไม่มีการหยุดพัก<br>

(b) สัญญาณ Opcode ไม่ได้ทำหน้าที่สั่งเปิด/ปิดการคำนวณ แต่ทำหน้าที่เป็น Select Line คอยเลือกว่าจะนำผลลัพธ์จากบล็อกฟังก์ชันใดส่งผ่านออกไปยังเอาต์พุตเพียงค่าเดียว<br>

(c) โครงสร้างการทำงานเช่นนี้ตรงกับนิยามทางกายภาพของ Multiplexer ขนานขนาดใหญ่ ที่รวบรวมเอาต์พุตของวงจรคำนวณทั้งหมดมาไว้ที่พอร์ตอินพุตของ MUX แล้วใช้ Opcode เป็นตัวเลือกสัญญาณออกสู่ภายนอก

### ■ Exercise 6: ออกแบบโมดูล Priority Encoder 4-to-2 จากข้อกำหนด (หน้า 5)

ข้อกำหนด: ใช้เฉพาะ always ร่วมกับ if-else (ห้ามใช้ case), ลำดับความสำคัญ $in[3] > in[2] > in[1] > in[0]$

```
module priority_encoder (
    input  wire [3:0] in,    // 4 request lines (in[3] □□□□□□□□□□□□□□□□□□□□□□□□□□□□□)
    output reg        val,   // □□□□□ 1 □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
    output reg  [1:0] code   // □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
);

    always @(*) begin
        if (in[3]) begin
            val  = 1'b1;
            code = 2'b11; // in[3] □□□
        end else if (in[2]) begin
            val  = 1'b1;
            code = 2'b10; // in[2] □□□
        end else if (in[1]) begin
            val  = 1'b1;
            code = 2'b01; // in[1] □□□
        end else if (in[0]) begin
            val  = 1'b1;
            code = 2'b00; // in[0] □□□
        end else begin
            val  = 1'b0;  // □□□□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□
            code = 2'b00;
        end
    end

endmodule
```

### ■ Exercise 7: ออกแบบวงจร ALU 5 บิต และการแปลงรหัส BCD (หน้า 7-8)

##### 7A: โมดูล alu.v (หน้า 8)

- สังเคราะห์เป็น Combinational ล้วน รองรับ 8 ฟังก์ชัน และขยายบิตเป็น 6 บิตเพื่อรองรับตัวทดของการบวก

```
module alu (
    input  wire [4:0] a,       // □□□□□□□□ A (5 □□□)
    input  wire [4:0] b,       // □□□□□□□□ B (5 □□□)
    input  wire [2:0] op,      // □□□□□□□□□□□□ 3 □□□
    output reg  [5:0] result   // □□□□□□□□□□□□□ 6 □□□
);

    always @(*) begin
```

```
        case (op)
            3'b000: result = {1'b0, a} + {1'b0, b}; // ADD (□□□□□□□)
            3'b001: result = {1'b0, a} - {1'b0, b}; // SUB (□□□□□□)
            3'b010: result = {1'b0, a & b};         // AND
            3'b011: result = {1'b0, a | b};         // OR
            3'b100: result = {1'b0, a ^ b};         // XOR
            3'b101: result = {1'b0, ~a};            // NOT A (□□□□□□□□ A)
            3'b110: result = a << 1;                // SHL (□□□□□□□□□□□ 1 □□□ □□□□□□□□□ 6 □□□)
            3'b111: result = {1'b0, a >> 1};        // SHR (□□□□□□□□□□ 1 □□□)
            default: result = 6'd0;                 // □□□□□□□□ Inferred Latch
        endcase
    end

endmodule
```

##### 7B: โมดูลปรับแก้ BCD bcd_correct.v (เติมค่าในช่องว่าง - หน้า 8)

```
module bcd_correct (
  input  [5:0] value,  // □□□□□□□□□□□□□□□ 0-63
  output [3:0] tens,   // □□□□□□□□ 0-6
  output [3:0] units   // □□□□□□□□□□ 0-9
);
  assign tens  = value / 10;  // 1. □□□□□□□□□□□□□□□□□□□□□□□□□□□□□: /
  assign units = value % 10;  // 2. □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□: %
endmodule
```

- เฉลยคำตอบในช่องว่าง:

ช่องที่ 1: /

ช่องที่ 2: %

---

**■ Exercise 8: ตารางบันทึกผลการทดสอบบนบอร์ด DE1 (หน้า 8-9)**

กำหนดค่าอินพุต: $A = 5'\text{b00101} \ (5)$ และ $B = 5'\text{b00011} \ (3)$ (ยกเว้นแถวสุดท้าย $A=31$)

| `Op` | คำสั่ง | การคำนวณ | ผลลัพธ์ฐานสิบที่แสดงบน `HEX1:HEX0` |
|---|---|---|---|
| 000 | ADD | $5 + 3 = 8$ | <span style="color: #0056b3; font-w |
| 001 | SUB | $5 - 3 = 2$ | <span style="color: #0056b3; font-w |
| 010 | AND | $5'\text{b00101} \ \& \ 5'\text{b00 | <span style="color: #0056b3; font-w |
| 011 | OR | $5'\text{b00101} \ \ \ 5'\text{b00011} = 5'\text{b0011 | <span style="color: #0056b3; font-w |
| 100 | XOR | $5'\text{b00101} \ \text{\textascii | <span style="color: #0056b3; font-w |
| 101 | NOT A | $\sim 5'\text{b00101} = 5'\text{b11 | <span style="color: #0056b3; font-w |
| 110 | SHL | $5'\text{b00101} \ll 1 = 6'\text{b0 | <span style="color: #0056b3; font-w |
| 111 | SHR | $A = 5'\text{b11111} \ (31) \gg 1 = | <span style="color: #0056b3; font-w |

---

## 4. ข้อควรระวังและประเด็นทางฮาร์ดแวร์ที่สำคัญ

- การขยายขนาดบิต (Bit-width Extension): ในการทำคำสั่งบวก (ADD) ระหว่างตัวเลข 5 บิตสองตัว หากไม่ต่อเติมบิต 0 ข้างหน้าเป็น 6 บิต {1'b0, a} เมื่อเกิดกรณี $31 + 31 = 62$ บิตตัวทดที่ 5 จะล้นหาย (Overflow) ส่งผลให้ BCD Decoder ถอดรหัสผิดพลาด

- การสลับบิตคำสั่งด้วยปุ่มกด (btn_toggle): วงจร btn_toggle ใช้ขอบขาลงของปุ่มกด (negedge key_n) เป็นสัญญาณกระตุ้นการสลับค่า $q \Leftarrow \sim q$ โดยไม่ต้องพึ่ง System Clock ซึ่งในทางปฏิบัติบนบอร์ดจริงอาจพบอาการ Contact Bouncing ได้ แต่ในแบบฝึกหัดนี้ถือว่าโมดูลทีให้มาทำงานในอุดมคติ

---

## ☐ สรุปไฟล์แลปที่จัดทำเสร็จแล้ว

- ไฟล์เอกสาร Lab 4 ที่กรอกเฉลยด้วยหมึกสีน้ำเงินครบถ้วนทุกหน้า: [lab4_Answered.pdf](file:///d:/vhdl/lab/lab4_Answered.pdf)

---

ต้องการให้อาจารย์อธิบายต่อใน Week 5 (Lecture: Sequential Logic & Lab 5) เลยหรือไม่ครับ?

# [WEEK] Week 5: Sequential Logic Design

## [LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

# [WEEK] Week 5 - Lecture: Sequential Logic (Flip-Flops & Registers)

แหล่งอ้างอิงหลัก: ไฟล์ [Week5_sequential_logic.pdf](file:///d:/vhdl/lec/Week5_sequential_logic.pdf) (สไลด์ที่ 1-38)

### 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามที่เอกสารระบุ)

อ้างอิงจากสไลด์ที่ 1, 2 และ 38:

- หัวข้อหลัก: Sequential Logic (Flip-Flops & Registers: Circuit Elements, Flip-Flop Types, D Flip-Flop, Registers, Initialization, Reset, Blocking vs Non-blocking, Combinational + FF, Shift Register)

- วัตถุประสงค์การเรียนรู้:

1. เข้าใจความแตกต่างระหว่าง Combinational Logic กับ Sequential Logic ที่มีหน่วยความจำ (Memory) และรู้จัก 3 องค์ประกอบหลักของวงจร Sequential ([สไลด์ที่ 3-4](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L3))

2. รู้จักฟลิปฟลอปประเภทต่างๆ (SR, JK, T, D) และเข้าใจเหตุผลทางวิศวกรรมว่าเหตุใดชิป FPGA จึงสร้างขึ้นจาก D Flip-Flop เท่านั้น ([สไลด์ที่ 5-10](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L5))

3. เขียนโค้ด D Flip-Flop และสร้างรีจิสเตอร์ขนาด $N$ บิต (Parameterized Register) พร้อมทั้งกำหนดค่าเริ่มต้นและระบบรีเซ็ต (Synchronous vs Asynchronous Reset) ได้อย่างถูกต้อง ([สไลด์ที่ 11-17](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L11))

4. เข้าใจความแตกต่างเชิงลึกและกฎเหล็กระหว่าง Blocking Assignment (=) กับ Non-blocking Assignment (<=) เพื่อป้องกัน Race Condition ([สไลด์ที่ 18-21](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L18))

5. ออกแบบวงจรผสมระหว่าง Combinational กับ Flip-Flop, เข้าใจโครงสร้าง Shift Register และโครงสร้างวงจรแบบหลายสเตจ (Pipelined Multi-Stage Circuits) ([สไลด์ที่ 22-33](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L22))

### 2. แนวคิดหลัก (เรียงลำดับจากพื้นฐานไปสู่ระดับลึก)

#### 2.1 นิยามของ Sequential Logic และ 3 องค์ประกอบหลัก

- นิยาม ([สไลด์ที่ 3](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L3)):

Combinational Logic: เอาต์พุตขึ้นอยู่กับอินพุตปัจจุบันเท่านั้น (ไม่มีประวัติในอดีต ไม่มีหน่วยความจำ)

Sequential Logic: เอาต์พุตขึ้นอยู่กับอินพุตปัจจุบัน ร่วมกับสถานะที่บันทึกไว้ในอดีต (Stored State) ซึ่งถูกจัดเก็บอยู่ในฟลิปฟลอป (Flip-Flop)

- องค์ประกอบ 3 ส่วนของวงจร Sequential ([สไลด์ที่ 4](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L4)):

1. Next-State Logic: วงจร Combinational ที่คำนวณว่าสถานะถัดไปควรเป็นค่าใด โดยนำสถานะปัจจุบันและอินพุตมาประมวลผล

2. State Register: กลุ่มของฟลิปฟลอปที่ทำหน้าที่เก็บสถานะปัจจุบัน (เป็น ที่เดียวที่มีหน่วยความจำ ในระบบ)

3. Output Logic: วงจร Combinational ที่แปลงค่าสถานะปัจจุบัน (และอาจรวมถึงอินพุต) ให้ออกมาเป็นสัญญาณเอาต์พุตภายนอก

Feedback Path: เส้นทางสัญญาณป้อนกลับที่นำเอาต์พุตจาก State Register วนลูปกลับไปเป็นอินพุตของ Next-State Logic

#### 2.2 ชนิดของฟลิปฟลอป และเหตุผลที่เลือกใช้ D Flip-Flop บน FPGA

- ตารางเปรียบเทียบชนิดของ Flip-Flop ([สไลด์ที่ 9](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L9)):

| ชนิด (Type) | สัญญาณอินพุต | สมการสถานะถัดไป ($Q^+$) | มีการสลับค่า (Toggle)? | มีสภาวะไม่พึงประสงค์ (Invalid State) | ความเหมาะสมในการใช้งาน |
|---|---|---|---|---|---|
| SR | $S, R$ | $Q^+ = S + R'Q$ | ไม่มี | มี เมื่อ $S=R=1$ | วงจร Latch ดั้งเดิม |

| | | | | | |
|---|---|---|---|---|---|
| D | $D$ | $Q^+ = D$ | ไม่มี | ไม่มี | สากล (Universal) สำหรับ Verilog / F |
| JK | $J, K$ | $Q^+ = JQ' + K'Q$ | มี (เมื่อ $J=K=1$) | ไม่มี | วงจรนับในตำราทฤษฎี |
| T | $T$ | $Q^+ = T \oplus Q$ | มี (เมื่อ $T=1$) | ไม่มี | โครงสร้างพื้นฐานของวงจรนับ |

- เหตุผลที Verilog และ FPGA ใช้ D Flip-Flop ล้วน ([สไลด์ที 10](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L10)):

1. โครงสร้างฮาร์ดแวร์จริงของชิป: สถาปัตยกรรม Logic Element (LE) ของ Cyclone III บนบอร์ด DE0 ถูกสร้างขึนจาก D Flip-Flop เพียงชนิดเดียว ภายในชิปไม่มีฮาร์ดแวร์ SR, JK หรือ T primitive อยู่จริง

2. ความตรงไปตรงมา: คำสัง always @(posedge clk) แมปเข้ากับพิน D และ Clock ของ D-FF ได้ตรงทีสุดโดยไม่มีความกำกวม และไม่มีสภาวะ Invalid State

### 2.3 การเขียนโค้ด D Flip-Flop และการสร้าง Register

- โค้ดมาตรฐานของ D-FF (dff.v - [สไลด์ที 8](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L8)):

```
module dff (
    input clk, rst_n, d,
    output reg q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= 1'b0; // Asynchronous reset □□□ active-low
        else
            q <= d;    // □□□□□□□□□□□□ D □□□□ Q □□□□□□□□□□□□□□□□□□□□□□□ clk
    end
endmodule
```

- การสร้าง Register ขนาด $N$ บิต ([สไลด์ที 11, 14-15](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L11)):

Register ขนาด 8 บิต คือการนำ D Flip-Flop 8 ตัวมาต่อขนานกันโดยแชร์สัญญาณ Clock และ Reset ร่วมกัน

นิยมใช้ parameter W = 8 เพือให้โมดูลสามารถปรับเปลียนขนาดความกว้างบิตได้ตามต้องการ

### 2.4 การรีเซ็ตและการจัดลำดับความสำคัญ (Reset & Priority)

อ้างอิงจากสไลด์ที 12-17:

- Asynchronous Reset: ตอบสนองทันทีทีสัญญาณรีเซ็ตเปลียนระดับ โดยไม่ต้องรอขอบสัญญาณนาฬิกา (ใส่ใน Sensitivity List: always @(posedge clk or negedge rst_n)) นิยมใช้เป็นหลักในหลักสูตรนี
- Synchronous Reset: ทำการรีเซ็ตเฉพาะเมือเกิดขอบขาขึนของ Clock เท่านัน (always @(posedge clk))
- กฎลำดับความสำคัญ (Priority Rule): ภายในบล็อก always ต้องตรวจสอบเงือนไข if (!rst_n) เป็นลำดับแรกสุดเสมอ จากนันจึงตรวจสอบ else if (en) ห้ามสลับเอา Enable มาก่อน Reset เด็ดขาด

### 2.5 กฎเหล็ก: Blocking (`=`) vs Non-blocking (`<=`)

อ้างอิงจากสไลด์ที 18-21:

- Blocking Assignment (=): ประเมินค่าและเขียนทับตัวแปรทันทีทีละบรรทัด (เหมือนโปรแกรมภาษา C) ใช้สำหรับ Combinational Logic ล้วน
- Non-blocking Assignment (<=): ประเมินค่าทางขวามือทังหมดพร้อมกัน แล้วอัปเดตตัวแปรทางซ้ายมือพร้อมกันทีขอบสัญญาณนาฬิกา เป็นกฎเหล็กสำหรับ Sequential Logic (posedge clk)
- อันตรายจากการใช้ = ใน Sequential Logic ([สไลด์ที 20](file:///d:/vhdl/lec/Week5_sequential_logic.pdf#L20)):

หากเขียน q1 = d; q2 = q1; q3 = q2; ด้วย Blocking สัญญาณ d จะไหลทะลุผ่านทุกสเตจไปจนถึง q3 ภายใน Clock Cycle เดียว ทำให้ไม่เกิดการหน่วงเวลาแบบ Shift Register และเกิด Race Condition ในการจำลองวงจร

### 2.6 โครงสร้างวงจรแบบท่อลำเลียง (Multi-Stage Pipelined Circuits)

อ้างอิงจากสไลด์ที 26-33:

- เป็นการวางรีจิสเตอร์ (Pipeline Registers) คันระหว่างวงจร Combinational แต่ละช่วง (เช่น Stage 1 $\rightarrow$ Reg 1 $\rightarrow$ Stage 2 $\rightarrow$ Reg 2) เพือตัดทอน Critical Path ทำให้ระบบสามารถทำงานทีความถีสัญญาณนาฬิกาสูงขึนได้ ซึงเป็นหลักการเดียวกับทีใช้ในสถาปัตยกรรม RISC-V Pipeline

### 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. วงจร Sequential มีสถานะจดจำได้: เกิดจากการใช้ State Register (D Flip-Flops) ทำงานร่วมกับ Next-State Logic และ Feedback Path

2. D Flip-Flop คือหัวใจของ FPGA: ชิปฮาร์ดแวร์จริงบรรจุเฉพาะ D-FF

3. ใช้ Non-blocking (<=) กับ Sequential เสมอ: เพื่อให้ทุกฟลิปฟลอปทำงานพร้อมกันอย่างแท้จริงทีขอบสัญญาณนาฬิกา

4. ลำดับการรีเซ็ต: ต้องให้ความสำคัญสูงสุดกับการรีเซ็ต (if (!rst_n)) ก่อนเงื่อนไขการทำงานอื่น

---

## [LAB] ส่วนที่ 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 5

# [LAB] Week 5 - Lab 5: Sequential Logic: Flip-Flops & Registers

แหล่งอ้างอิงหลัก: ไฟล์ [lab 5.pdf](file:///d:/vhdl/lab/lab%205.pdf) (หน้า 1-8)

ไฟล์เฉลยปฏิบัติการฉบับสมบูรณ์: [lab5_Answered.pdf](file:///d:/vhdl/lab/lab5_Answered.pdf) (จัดทำและกรอกคำตอบด้วย หมึกสีน้ำเงิน ครบถ้วนทุกข้อ)

---

### 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร)

อ้างอิงจากหน้า 1 และหน้า 5:

- วัตถุประสงค์เชิงพฤติกรรม (Learning Objectives):

1. บรรยายองค์ประกอบ 3 ส่วนของวงจร Sequential: Next-state logic, State register และ Output logic

2. อธิบายเหตุผลที D flip-flop เป็นฟลิปฟลอปชนิดเดียวทีถูกสร้างบนฮาร์ดแวร์ FPGA
3. เขียนโค้ด D flip-flop และ Parameterized N-bit register พร้อมระบบรีเซ็ตทีถูกต้อง
4. แยกแยะความแตกต่างระหว่าง Blocking (=) กับ Non-blocking (<=) และหลีกเลียงข้อผิดพลาดทีเกิดขึน

5. ใช้งาน Verilog Parameter เพื่อนำโมดูลกลับมาใช้ซ้ำ (เช่น ความกว้างบิต หรือค่าคีย์ประจำรอบ)
6. ไล่ค่าสัญญาณ (Trace) ในวงจร Sequential แบบหลายสเตจ (Pipelined Circuit) และต่อเชือมไฟล์โมดูลย่อยเข้าด้วยกัน

7. พัฒนาและตรวจสอบวงจร 8-bit Shift Register with Enable และวงจร Mini Block Cipher 2 รอบ บนบอร์ด DE0

- ภารกิจหลัก (Mini Block Cipher 2-Stage Pipeline):

สร้างวงจรเข้ารหัสข้อมูลบล็อกขนาดเล็ก 8 บิต จำนวน 2 รอบ (คล้ายสถาปัตยกรรม AES)

แต่ละรอบประกอบด้วย 3 ฟังก์ชัน Combinational: SubNibbles (แทนค่าผ่าน S-box), Permute (สลับตำแหน่ง Nibble บน-ล่าง) และ AddRoundKey (XOR กับ Round Key)

ใช้รีจิสเตอร์คันระหว่างรอบ (Pipeline Stage) และแสดงผลลัพธ์ Ciphertext ฐาน 16 บนหน้าจอ HEX1:HEX0

---

### 2. การเชือมโยงกับ Lecture ของ Week 5

| แนวคิดจาก Lecture Week 5 | สไลด์อ้างอิง | การนำมาใช้ใน Lab 5 |
|---|---|---|
| Sequential Circuit Elements | สไลด์ 4 | การจับคู่ค่าศัพท์ใน Exercise 1 (Next |
| Reset Priority & Active-Low | สไลด์ 16, 36 | การแก้บักใน Exercise 2 และการเขียน |
| Blocking vs Non-blocking | สไลด์ 18-21 | การทำนายผลลัพธ์ใน Exercise 4 (ตาราง |
| Shift Register with Enable | สไลด์ 24-25, 34 | การทำนายค่า `q_reg` เมือกดปุ่มใน E |
| Parameterized Register | สไลด์ 14, 30 | การออกแบบโมดูล `register.v` ด้วย `p |
| Multi-Stage Pipelining | สไลด์ 26-33 | การต่อวงจรเข้ารหัส 2 รอบใน `mini_ci |

---

### 3. เฉลยและอธิบายโค้ด Verilog ทีละแบบฝึกหัด (ทุกคำตอบเป็นสีน้ำเงิน)

**■ Exercise 1: องค์ประกอบของวงจร Sequential (หน้า 1)**

| ข้อที (#) | Match (A-E) | คำอธิบาย (Description) |
|---|---|---|
| 1 | <span style="color: #0056b3; font-w | Combinational logic that computes w |

| | | |
|---|---|---|
| 2 | <span style="color: #0056b3; font-w | The bank of flip-flops that stores |
| 3 | <span style="color: #0056b3; font-w | The connection where the state regi |
| 4 | <span style="color: #0056b3; font-w | Combinational logic that turns the |
| 5 | <span style="color: #0056b3; font-w | The periodic signal whose edge is t |

### ■ Exercise 2: ค้นหาและแก้ไขข้อผิดพลาด 5 จุดในโมดูล `dff_reg` และ `Top` (หน้า 2)

โค้ดต้นฉบับที่มีข้อผิดพลาด:

```
 1  module dff_reg (
 2    input        clk, rst_n, en       // ??????? 1: Syntax error (??????????)
 3    input  [7:0] d,
 4    output reg [7:0] q
 5  );
 6
 7    always @(posedge clk, rst_n) begin // ??????? 2: Sensitivity list error
 8      if (rst_n)                       // ??????? 3: Reset polarity error
 9        q <= 8'b0;
10      else if (en)
11        q = d;                         // ??????? 4: Blocking assignment error
12    end
13  endmodule
...
23    dff_reg u1 (.clk(clk), .rst_n(rst_n), .en(1'b1), .d(data_in), .Q(q_wire)); // ??????? 5
```

- ตารางเฉลยข้อผิดพลาดทั้ง 5 จุด:

| ข้อผิดพลาดที่ (#) | ตำแหน่ง / บรรทัด | สาเหตุที่ผิด | โค้ดที่แก้ไขถูกต้อง |
|---|---|---|---|
| 1 | <span style="color: #0056b3; font-w | Synta | <span style="color: #0056b3; font-w |
| 2 | <span style="color: #0056b3; font-w | Sensi | <span style="color: #0056b3; font-w |
| 3 | <span style="color: #0056b3; font-w | Reset | <span style="color: #0056b3; font-w |
| 4 | <span style="color: #0056b3; font-w | Block | <span style="color: #0056b3; font-w |
| 5 | <span style="color: #0056b3; font-w | Wirin | <span style="color: #0056b3; font-w |

### ■ Exercise 3 / 4: เปรียบเทียบผลลัพธ์ Blocking vs Non-blocking (หน้า 3-4)

กำหนดสภาวะเริ่มต้น: rst_n = 1, en = 1, d = 1, และ q1 = q2 = q3 = q4 = 0 ก่อนเกิด Clock Edge แรก

| สัญญาณ | `confusing_reg` (Blocking `=`) - หล | `correct_reg` (Non-blocking `<=`) | `correct_reg` (Non-blocking `<=`) - |
|---|---|---|---|
| `q1` | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| `q2` | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| `q3` | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| `q4` | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| `q_out` | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

- บทวิเคราะห์: ใน confusing_reg เมื่อใช้ = คำสั่งจะทำงานตามลำดับทันที ส่งผลให้ค่า d=1 ไหลทะลุผ่าน q1 $\rightarrow$ q2 $\rightarrow$ q3 $\rightarrow$ q4 กลายเป็น 1 ทั้งหมดตั้งแต่ Edge แรก ส่วน correct_reg ที่ใช้ <= ข้อมูลจะค่อยๆ เลื่อนไปทีละ 1 สเตจต่อ 1 ขอบสัญญาณนาฬิกาอย่างถูกต้องตามพฤติกรรมฮาร์ดแวร์

### ■ Exercise 5: ไล่ค่าสัญญาณวงจร 8-bit Shift Register (หน้า 4)

คำสั่งเลื่อนบิต: q_reg <= {q_reg[6:0], serial_in}; (เลื่อนซ้าย โดยบิตใหม่เข้าที่ LSB ทางขวา)

เริ่มต้นหลังกดปุ่ม Reset: q_reg = 0000 0000

| การกดปุ่ม `KEY[1]` ครั้งที่ | ค่า `SW[0]` (`serial_in`) | ค่า `q_reg` หลังการกดปุ่ม (ไบนารี 8 | ค่าฐาน 16 (Hex) |
|---|---|---|---|
| 1 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 2 | 0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 3 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 4 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

| | | | |
|---|---|---|---|
| 5 | 0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

### ■ Exercise 6 & 7: วงจรเข้ารหัสบล็อก 2 รอบ (Mini Block Cipher - หน้า 6-7)

##### โค้ดเดิมค่าใน round_function.v (หน้า 6)

```
  assign sub_hi      = sbox(data_in[7:4]);                   // SubNibbles, hi nibble
  assign sub_lo      = sbox(data_in[3:0]);                   // SubNibbles, lo nibble (□□□□
  assign substituted = {sub_hi, sub_lo};
  assign permuted    = {substituted[3:0], substituted[7:4]};  // Permute: □□□□□□□□□□□□ Nibble
  assign data_out    = permuted ^ ROUND_KEY;                 // AddRoundKey: XOR □□□ Round
```

##### 7A: โมดูลบันทึกอินพุต input_reg.v (เติมค่าในช่องว่าง 3 จุด - หน้า 6)

```
module input_reg (
  input        clk, rst_n,
  input  [7:0] sw,
  output reg [7:0] data_in_reg
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) data_in_reg <= 8'b0;
    else        data_in_reg <= sw;
endmodule
```

- เฉลยคำตอบในช่องว่าง:

ช่องที่ 1: posedge

ช่องที่ 2: 8'b0

ช่องที่ 3: <=

##### 7B: โมดูลรีจิสเตอร์แบบพารามิเตอร์ register.v (เติมค่าในช่องว่าง 3 จุด - หน้า 7)

```
module register #(parameter W = 8) (
  input             clk, rst_n,
  input  [W-1:0]    d,
  output reg [W-1:0] q
);
  always @(posedge clk or negedge rst_n)
    if (!rst_n) q <= {W{1'b0}};
    else        q <= d;
endmodule
```

- เฉลยคำตอบในช่องว่าง:

ช่องที่ 1: !rst_n

ช่องที่ 2: {W{1'b0}} (หรือ 0)

ช่องที่ 3: d

##### 7D: การต่อวงจรระดับบนสุด mini_cipher_top.v (หน้า 7)

```
module mini_cipher_top (
  input        CLOCK_50,
  input  [7:0] SW,
  input        KEY0,        // rst_n
  output [6:0] HEX1, HEX0
);
  wire clk = CLOCK_50, rst_n = KEY0;
  wire [7:0] data_in_reg, round1_c, reg1_q, round2_c, reg2_q;

  // □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□ Clock Domain
  input_reg u_in (.clk(clk), .rst_n(rst_n), .sw(SW), .data_in_reg(data_in_reg));

  // Round 1: □□□□□□□□□□□□□□□□□□□□□ 1 (Key = 0xA5) + Pipeline Register 1
  round_function #(.ROUND_KEY(8'hA5)) u_r1 (.data_in(data_in_reg), .data_out(round1_c));
  register #(.W(8)) u_reg1 (.clk(clk), .rst_n(rst_n), .d(round1_c), .q(reg1_q));

  // Round 2: □□□□□□□□□□□□□□□□□□□□□ 2 (Key = 0x3C) + Pipeline Register 2
  round_function #(.ROUND_KEY(8'h3C)) u_r2 (.data_in(reg1_q), .data_out(round2_c));
  register #(.W(8)) u_reg2 (.clk(clk), .rst_n(rst_n), .d(round2_c), .q(reg2_q));
```

```
  // □□□□□□□□□□ Ciphertext □□□□□□□□□□□□□ 7-Segment (□□□□□□□□□□□□□□□□□□□□□□□□□)
  hex7seg u_h1 (.value(reg2_q[7:4]), .seg(HEX1)); // Hi nibble -> HEX1
  hex7seg u_h0 (.value(reg2_q[3:0]), .seg(HEX0)); // Lo nibble -> HEX0

endmodule
```

**■ Exercise 8: ตารางบันทึกผลการทดสอบวงจรเข้ารหัสบนบอร์ด DE0 (หน้า 8)**

| สวิตช์ `SW[7:0]` (Plaintext) | ผลลัพธ์หลังผ่าน Round 1 (`reg1_q`) | ผลลัพธ์ Ciphertext ที่แสดงบน `HEX1: |
|---|---|---|
| `11000011` (`0xC3`) | `0xB0` (ตามตัวอย่างในเอกสาร) | <span style="color: #0056b3; font-w |
| `00000000` (`0x00`) | `0x4B` | <span style="color: #0056b3; font-w |
| `11111111` (`0xFF`) | `0xD2` | <span style="color: #0056b3; font-w |
| `01011101` (`0x5D`) | `0x3A` | <span style="color: #0056b3; font-w |
| `10100101` (`0xA5`) | `0x53` | <span style="color: #0056b3; font-w |

### 4. ข้อควรระวังในการทำแลป (ตามเอกสารสไลด์ที่ 36)

1. ห้ามผสม = และ <= ในบล็อก always เดียวกัน: ในบล็อก Sequential ให้ใช้ <= เท่านั้น

2. การจัดลำดับเงื่อนไข Reset: ต้องตรวจเงื่อนไข if (!rst_n) ก่อน else if (en) เสมอ เพื่อให้การรีเซ็ตมีผลเหนือกว่าสภาวะ Enable

3. การกำหนดค่าเริ่มต้นในตัวแปร: การเขียน reg [7:0] q = 8'b0; หรือการใช้บล็อก initial อาจทำงานได้ในการจำลอง (Simulation) แต่ไม่สามารถใช้แทนวงจรฮาร์ดแวร์รีเซ็ตจริงบนบอร์ด FPGA ได้

## ☐ สรุปไฟล์แลปที่จัดทำเสร็จแล้ว

- ไฟล์เอกสาร Lab 5 ที่กรอกเฉลยด้วยหมึกสีน้ำเงินครบถ้วนทุกหน้า: [lab5_Answered.pdf](file:///d:/vhdl/lab/lab5_Answered.pdf)

ต้องการให้อาจารย์อธิบายต่อใน Week 6 (Lecture: Counters & Shift Registers & Lab 6) เลยหรือไม่ครับ?

# [WEEK] Week 6: Counters & Shift Registers

## [LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

# [WEEK] Week 6 - Lecture: Counters & Shift Registers

แหล่งอ้างอิงหลัก: [Week6_Counters_ShiftRegisters.pdf](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf) (สไลด์ 1-30)

### 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามที่เอกสารระบุ)

อ้างอิงจาก [สไลด์ 1-2, 30](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=1):

- หัวข้อหลัก: Counters & Shift Registers: จาก Counter สู่ Serial Communication (Up/Down, Mod-N, Clock & PLL, PWM, Timer, SISO/SIPO/PISO/PIPO, UART/SPI, LFSR)
- วัตถุประสงค์และ Agenda 5 ส่วน:

1. Counters: ทำความเข้าใจ Up/Down Counter, สัญญาณ Enable, Mod-N Counter, Ring Counter และ Johnson Counter เพื่อเป็นรากฐานของ Timer และ Clock Divider

2. Clock & PLL: เข้าใจคุณสมบัติของ Clock, ปัญหา Clock Jitter, Clock Skew, ทฤษฎี Phase-Locked Loop (PLL) และการคำนวณอัตราทดความถี่ $f_{out} = f_{in} \times \frac{N}{M \times K}$

3. Counter Applications: ประยุกต์ใช้ Counter ในงานจริง ได้แก่ Timer, Watchdog, Pulse Width Modulation (PWM), Frequency Meter, Address Generator และ Gray Code Counter

4. Shift Registers: โครงสร้างและการทำงานของ SISO, SIPO, PISO, PIPO และ Universal Shift Register

5. Serial I/O & LFSR: การแปลง Parallel $\leftrightarrow$ Serial สำหรับ UART/SPI, Linear Feedback Shift Register (LFSR) สำหรับสร้างเลขสุ่มเทียม/CRC และ Barrel Shifter

---

## 2. แนวคิดหลัก (เรียงจากพื้นฐานไปสู่ระดับสูง)

### 2.1 นิยามและคำศัพท์สำคัญ (Definitions & Key Terminology)

- Counter ([สไลด์ 3](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=3)): วงจร Sequential ที่เปลี่ยนค่าตามขอบสัญญาณ Clock เกิดจาก Register + Combinational Logic (Incrementer หรือ Decrementer) โดยทุก positive edge ของ Clock ค่าใน Register จะถูกแทนที่ด้วยค่าถัดไป ($next = current + 1$ สำหรับ Up Counter หรือ $next = current - 1$ สำหรับ Down Counter)

- Mod-N Counter ([สไลด์ 4, 8, 10](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=4)): Counter ที่นับได้ทั้งหมด $N$ สถานะ (States) ที่แตกต่างกันแล้ววนกลับ (Wrap) โดย Binary Counter ขนาด $N$-bit ปกติจะมี $2^N$ สถานะ (Mod-$2^N$) ส่วน Counter ที่ต้องการจำนวนสถานะ $N$ ใดๆ จะต้องใช้จำนวนบิตอย่างน้อย $\lceil \log_2 N \rceil$ บิต

- Clock Jitter vs Clock Skew ([สไลด์ 2, 11, 15](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=15)):

Clock Jitter (ความสั่นทางเวลา): ขอบ Clock มาไม่ตรงเวลาในแต่ละรอบ (Temporal variation จาก Cycle สู่ Cycle) ทำให้ Timing Margin ลดลง

Clock Skew (ความต่างเชิงพื้นที่): สัญญาณ Clock เดินทางไปถึง Flip-Flop แต่ละตัวไม่พร้อมกัน อันเนื่องมาจากความยาวสายและ Propagation Delay บนชิป

- Phase-Locked Loop (PLL) ([สไลด์ 12-15](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=12)): วงจรสังเคราะห์ความถี่ฮาร์ดแวร์แบบ Dedicated Block บนชิป FPGA ที่สามารถคูณ (Multiply) และหาร (Divide) ความถี่ได้อย่างแม่นยำ พร้อมทั้งให้ค่า Jitter ต่ำ และเชื่อมต่อเข้ากับ Dedicated Global Clock Network

- Shift Register ([สไลด์ 18, 20](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=20)): Flip-Flop ที่นำมาต่ออนุกรมกันเป็นแถว โดย Output ของตัวหน้าต่อเข้าเป็น Input ของตัวถัดไป ทำหน้าที่ "ย้าย" ข้อมูลที่มีอยู่ไปตำแหน่งข้างๆ ทุกๆ ขอบขาขึ้นของ Clock

- SISO / SIPO / PISO / PIPO ([สไลด์ 21-22](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=21)):

SISO (Serial-In Serial-Out): เข้าทีละบิต ออกทีละบิต (ใช้เป็น Delay Line หรือ Buffer)

SIPO (Serial-In Parallel-Out): เข้าทีละบิต ออกพร้อมกันทุกบิต (ใช้ในภาครับ UART RX, ชิป 74HC595)

PISO (Parallel-In Serial-Out): เข้าพร้อมกันทุกบิต ออกทีละบิต (ใช้ในภาคส่ง UART TX, SPI Master)

PIPO (Parallel-In Parallel-Out): เข้าพร้อมกัน ออกพร้อมกัน (คือ Register ทั่วไปที่เรียนในสัปดาห์ที่ 5)

- Ring Counter vs Johnson Counter ([สไลด์ 8, 10](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=8)):

Ring Counter (Circular Shift): นำบิตบนสุด (MSB) วนกลับมาป้อน LSB วงจร $N$-bit จะมี $N$ สถานะ (เช่น 4-bit มี 4 สถานะ: $1000 \rightarrow 0100 \rightarrow 0010 \rightarrow 0001 \rightarrow 1000$)

Johnson Counter (Inverted Feedback): นำค่ากลับด้านของบิตบนสุด ($\sim\text{MSB}$) วนกลับมาป้อน LSB วงจร $N$-bit จะมี $2N$ สถานะ (เช่น 4-bit มี 8 สถานะ: $0000 \rightarrow 1000 \rightarrow 1100 \rightarrow 1110 \rightarrow 1111 \rightarrow 0111 \rightarrow 0011 \rightarrow 0001 \rightarrow 0000$)

- Linear Feedback Shift Register (LFSR) ([สไลด์ 26](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=26)): Shift Register ที่ต่อสายแบบ XOR ย้อนกลับจากตำแหน่ง Taps ที่กำหนดเพื่อสร้างลำดับตัวเลขสุ่มเทียม (Pseudo-Random) ที่มีสถานะสูงสุด $2^N - 1$ (ไม่ผ่านสถานะ 0 ทั้งหมด)

---

### 2.2 วงจร โค้ดตัวอย่าง และสูตรการคำนวณในเอกสาร

##### (1) วงจร Counter พื้นฐานใน Verilog HDL

- 4-bit Up Counter พร้อม Synchronous Reset ([สไลด์ 5](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=5)):

```
module counter_up (
  input  wire       clk,
  input  wire       rst_n,
  output reg  [3:0] count
);
```

```
  always @(posedge clk) begin
    if (!rst_n)
      count <= 4'b0000;
    else
      count <= count + 1; // □□□□□ 1111 + 1 □□ wrap □□□□□□□□ 0000 □□□□□□□□□
  end
endmodule
```

- Counter พร้อมสัญญาณ Enable และ Up/Down ([สไลด์ 6-7](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=6)):

ลำดับ Priority ใน always block คือ rst_n $\rightarrow$ enable $\rightarrow$ dir:

```
module counter_updown (
  input  wire       clk, rst_n,
  input  wire       dir,    // 1=up, 0=down
  input  wire       enable,
  output reg  [3:0] count
);
  always @(posedge clk) begin
    if (!rst_n)
      count <= 4'b0;
    else if (enable) begin
      if (dir)
        count <= count + 1;
      else
        count <= count - 1;
    end
  end
endmodule
```

##### (2) Clock Divider จาก 50 MHz สู่ 1 Hz ([สไลด์ 11](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=11))

- สูตรคำนวณ: Toggle ครึ่งคาบทุกๆ $\text{Toggle Count} = \frac{f_{clk}}{2 \times f_{target}}$ รอบ

- คำนวณ 1 Hz จาก 50 MHz:

$$\frac{50,000,000\text{ Hz}}{2 \times 1\text{ Hz}} = 25,000,000\text{ clock cycles}$$

เคาน์เตอร์เริ่มนับจาก $0$ ถึง $24,999,999$ (ใช้ขนาด 25 บิต เพราะ $2^{24}-1 = 16.7\text{M} < 25\text{M} < 2^{25} = 33.5\text{M}$)

```
module clk_divider (
  input  wire clk_50m,
  input  wire rst_n,
  output reg  clk_1hz
);
  reg [24:0] divider;
  always @(posedge clk_50m) begin
    if (!rst_n) begin
      divider <= 0;
      clk_1hz <= 0;
    end else if (divider == 24_999_999) begin
      divider <= 0;
      clk_1hz <= ~clk_1hz;
    end else
      divider <= divider + 1;
  end
endmodule
```

##### (3) ทฤษฎีและการคำนวณ Phase-Locked Loop (PLL) ([สไลด์ 12-13](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=12))

- สูตรเต็มของ PLL:

$$f_{out} = \frac{f_{in} \times N}{M \times K}$$

โดยที่:

$M$ = Input clock divider

$N$ = Feedback multiplier

$K$ = Output post-divider

$f_{VCO} = \frac{f_{in} \times N}{M}$ = ความถี Voltage-Controlled Oscillator

- เงื่อนไขทางฮาร์ดแวร์ (Cyclone III บน Altera DE0):

$f_{VCO}$ ต้องอยู่ในช่วง $300\text{ MHz} \le f_{VCO} \le 600\text{ MHz}$

- ขันตอนการคำนวณ (Worked Example จากสไลด์ 12):

1. หาอัตราส่วน $\frac{f_{out}}{f_{in}}$ เป็นเศษส่วนอย่างต่ำ

2. ตรวจสอบว่า $f_{VCO}$ อยู่ในช่วง $300\text{-}600\text{ MHz}$ หรือไม่

3. หากต่ำกว่า $300\text{ MHz}$ ให้ขยายตัวคูณ $N$ ขึน พร้อมกำหนดตัวหาร $K$ ให้ $f_{out} = \frac{f_{VCO}}{K}$ ตรงตามเป้าหมาย

##### (4) Pulse Width Modulation (PWM) Generator ([สไลด์ 18](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=18))

- โครงสร้างใช้ Counter + Comparator:

$$\text{Duty Cycle (\%)} = \frac{\text{duty}}{2^N} \times 100\%, \quad f_{PWM} = \frac{f_{clk}}{2^N}$$

- โค้ด Verilog 8-bit PWM:

```
module pwm (
  input  wire       clk, rst_n,
  input  wire [7:0] duty,
  output wire       pwm_out
);
  reg [7:0] cnt;
  always @(posedge clk) begin
    if (!rst_n)
      cnt <= 0;
    else
      cnt <= cnt + 1; // free-run wrap
  end
  assign pwm_out = (cnt < duty);
endmodule
```

---

### 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. Counter คือรากฐานของ Digital Timing: Counter ตัวเดียวกันเมือเปลียนเงือนไข wrap และจุดต่อ output สามารถทำหน้าทีเป็น Timer, Watchdog, Clock Divider, PWM Generator, ROM Address Generator หรือ State Register ของ FSM ได้ ([สไลด์ 3, 16](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=16))

2. Clock Divider ด้วย Counter vs PLL: Counter Divider ทำได้เฉพาะการ "หาร" ความถีลงมา และสัญญาณ output เป็น Gated Clock ทีมี Jitter สูง ส่วน PLL สามารถทำได้ทังการ "คูณและหาร" ความถี ได้สัญญาณ Jitter ต่ำ และเชือมต่อกับ Dedicated Global Clock Network ([สไลด์ 15](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=15))

3. Shift Register คือสะพานเชือม Parallel และ Serial: สัญญาณความเร็วสูงและระบบสือสารในปัจจุบัน (UART, SPI, I2C, SERDES, JTAG) ใช้ Shift Register เป็นตัวแปลงข้อมูลแบบขนานกับอนุกรมทังหมด โดยฝังส่งใช้ PISO และฝังรับใช้ SIPO ([สไลด์ 21, 25](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=21))

4. Ring vs Johnson vs LFSR: Ring Counter ถอดรหัสง่ายสุดแต่เปลือง Flip-Flop ($N$ สถานะ), Johnson Counter ได้สถานะ $2N$ และไม่มี Glitch ขณะถอดรหัส, ส่วน LFSR ให้ลำดับเลขสุ่มเทียมยาวถึง $2^N - 1$ สถานะ โดยใช้ Logic เพียง XOR gates ([สไลด์ 8, 26](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=8))

---

### 4. จุดทีเอกสารไม่ได้ระบุ / ข้อขัดแย้งใน Lecture

- Barrel Shifter: ในสไลด์หน้า 2 (Agenda) และสไลด์หน้า 30 (Summary) มีการระบุหัวข้อ Barrel Shifter แต่ภายในสไลด์เนือหาไม่มีสไลด์ทีอธิบายวงจรหรือโค้ดของ Barrel Shifter โดยเฉพาะ ([Week6_Counters_ShiftRegisters.pdf](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf))
- ช่วงค่า VCO: สไลด์ 12-13 กำหนดชัดเจนว่า Cyclone III VCO อยู่ในช่วง $300\text{-}600\text{ MHz}$ แต่ไม่ได้กล่าวถึงค่าความคลาดเคลือนของ Phase Error หรือ Lock Time แบบเป็นตัวเลขเจาะจง

---

## [LAB] ส่วนที 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 6

# [LAB] Week 6 - Lab 6: Counters, Clock/PLL & Shift Registers

แหล่งอ้างอิงหลัก: [Lab6.pdf](file:///d:/vhdl/lab/Lab6.pdf) (หน้า 1-7)

ไฟล์เฉลยแลปฉบับสมบูรณ์: [Lab6_Answered.pdf](file:///d:/vhdl/lab/Lab6_Answered.pdf) (สร้างและเติมคำตอบตัวอักษรสีน้ำเงินลงในไฟล์ PDF เรียบร้อยแล้ว)

## 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร)

อ้างอิงจาก [Lab6.pdf หน้า 1, 3-4](file:///d:/vhdl/lab/Lab6.pdf#page=1):

- วัตถุประสงค์:

1. ฝึกทักษะการวิเคราะห์ข้อสอบความรู้พื้นฐานเรื่อง Counter, PLL และ Shift Register (True/False & Debugging)

2. ฝึกคำนวณพารามิเตอร์ $M, N, K$ และ $f_{VCO}$ ของ ALTPLL บน FPGA Cyclone III

3. ออกแบบและต่อประสานระบบดิจิทัลแบบ Multi-Clock Domain โดยใช้ ALTPLL ตัวเดียวสร้างความถี่ 3 ย่านพร้อมกัน ($100\text{ MHz}, 20\text{ MHz}, 5\text{ MHz}$) จาก CLOCK_50

4. นำสัญญาณนาฬิกาแต่ละความถี่ไปควบคุมโมดูลย่อย:

pwm.v บนโดเมน $100\text{ MHz}$ เพื่อหรี่ไฟ LEDG[9] 16 ระดับ

tick_gen.v และ ring_counter.v บนโดเมน $20\text{ MHz}$ เพื่อสร้างไฟวิ่ง LEDG[7:0] ปรับทิศทางและความเร็วได้

heartbeat.v บนโดเมน $5\text{ MHz}$ เพื่อกระพริบ LEDG[8] จังหวะ 1 Hz แสดงสถานะการทำงานของระบบ

## 2. เชื่อมโยงกับ Lecture

- Exercise 1 (True/False) & Exercise 2 (Up/Down Counter): โยงโดยตรงกับ [สไลด์ 3-7](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=3) เรื่องนิยาม Counter, จำนวนสถานะ $2^N$, ขนาดบิต $\lceil \log_2 N \rceil$, Clock Jitter/Skew, และโครงสร้าง always @(posedge clk) พร้อม enable

- Exercise 3 (Debug Clock Divider): นำทฤษฎี Clock Divider จาก [สไลด์ 11](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=11) มาจับข้อผิดพลาดเรื่อง reg/wire, ขนาดบิตของตัวนับ $25\text{M}$, การใช้ Non-blocking (<=), และจำนวนรอบนับ $N-1$

- Exercise 4 (PLL Calculation): นำสูตร $f_{out} = \frac{f_{in} \times N}{M \times K}$ และข้อจำกัด VCO ช่วง $300\text{-}600\text{ MHz}$ จาก [สไลด์ 12-13](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=12) มาใช้คำนวณหาค่า $M, N, K, f_{VCO}$

- Exercise 5 (Shift Register & Concatenation): นำนิยาม SISO, SIPO, PISO, PIPO และการเลื่อนบิตด้วย Concatenation Operator {} จาก [สไลด์ 20-22, 25](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=20) มาจับคู่การใช้งานจริง

- Exercise 9 (System Project): นำโมดูล PWM ([สไลด์ 18](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=18)), Ring Counter ([สไลด์ 8](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=8)), Clock Divider/Tick Generator ([สไลด์ 11](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=11)), และ ALTPLL Megafunction ([สไลด์ 14](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=14)) มาประกอบเป็นระบบเดียวกัน

## 3. เฉลยและอธิบายเนื้อหา Lab 6 ทีละส่วน (คำตอบแสดงด้วยสีน้ำเงิน)

**[หมายเหตุ] Exercise 1 - True or False (หน้า 1)**

| No | ข้อความ | T/F | ถ้า F $\rightarrow$ เขียนคำตอบที่ถู |
|---|---|---|---|
| 1 | Counter คือ register ที่เพิ่ม next- | <span style="color: #0056b3; font-w | (ถูกต้องตาม [สไลด์ 3](file:///d:/vh |
| 2 | 4-bit binary up counter นับได้ 32 ค | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 3 | Mod-10 counter ต้องใช้อย่างน้อย 5 b | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 4 | Clock skew คือขอบ clock ที่มาไม่ตรง | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 5 | Clock divider ที่ใช้ counter toggle | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 6 | PLL สามารถทั้งคูณและหารความถี่ cloc | <span style="color: #0056b3; font-w | (ถูกต้องตาม [สไลด์ 15](file:///d:/v |
| 7 | PWM generator สร้างได้จาก counter ต | <span style="color: #0056b3; font-w | (ถูกต้องตาม [สไลด์ 18](file:///d:/v |
| 8 | SIPO ใช้ในฝั่งส่งข้อมูลของ UART (UA | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 9 | Ring counter แบบ 4-bit มี 4 states | <span style="color: #0056b3; font-w | (ถูกต้อง: Ring มี $N$ states, Johns |

| | | | |
|---|---|---|---|
| 10 | Barrel shifter เลื่อนข้อมูลได้หลายต | <span style="color: #0056b3; font-w | (ถูกต้องตามทฤษฎีและ [สไลด์ 2, 30](f |

### [หมายเหตุ] Exercise 2 - Complete the Up/Down Counter (หน้า 1-2)

โค้ดทีเติมสมบูรณ์ในช่องว่าง (1)-(6):

```
module counter_updown (
  input  wire       clk, rst_n,
  input  wire       dir,
  input  wire       enable,
  output reg [3:0] count          // □□□□□□□□□ (1) □□□□ reg
);

always @(posedge clk) begin       // □□□□□□□□□ (2) □□□□ posedge
  if (!rst_n)                     // □□□□□□□□□ (3) □□□□ !rst_n
    count <= 4'b0;
  else if (enable)                // □□□□□□□□□ (4) □□□□ enable
    if (dir)
      count <= count + 1'b1;      // □□□□□□□□□ (5) □□□□ 1'b1 (□□□□ 1)
    else
      count <= count - 1'b1;      // □□□□□□□□□ (6) □□□□ 1'b1 (□□□□ 1)
end

endmodule
```

ตารางเฉลยในหน้า 2:

| ช่อง | คำตอบ | เพราะอะไร? |
|---|---|---|
| (1) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| (2) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| (3) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| (4) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| (5) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| (6) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

### [หมายเหตุ] Exercise 3 - Find the 4 Bugs in Clock Divider (หน้า 2)

โค้ดต้นฉบับที่มีจุดผิด:

```
1   module clk_divider (
2    input  wire  clk_50m, rst_n,
3    output wire  clk_1hz
4   );
5   reg [23:0] divider;
6   always @(posedge clk_50m) begin
7     if (!rst_n) begin
8       divider = 0;
9         clk_1hz = 0;
10      end else if (divider == 25_000_000) begin   // BUG 4
11        divider <= 0;
12        clk_1hz <= ~clk_1hz;
13      end else
14        divider <= divider + 1;
15    end
16  endmodule
```

ตารางวิเคราะห์และแก้ไข Bug ทั้ง 4 จุด:

| Bug | บรรทัด | ผิดอย่างไร | แก้เป็น |
|---|---|---|---|
| 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 2 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 3 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 4 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

### [หมายเหตุ] Exercise 4 - PLL Calculation (หน้า 2-3)

สูตร: $f_{out} = \frac{f_{in} \times N}{M \times K}, \quad f_{VCO} = \frac{f_{in} \times N}{M} \quad (300\text{ MHz} \le f_{VCO} \le 600\text{ MHz})$

| $f_{in}$ | $f_{out}$ ที่ต้องการ | $M$ | $N$ | $f_{VCO}$ | $K$ | $f_{out}$ จริง |
|---|---|---|---|---|---|---|
| 50 MHz | 100 MHz | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 50 MHz | 25 MHz | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 50 MHz | 75 MHz | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 50 MHz | 40 MHz | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 50 MHz | 70 MHz (XGA) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

(หมายเหตุ: แถว 40 MHz สามารถใช้ชุด $M=5, N=32, f_{VCO}=320\text{ MHz}, K=8$ ตามที่ปรากฏใน [สไลด์ 13](file:///d:/vhdl/lec/Week6_Counters_ShiftRegisters.pdf#page=13) ได้เช่นเดียวกัน)

---

### [หมายเหตุ] Exercise 5 - Shift Register Types & Code (หน้า 3)

(a) จับคู่ประเภท Shift Register กับการใช้งาน:

| การใช้งาน | ประเภท | เหตุผล |
|---|---|---|
| UART TX - ส่งข้อมูล 8 bit ออกทางสาย | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| ชิป 74HC595 - ใช้ 3 ขา MCU คุม LED | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| Delay line - หน่วงสัญญาณ 4 clock cy | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| Register ธรรมดาที่โหลดและอ่านพร้อมก | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

(b) การเลื่อนบิตด้วย Concatenation Operator {}:

```
// Shift LEFT: bit ■■■■■■■■■■■■■ LSB, bit ■■■■■■■■■■■■■■■
q <= { q[6:0] , ser_in };       // ■■■■■■■■■■ (1) ■■■■■ q[6:0]

// Shift RIGHT: bit ■■■■■■■■■■■■■ MSB, bit ■■■■■■■■■■■■■■■■■
q <= { ser_in , q[7:1] };       // ■■■■■■■■■■ (2) ■■■■■ q[7:1]

// Rotate LEFT: bit ■■■■■■■■■■■■■■■■■■■ LSB
q <= { q[6:0] , q[7] };         // ■■■■■■■■■■ (3) ■■■■■ q[7]

// Ring counter 4-bit: 1000 -> 0100 -> 0010 -> 0001 -> 1000 (Shift Right with Wrap)
q <= { q[0] , q[3:1] };         // ■■■■■■■■■■ (4) ■■■■■ q[0] , q[3:1]
```

ตารางสรุปคำตอบ:

| ช่อง | คำตอบ |
|---|---|
| (1) | <span style="color: #0056b3; font-w |
| (2) | <span style="color: #0056b3; font-w |
| (3) | <span style="color: #0056b3; font-w |
| (4) | <span style="color: #0056b3; font-w |

---

### [หมายเหตุ] Exercise 9 - Multi-Clock Domain Design & Verification (หน้า 3-7)

##### ไฟล์ 1: my_pll.v

สร้างจาก ALTPLL IP Catalog Wizard (Input: CLOCK_50 = 50 MHz):

- c0: 100 MHz ($M=1, N=6, K=3$)
- c1: 20 MHz ($M=5, N=10, K=5$ หรือ $M=1, N=6, K=15$)
- c2: 5 MHz ($M=1, N=6, K=60$)
- Output: locked signal

##### ไฟล์ 2: pwm.v (Clock Domain c0: 100 MHz)

```
module pwm (
  input  wire       clk,
  input  wire       rst_n,
  input  wire [3:0] level,
  output wire       pwm_out
```

```
);
  reg [3:0] cnt;
  always @(posedge clk) begin
    if (!rst_n)
      cnt <= 4'd0;
    else
      cnt <= cnt + 1'b1;
  end
  assign pwm_out = (cnt < level);
endmodule
```

##### ไฟล์ 3: tick_gen.v (Clock Domain c1: 20 MHz)

**สร้างสัญญาณ Tick พัลส์ขนาด 1 clock cycle:**

- $f_{clk} = 20\text{ MHz} = 20,000,000\text{ Hz}$
- **ทีความเร็ว 2 Hz (speed = 0):** $20,000,000 / 2 = 10,000,000$ **รอบ** $\rightarrow CNT_SLOW = 9,999,999$

- **ทีความเร็ว 10 Hz (speed = 1):** $20,000,000 / 10 = 2,000,000$ **รอบ** $\rightarrow CNT_FAST = 1,999,999$

```
module tick_gen #(
  parameter integer CNT_SLOW = 9_999_999, // 20MHz / 2Hz - 1
  parameter integer CNT_FAST = 1_999_999  // 20MHz / 10Hz - 1
)(
  input  wire  clk,
  input  wire  rst_n,
  input  wire  speed,
  output wire  tick
);
  reg [23:0] cnt;

  wire [23:0] limit = speed ? CNT_FAST : CNT_SLOW;

  always @(posedge clk)
    if (!rst_n)
       cnt <= 24'd0;
    else if (cnt >= limit)
       cnt <= 24'd0;
    else
       cnt <= cnt + 1'b1;

  assign tick = (cnt == limit);
endmodule
```

##### ไฟล์ 4: ring_counter.v (Clock Domain c1: 20 MHz)

```
module ring_counter (
  input  wire       clk, rst_n,
  input  wire       tick,
  input  wire       dir,
  input  wire       pause_n,
  output reg  [7:0] led
);
  always @(posedge clk) begin
    if (!rst_n)
      led <= 8'b1000_0000;
    else if (!pause_n)
      led <= led;
    else if (tick) begin
      if (dir) led <= { led[6:0] , led[7] };   // □□□□□□□□ Rotate Left
      else     led <= { led[0] , led[7:1] };   // □□□□□□□□ Rotate Right
    end
  end
endmodule
```

##### ไฟล์ 5: heartbeat.v (Clock Domain c2: 5 MHz)

**สร้างสัญญาณกระพริบ 1 Hz จากความถี Clock 5 MHz:**

- $\text{Toggle Count} = \frac{5,000,000\text{ Hz}}{2 \times 1\text{ Hz}} = 2,500,000\text{ cycles}$

- นับ $0$ ถึง $2,499,999$ (ใช้ Register ขนาด 23 บิต)

```
module heartbeat (
  input  wire  clk,
  input  wire  rst_n,
  output reg   led
);
  reg [22:0] cnt;
  always @(posedge clk) begin
    if (!rst_n) begin
      cnt <= 23'd0;
      led <= 1'b0;
    end else if (cnt == 23'd2_499_999) begin // 5MHz / (2 x 1Hz) - 1
      cnt <= 23'd0;
      led <= ~led; // toggle 1 Hz
    end else begin
      cnt <= cnt + 1'b1;
    end
  end
endmodule
```

##### ไฟล์ 6: DE0_led_effects.v (Top-Level Module)

เชื่อมต่อทุก Module และแจกจ่าย Clock แต่ละ Domain (ตามโค้ดต้นฉบับในหน้า 6-7):

```
module DE0_led_effects (
    input wire CLOCK_50,
    input wire [2:0] KEY,
    input wire [9:0] SW,
    output wire [9:0] LEDR
);

    wire clk_100m, clk_20m, clk_5m, locked;

    my_pll u_pll (
        .inclk0 (CLOCK_50),
        .c0 (clk_100m),
        .c1 (clk_20m),
        .c2 (clk_5m),
        .locked (locked)
    );

    wire rst_n = KEY[0] & locked;

    pwm u_pwm (
        .clk (clk_100m),
        .rst_n (rst_n),
        .level (SW[7:4]),
        .pwm_out (LEDG[9])
    );

    wire tick;
    wire [7:0] led_bus;

    tick_gen u_tick (
        .clk (clk_20m),
        .rst_n (rst_n),
        .speed (SW[1]),
        .tick (tick)
    );

    ring_counter u_ring (
        .clk (clk_20m),
        .rst_n (rst_n),
        .tick (tick),
        .dir (SW[0]),
        .pause_n (KEY[1]),
        .led (led_bus)
```

```
    );

    assign LEDR[7:0] = led_bus;

    heartbeat u_hb (
        .clk (clk_5m),
        .rst_n (rst_n),
        .led (LEDR[8])
    );
endmodule
```

---

**[ตาราง] Compilation Summary & Verification Checklist (หน้า 7)**

ตารางผลการสังเคราะห์วงจร (Quartus II Synthesis Summary):

| รายการ | ค่าที่ได้ |
|---|---|
| Total logic elements | <span style="color: #0056b3; font-w |
| Total registers | <span style="color: #0056b3; font-w |
| Total PLLs | <span style="color: #0056b3; font-w |
| Fmax ของ clk_100m domain | <span style="color: #0056b3; font-w |
| Fmax ของ clk_20m domain | <span style="color: #0056b3; font-w |
| Fmax ของ clk_5m domain | <span style="color: #0056b3; font-w |

(หมายเหตุ: ในเอกสารไม่ได้ระบุตัวเลขผลการ Compile สำเร็จรูปไว้ ตัวเลขด้านบนเป็นการประเมินทางทฤษฎีจากการนับจำนวน Flip-Flop: cnt 4-bit ใน PWM + cnt 24-bit ใน Tick Gen + led 8-bit ใน Ring Counter + cnt 23-bit ใน Heartbeat รวม 59 registers)

ตารางผลการทดสอบบนบอร์ดฮาร์ดแวร์จริง:

| การทดสอบ | ผลที่ควรได้ | ☐ |
|---|---|---|
| เปิดเครื่อง | LEDR[8] กระพริบสม่ำเสมอ 1 ครั้ง/วิน | <span style="color: #0056b3; font-w |
| กด KEY[0] | LEDR[7] ติดดวงเดียว (1000_0000) | <span style="color: #0056b3; font-w |
| SW[0] = 1 / 0 | ไฟวิ่งซ้าย / ขวา แล้ววนกลับ ไม่หายไ | <span style="color: #0056b3; font-w |
| SW[1] = 0 $\rightarrow$ 1 | ความเร็วเพิ่มจาก 2 Hz เป็น 10 Hz | <span style="color: #0056b3; font-w |
| กด KEY[1] ค้าง | ไฟวิ่งหยุด แต่ LEDR[8] ยังกระพริบต่ | <span style="color: #0056b3; font-w |
| SW[7:4] = 0000 $\rightarrow$ 1111 | LEDR[9] สว่างขึ้น 16 ระดับ ไม่กระพร | <span style="color: #0056b3; font-w |

## 4. ข้อควรระวังและข้อผิดพลาดที่พบบ่อย (Common Mistakes)

1. Clock Domain Crossing (CDC) และการค้างสถานะ: การส่งสัญญาณระหว่างโมดูลที่ทำงานคนละความถี่นาฬิกา เช่น tick_gen กับ ring_counter แม้จะอยู่บนโดเมน $20\text{ MHz}$ เหมือนกัน แต่การสร้างสัญญาณพัลส์ต้องกว้างเพียง 1 clock cycle เท่านั้น หากสร้างพัลส์ยาวเกินไป ring_counter จะหมุนบิตซ้ำหลายครั้งใน 1 จังหวะ

2. การลืม Reset วงจร Ring Counter ด้วยค่า One-Hot: Ring Counter ต้องถูก Reset ให้มีบิต 1 เพียงตัวเดียวเสมอ (เช่น 8'b1000_0000) หาก Reset เป็น 0000_0000 ไฟจะไม่ติดและจะไม่มีบิต 1 วิ่งออกมาเลย

3. การใช้ Blocking Assignment ในโมดูล Sequential: ดังที่เน้นย้ำใน Exercise 3 Bug 3 หากใช้ = แทน <= ใน always @(posedge clk) จะทำให้ลำดับการอัปเดตตัวแปรผิดพลาดและเกิดพฤติกรรมที่ไม่พึงประสงค์ในการสังเคราะห์วงจรจริง

---

## 5. ตารางโยง Lecture $\leftrightarrow$ Lab 6

| แนวคิดจาก Lecture (Week 6) | อยู่ในส่วนใดของโค้ด / Lab 6 | คำอธิบายการนำมาประยุกต์ใช้ |
|---|---|---|
| Up/Down Counter & Enable ([สไลด์ 6- | Exercise 2 (`counter_updown.v`) | ใช้สร้างตัวนับ 4 บิตที่ควบคุมทิศทาง |
| Clock Divider Calculation ([สไลด์ 1 | Exercise 3 & File 5 (`heartbeat.v`) | คำนวณขอบเขตการนับ $\frac{f_{clk}}{2 |
| PLL Parameter Calculation ([สไลด์ 1 | Exercise 4 & File 1 (`my_pll.v`) | คำนวณหาค่า $M, N, K$ เพื่อให้ได้ควา |
| PWM Generation Logic ([สไลด์ 18](fi | File 2 (`pwm.v`) | ใช้ตัวนับ 4-bit Free-running counte |
| Shift Register Concatenation ([สไลด | Exercise 5 & File 4 (`ring_counter. | ใช้ `{led[6:0], led[7]}` และ `{led[ |

| | | |
|---|---|---|
| Tick Generator Pattern ([สไลด์ 11, | File 3 (`tick_gen.v`) | สร้างสัญญาณพัลส์ 1 cycle เพื่อกระตุ |

### 6. ส่วนที่เอกสารไม่ได้ระบุ / ข้อขัดแย้งใน Lab 6

1. การกระโดดข้ามข้อของ Lab Sheet: ใน [Lab6.pdf](file:///d:/vhdl/lab/Lab6.pdf) หลังจากจบ Exercise 5 ในหน้า 3 หัวข้อถัดไปถูกตั้งชื่อว่า "Exercise 9 - Design & Verification" ทันที โดยที่ไม่มี Exercise 6, 7 และ 8 ปรากฏอยู่ในเอกสาร

2. ความสับสนเรื่องชื่อพอร์ต LEDR และ LEDG:

ในหน้า 6 โค้ด DE0_led_effects.v บรรทัดที่ 5 ประกาศพอร์ตเอาต์พุตว่า output wire [9:0] LEDR

แต่ในบรรทัดที่ 26 กลับเรียกใช้งาน .pwm_out (LEDG[9]) ซึ่งพอร์ต LEDG ไม่ได้ถูกประกาศไว้ในพอร์ตลิสต์ของ Top-Level (หาก Compile โดยตรงใน Quartus II จะเกิดข้อผิดพลาด Undeclared identifier)

นอกจากนี้ บอร์ด Altera DE0 จะมีหลอดไฟ LED สีเขียว (LEDG[9:0]) เท่านั้น ไม่มีหลอดไฟสีแดง (LEDR) เหมือนบอร์ดรุ่น DE1/DE2
แต่ในตารางผลการทดสอบหน้า 7 เอกสารกลับใช้ชื่อว่า LEDR[8], LEDR[7], LEDR[9] สลับกัน

---

ต้องการให้ดำเนินการต่อใน Week 7 (Finite State Machine - Part I / Moore FSM & Traffic Light Controller) เลยหรือไม่ครับ?

---

# [WEEK] Week 7: Finite State Machine (FSM) Design

## [LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

# สรุปการเรียนการสอน Week 7: Lecture & Lab 7

## [WEEK] Week 7 - Lecture: Finite State Machine (FSM) Design with Verilog HDL

แหล่งอ้างอิง: [Week7_FSM.pdf](file:///d:/vhdl/lec/Week7_FSM.pdf) (Slide 1-33)

### 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามเอกสาร Slide 2)

1. เข้าใจหลักการทำงานและโครงสร้างพื้นฐานของ Finite State Machine (FSM)
2. เข้าใจความแตกต่าง ข้อดี ข้อจำกัด ระหว่าง Moore FSM และ Mealy FSM
3. เข้าใจรูปแบบการเข้ารหัสสถานะ (State Encoding Styles): Binary, Gray Code, One-hot
4. รู้วิธีการเขียนโค้ด Verilog HDL สำหรับ FSM ด้วยมาตรฐาน 3-Always-Block Style (Moore) และ 2-Always-Block Style (Mealy)

5. ศึกษาการประยุกต์ใช้งาน FSM ผ่านตัวอย่างวงจรจริง:
- Lab A: Traffic Light Controller (Moore FSM)
- Lab B: Sequence Detector "1011" (Mealy FSM, Overlapping)

---

### 2. แนวคิดหลัก (Core Concepts)

#### 2.1 นิยามและองค์ประกอบ 4 ส่วนของ FSM (Slide 3)

FSM คือวงจร sequential logic ที่มีสถานะการทำงานจำกัดและเปลี่ยนแปลงตามสัญญาณนาฬิกา ประกอบด้วย 4 องค์ประกอบหลัก:

1. State Register (หน่วยความจำสถานะ): Flip-flops เก็บสถานะปัจจุบัน (current_state) ทำงานแบบ sequential ตอบสนองตามขอบขา clock

2. Next-State Logic (ตรรกะคำนวณสถานะถัดไป): วงจร combinational ทำหน้าที่คำนวณสถานะถัดไป (next_state) โดยขึ้นกับ current_state และ input

3. Output Logic (ตรรกะสร้างสัญญาณเอาต์พุต): วงจร combinational สร้างเอาต์พุตตามประเภทของ FSM
4. Inputs / Outputs: สัญญาณนำเข้าเพื่อควบคุมการเปลี่ยนสถานะ และสัญญาณส่งออกไปควบคุมระบบภายนอก

---

#### 2.2 การเปรียบเทียบ Moore FSM กับ Mealy FSM (Slide 4-6)

| มิติการเปรียบเทียบ | Moore FSM (Slide 4-5) | Mealy FSM (Slide 4, 6) |
|---|---|---|

| | | |
|---|---|---|
| ฟังก์ชันของ Output | ขึ้นกับ Current State เพียงอย่างเดี | ขึ้นกับ Current State และ Current I |
| การตอบสนอง (Response Time) | ช้ากว่า 1 clock cycle (ต้องรอเปลีย | เร็วกว่า (ตอบสนองใน cycle เดียวกันท |
| ความเสียงต่อ Glitch | ปลอดภัย ไม่มี Glitch (Synchronous, | มีความเสียง Glitch สูง (Combinatio |
| จำนวน State ที่ต้องใช้ | มักใช้จำนวนสถานะ มากกว่า | มักใช้จำนวนสถานะ น้อยกว่าหรือเท่ากั |
| การเขียน State Diagram | เขียนค่า Output กำกับ ภายในวงกลมของ | เขียนค่า Output กำกับ บนลูกศรเส้นเช |

### 2.3 รูปแบบการเข้ารหัสสถานะ (State Encoding Styles) (Slide 7)

สำหรับ FSM ที่มี $N$ สถานะ มีวิธีแทนรหัสบิตของแต่ละ State ดังนี:

1. Binary Encoding:

- ใช้จำนวน Flip-Flop ขั้นต่ำ: $\lceil \log_2 N \rceil$ บิต
- เช่น 6 สถานะ ใช้ $\lceil \log_2 6 \rceil = 3$ Flip-Flops
- ข้อดี: ประหยัดรีซอร์ส Flip-Flop
- ข้อจำกัด: วงจร Next-State Logic มีความซับซ้อนมากกว่า

2. Gray Code:

- มีการเปลี่ยนค่าเพียง 1 บิตในทุกๆ การเปลี่ยนสถานะที่ต่อเนื่องกัน
- ข้อดี: ลด switching noise และลดโอกาสเกิด glitch ใน combinational logic

3. One-Hot Encoding:

- ใช้ 1 Flip-Flop ต่อ 1 สถานะ ($N$ Flip-Flops สำหรับ $N$ สถานะ)
- แต่ละสถานะมีบิตเป็น '1' เพียงบิตเดียว (เช่น 000001, 000010, 000100)
- ข้อดี: Next-state logic ถอดรหัสได้ง่ายและเร็วมาก เหมาะกับสถาปัตยกรรม FPGA ที่มี Flip-Flops จำนวนมาก

### 2.4 โครงสร้างโค้ด Verilog สำหรับ FSM (Slide 11-13, 23)

##### ก) Moore FSM: โครงสร้าง 3-Always-Block (แนะนำตามมาตรฐานอุตสาหกรรม - Slide 11)

```
// Block 1: State Register (Sequential)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        current_state <= IDLE;
    else
        current_state <= next_state;
end

// Block 2: Next-State Logic (Combinational)
always @(*) begin
    next_state = current_state; // Default value ป้องกัน Latch
    case (current_state)
        IDLE: if (in) next_state = S1;
        S1:   if (!in) next_state = IDLE; else next_state = S2;
        default: next_state = IDLE;
    endcase
end

// Block 3: Output Logic (Combinational)
always @(*) begin
    out = 1'b0; // Default value
    case (current_state)
        IDLE: out = 1'b0;
        S1:   out = 1'b0;
        S2:   out = 1'b1;
        default: out = 1'b0;
    endcase
end
```

##### ข) Mealy FSM: โครงสร้าง 2-Always-Block (Slide 23)

- Block 1 (Sequential): State register (always @(posedge clk or negedge rst_n))
- Block 2 (Combinational): รวม Next-State Logic และ Output Logic ไว้ในบล็อก always @() เดียวกัน โดย output ถูกกำหนดค่าภายในเงื่อนไขของ input

**2.5 ตัวอย่างการประยุกต์ใช้งานในสไลด์ (Slide 14-33)**

- Lab A - Traffic Light Controller (Moore FSM, Slide 14-22):

- ควบคุมไฟจราจร 3 สี (RED: 3'b100, GREEN: 3'b001, YELLOW: 3'b010)

- ใช้ Timer นับเวลาหน่วงในการเปลี่ยนสถานะ (Red 10s, Green 8s, Yellow 2s)

- Lab B - Sequence Detector "1011" (Mealy FSM, Slide 29-33):

- Overlapping vs Non-overlapping: เอกสารระบุว่า Overlapping อนุญาตให้นำบิตท้ายที่ตรวจพบแล้วมาใช้ซ้ำ (reuse) เป็นบิตเริ่มต้นของลำดับถัดไปได้ (เช่น 1011011 จะตรวจพบ 2 ครั้ง)

---

## 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. Moore FSM เอาต์พุตขึ้นกับ Current State เท่านั้น ให้สัญญาณที่ปลอดภัย ไร้ Glitch

2. Mealy FSM เอาต์พุตขึ้นกับ Current State และ Current Input ตอบสนองเร็วกว่าแต่อาจเกิด Glitch

3. ในการเขียน Combinational Block สำหรับ Next-State Logic และ Output Logic ต้องใส่ default assignment หรือ default case เสมอ เพื่อป้องกันการเกิด Inferred Latch

4. การเข้ารหัสแบบ One-hot เหมาะกับ FPGA เพราะแลก Flip-flop ที่มีเหลือเฟือกับความเร็วและลดความซับซ้อนของ Logic

---

## 4. จุดที่เอกสารไม่ได้ระบุ

- ในเอกสารไม่ได้ระบุตารางการสังเคราะห์รีซอร์ส (Logic Elements / Flip-Flops) เปรียบเทียบระหว่าง Binary และ One-hot บน FPGA จริง

---

# [LAB] ส่วนที่ 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 7

# [LAB] Week 7 - Lab 7: การออกแบบ Finite State Machine (FSM): Combination Lock System

แหล่งอ้างอิง: [Lab7.pdf](file:///d:/vhdl/lab/Lab7.pdf) (หน้าที่ 1-12)

ไฟล์ที่ทำเสร็จสมบูรณ์: [Lab7_Answered.pdf](file:///d:/vhdl/lab/Lab7_Answered.pdf)

## 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร หน้า 1)

1. ออกแบบและพัฒนาวงจรกลอนประตูดิจิทัล (3-digit Combination Lock System) ด้วย Moore FSM

2. บูรณาการระบบย่อยเข้าด้วยกัน: Edge Detector (ตรวจจับขอบขาปุ่มกด), Tick Generator (ฐานเวลา 1 Hz), FSM Controller, และ 7-Segment Display

3. รหัสผ่านตั้งต้นกำหนดไว้ 3 หลัก คือ PW0 = 3, PW1 = 7, PW2 = 1

4. มีระบบรักษาความปลอดภัย:

- ป้อนถูกครบ 3 หลัก $\rightarrow$ ปลดล็อก (UNLOCKED) เปิดไฟ 5 วินาที แล้ว Auto-lock กลับสู่ IDLE

- ป้อนผิด $\rightarrow$ เข้าสู่ WRONG หน่วงเวลา 1 วินาทีแล้วกลับไป IDLE พร้อมเพิ่มตัวนับความพยายาม (attempt_cnt)

- ป้อนผิดสะสมครบ 3 ครั้ง $\rightarrow$ ล็อกระบบ (LOCKOUT) ร้องเตือน (LEDR ติดหมด) และนับถอยหลัง 10 วินาทีบน HEX0 ก่อนจะปลดล็อกให้ลองใหม่

---

## 2. การเชื่อมโยงเนื้อหา Lecture ↔ Lab 7

- Moore Property: นำโครงสร้าง Moore 3-Always-Block จาก Lecture Slide 11 มาใช้ควบคุมเอาต์พุต unlocked, progress, และ alarm ทำให้สถานะเอาต์พุตไม่มี Glitch
- Clock Divider / Timers: นำหลักการ Counter จาก Week 6 และ Tick Gen มาสร้างจังหวะเวลา 1s สำหรับสถานะ WRONG, 5s สำหรับ UNLOCKED, และ 10s สำหรับ LOCKOUT
- Synchronous Edge Detection: ใช้ Shift Register 2 บิตมาทำ Edge Detector เพื่อแปลงสัญญาณปุ่มกดแบบ Active-Low ให้เป็นพัลส์ความกว้าง 1 clock cycle ป้องกันการป้อนค่าซ้ำหลายครั้งจากการกดปุ่มเพียงครั้งเดียว

---

## 3. เฉลยแบบฝึกหัดในเอกสาร Lab 7 (แสดงคำตอบเป็นสีน้ำเงิน)

**แบบฝึกหัดข้อ 1: ถูก / ผิด (True / False) (หน้า 1)**

1. Moore FSM มี output ที่ขึ้นอยู่กับ state ปัจจุบันเพียงอย่างเดียว ไม่ขึ้นกับ input $\rightarrow$ ถูก

2. Mealy FSM ต้องใช้ 3 always blocks เสมอในการเขียน Verilog $\rightarrow$ ผิด (Mealy มักใช้ 2-always block โดยรวม next-state logic และ output logic เข้าด้วยกัน)

3. One-hot encoding ใช้ flip-flop 1 ตัวต่อ 1 state $\rightarrow$ ถูก

4. ใน state diagram ของ Moore FSM ค่า output จะเขียนอยู่บนลูกศร transition $\rightarrow$ ผิด (Moore เขียน output ในวงกลม state ส่วนที่เขียนบนลูกศรคือ Mealy)

5. สัญญาณจากปุ่มกดเชิงกลจำเป็นต้องผ่าน Edge Detector ก่อนเข้า FSM เสมอ $\rightarrow$ ถูก

6. FSM ที่มี 8 states หากใช้ Binary encoding จะประหยัด flip-flop ได้มากกว่า One-hot encoding $\rightarrow$ ถูก (Binary ใช้ 3 FFs ส่วน One-hot ใช้ 8 FFs)

7. การลืมใส่ default case ใน Next-State Logic อาจทำให้ Quartus สังเคราะห์ Inferred Latch $\rightarrow$ ถูก

8. ใน combinational always block ควรใช้ non-blocking assignment (<=) $\rightarrow$ ผิด (combinational ต้องใช้ blocking assignment (=) เพื่อเลี่ยง race condition)

9. Output ของ Mealy FSM อาจเกิด glitch ได้หาก input เปลี่ยนแปลงระหว่างรอบสัญญาณนาฬิกา $\rightarrow$ ถูก

10. ถ้าไม่ใช้ edge detector การกดปุ่ม 1 ครั้งจะถือเป็นการป้อนรหัส 1 ตัวเสมอ $\rightarrow$ ผิด (การกดปุ่มนานหลายสิบ ms ที่สัญญาณนาฬิกา 50MHz จะทำให้ FSM มองเห็นสัญญาณค้างและทริกเกอร์ซ้ำนับล้านรอบ)

---

**แบบฝึกหัดข้อ 2: เติมคำในช่องว่าง (หน้า 1)**

1. โครงสร้างโค้ดของ Moore FSM ใช้ 3 always block แบ่งเป็น state register, next-state logic, และ output logic

2. โครงสร้างโค้ดของ Mealy FSM ใช้ 2 always block โดยรวม next-state logic กับ output logic ไว้ในบล็อกเดียวกัน

3. Output ของ Moore FSM เป็นฟังก์ชันของ state ปัจจุบัน เพียงอย่างเดียว

4. Output ของ Mealy FSM เป็นฟังก์ชันของ state ปัจจุบัน และ input ปัจจุบัน

5. ถ้า case statement ไม่ครบทุก case ใน combinational always block จะเกิดปัญหาที่เรียกว่า Latch ซึ่งเป็นสิ่งที่ไม่พึงประสงค์ใน synchronous design

6. การหาร CLOCK_50 (50 MHz) ให้ได้ความถี่ 1 Hz ต้องนับจาก 0 ถึง 49_999_999

7. ปุ่ม KEY บนบอร์ด DE0 เป็นแบบ active-low กล่าวคือ ไม่กด = 1, กด = 0

8. ใน sequential always block ควรใช้ <span style="color: #0056b3; font-weight: bold;">non-blocking</span> assignment (เครื่องหมาย <span style="color: #0056b3; font-weight: bold;"><=</span>) เพื่อป้องกัน race condition

9. Overlapping sequence detection คือการตรวจจับที่อนุญาตให้ลำดับที่ตรวจพบแล้ว reuse บิตท้ายกับลำดับถัดไปได้

10. FSM ที่มี 6 สถานะ ใช้ binary encoding ต้องการ state register ขนาด 3 บิต แต่ถ้าใช้ one-hot encoding ต้องการ 6 บิต

---

**แบบฝึกหัดข้อ 3: จับคู่คำศัพท์ FSM (หน้า 2)**

- A. State Register $\rightarrow$ คู่กับ 2 (sequential always block ที่อัปเดต state ตามขอบขา clock)
- B. Next-State Logic $\rightarrow$ คู่กับ 8 (combinational always block ที่คำนวณว่ารอบถัดไปจะไป state ไหน)
- C. Moore Output Logic $\rightarrow$ คู่กับ 4 (combinational logic ที่กำหนดค่า output โดยดูจาก state ปัจจุบันเท่านั้น)

- D. Mealy Output Logic $\rightarrow$ คู่กับ 1 (combinational logic ที่กำหนดค่า output โดยดูทั้ง state และ input)
- E. One-Hot Encoding $\rightarrow$ คู่กับ 3 (รูปแบบการเข้ารหัสที่ใช้ flip-flop 1 ตัวต่อ 1 state นิยมใช้ใน FPGA)
- F. Binary Encoding $\rightarrow$ คู่กับ 7 (รูปแบบการเข้ารหัสที่ใช้ flip-flop จำนวนน้อยที่สุด $\lceil \log_2 N \rceil$)
- G. Inferred Latch $\rightarrow$ คู่กับ 5 (ปัญหาที่เกิดจาก case หรือ if-else ไม่ครอบคลุมทุกกรณีใน combinational block)
- H. Edge Detector $\rightarrow$ คู่กับ 6 (วงจรที่แปลงสัญญาณปุ่มกดให้เป็น pulse กว้าง 1 clock cycle)

---

**แบบฝึกหัดข้อ 4-7: การหาบัคในโค้ด (หน้า 3)**

(หมายเหตุ: เอกสารพิมพ์หัวข้อว่า ข้อ 4-7 มี 4 ข้อ แต่ในเนื้อหาเอกสารมีโค้ดให้ตรวจสอบเพียง 3 บล็อก คือ โค้ดที่ 1, 2 และ 4 โดยไม่มีโค้ดที่ 3)

##### โค้ดที่ 1 (Moore FSM output logic - traffic light)

- บัคอยู่ตรงไหน: บรรทัดที่ 3-7 (ขาดกรณี YELLOW และไม่มี default)
- ผิดอย่างไร / ผลกระทบ: case statement ครอบคลุมไม่ครบทุกสถานะที่เป็นไปได้ ทำให้เกิด Inferred Latch ซึ่งทำให้วงจรทำงานผิดพลาดและไม่เป็น synchronous design ที่ดี
- แก้ไขอย่างไร: เพิ่ม YELLOW: LEDG = 3'b010; และเพิ่ม default: LEDG = 3'b001; ก่อนคำสั่ง endcase

##### โค้ดที่ 2 (Moore FSM - output logic for door lock)

- บัคอยู่ตรงไหน: บรรทัดที่ 5 (VERIFY: unlocked = (key_match) ? 1'b1 : 1'b0;)
- ผิดอย่างไร / ผลกระทบ: ผิดกฎของ Moore FSM เนื่องจากนำสัญญาณ input (key_match) มาประเมินผลค่า output โดยตรง ทำให้กลายเป็น Mealy logic และเสี่ยงต่อการเกิด glitch
- แก้ไขอย่างไร: ตัด key_match ออกจาก output logic โดยกำหนด VERIFY: unlocked = 1'b0; แล้วนำสัญญาณ key_match ไปตัดสินใจเปลี่ยนสถานะใน Next-State Logic แทน

##### โค้ดที่ 4 (Mealy FSM - next-state + output logic)

- บัคอยู่ตรงไหน: บรรทัดที่ 2 (always @(state))
- ผิดอย่างไร / ผลกระทบ: Sensitivity list ไม่สมบูรณ์ ขาดสัญญาณ in ทำให้เมื่อ in เปลี่ยนแปลงแต่ state ยังไม่เปลี่ยน บล็อกนี้จะไม่ถูกประเมินผลใหม่ เกิดการ mismatch ระหว่างการจำลองกับการทำงานจริง
- แก้ไขอย่างไร: แก้ไขเป็น always @() เพื่อให้ compiler รวมสัญญาณ input และ state ทั้งหมดที่ถูกอ่านเข้าสู่ sensitivity list โดยอัตโนมัติ

---

**โค้ด Verilog และการเติมเต็มส่วนที่เว้นว่างใน Lab 7**

##### 1. โมดูล edge_detector.v (หน้า 7)

```
module edge_detector (
    input  wire clk,
    input  wire rst_n,
    input  wire key_in,
    output wire pulse_out
);
    reg r1, r2;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r1 <= /* □□□□ 1 */ 1'b1;
            r2 <= /* □□□□ 2 */ 1'b1;
        end else begin
            r1 <= /* □□□□ 3 */ key_in;
            r2 <= /* □□□□ 4 */ r1;
```

```
        end
    end

    // □□□□□□□□ Falling Edge □□□□□□□□□□□□□ Active-Low
    assign pulse_out = /* □□□□ 5 */ r2 & ~r1;

endmodule
```

##### 2. โมดูล tick_gen.v (หน้า 8)

```
module tick_gen #(
    parameter CNT_MAX = 50_000_000
) (
    input  wire clk,
    input  wire rst_n,
    output reg  tick
);
    reg [25:0] cnt;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            cnt  <= 26'd0;
            tick <= 1'b0;
        end else begin
            if (cnt == /* □□□□ 1 */ 26'd49_999_999); begin
                cnt  <= /* □□□□ 2 */ 26'd0;
                tick <= 1'b1;
            end else begin
                tick <= /* □□□□ 3 */ 26'd0;
                cnt  <= /* □□□□ 4 */ cnt + 1'b1;
            end
        end
    end
endmodule
```

##### 3. โมดูล combo_lock_fsm.v (หน้า 9-10)

```
module combo_lock_fsm (
    input  wire        clk, rst_n, enter, tick,
    input  wire [3:0]  digit,
    output reg         unlocked,
    output reg  [1:0]  progress,
    output reg  [9:0]  alarm,
    output wire [3:0]  lock_timer_out,
    output reg  [1:0]  attempt_cnt_out
);
    localparam PW0 = 4'd3, PW1 = 4'd7, PW2 = 4'd1;
    localparam IDLE=3'd0, D1_OK=3'd1, D2_OK=3'd2, UNLOCKED=3'd3, WRONG=3'd4, LOCKOUT=3'd5;

    reg [2:0] state, next_state;
    reg [1:0] attempt_cnt;
    reg       wrong_timer;
    reg [2:0] unlock_timer;
    reg [3:0] lock_timer;

    // TODO 1: State Register (□□□□ 9)
    always @(posedge clk or negedge rst_n)
        if (!rst_n) state <= /* □□□□ 1 */ IDLE;
        else        state <= /* □□□□ 2 */ next_state;

    // TODO 2: Next-State Logic (□□□□ 9)
    always @(*) begin
        next_state = state;
        case (state)
            IDLE:     if (enter) next_state = /* □□□□ 3 */ (digit==PW0) ? D1_OK : WRONG;
            D1_OK:    if (enter) next_state = /* □□□□ 4 */ (digit==PW1) ? D2_OK : WRONG;
            D2_OK:    if (enter) next_state = /* □□□□ 5 */ (digit==PW2) ? UNLOCKED : WRONG;
            UNLOCKED: if (tick && unlock_timer == /* □□□□ 6 */ 3'd5) next_state = /* □□□□ 7
```

```
            WRONG:    if (tick && wrong_timer) next_state = /* □□□□ 8 */ (attempt_cnt>=2'd3)
            LOCKOUT:  if (tick && lock_timer == /* □□□□ 9 */ 4'd0) next_state = /* □□□□ 10 *
            default:  next_state = IDLE;
        endcase
    end

    // TODO 3: Output Logic (Moore Property) (□□□□ 9)
    always @(*) begin
        case (state)
            IDLE:     begin /* □□□□ 11 */ unlocked=1'b0; progress=2'b00; alarm=10'd0; end
            D1_OK:    begin /* □□□□ 12 */ unlocked=1'b0; progress=2'b01; alarm=10'd0; end
            D2_OK:    begin /* □□□□ 13 */ unlocked=1'b0; progress=2'b10; alarm=10'd0; end
            UNLOCKED: begin /* □□□□ 14 */ unlocked=1'b1; progress=2'b11; alarm=10'd0; end
            WRONG:    begin /* □□□□ 15 */ unlocked=1'b0; progress=2'b00; alarm=10'd0; end
            LOCKOUT:  begin /* □□□□ 16 */ unlocked=1'b0; progress=2'b00; alarm=10'h3FF; end
            default:  begin unlocked=1'b0; progress=2'b00; alarm=10'd0; end
        endcase
    end

    // TODO 4: Timers and Counters (Sequential Logic) (□□□□ 10)
    // 4a) Attempt Counter
    always @(posedge clk or negedge rst_n)
        if (!rst_n)                                attempt_cnt <= /* □□□□ 17 */ 2'd0;
        else if (state == WRONG && enter)          attempt_cnt <= /* □□□□ 18 */ attempt_cnt
        else if (/* □□□□ 19 */ (state == UNLOCKED)) attempt_cnt <= /* □□□□ 20 */ 2'd0;
        else if (/* □□□□ 21 */ (state == LOCKOUT))  attempt_cnt <= /* □□□□ 22 */ 2'd0;

    // 4b) WRONG 1-second delay
    always @(posedge clk or negedge rst_n)
        if (!rst_n)                                wrong_timer <= /* □□□□ 23 */ 1'b0;
        else if (state == WRONG && tick)           wrong_timer <= /* □□□□ 24 */ 1'b1;
        else if (state != WRONG)                   wrong_timer <= 1'b0;

    // 4c) UNLOCKED auto-lock timer (5s)
    always @(posedge clk or negedge rst_n)
        if (!rst_n)                                unlock_timer <= /* □□□□ 25 */ 3'd0;
        else if (state != UNLOCKED)                unlock_timer <= 3'd0;
        else if (state == UNLOCKED && tick)        unlock_timer <= /* □□□□ 26 */ unlock_time

    // 4d) LOCKOUT countdown timer (10s)
    always @(posedge clk or negedge rst_n)
        if (!rst_n)                                lock_timer <= 4'd10;
        else if (state != LOCKOUT)                 lock_timer <= /* □□□□ 27 */ 4'd10;
        else if (state == LOCKOUT && tick)         lock_timer <= /* □□□□ 28 */ lock_timer -

    assign lock_timer_out = lock_timer;
    assign attempt_cnt_out = attempt_cnt;
endmodule
```

### 12A. ตารางทำนาย Output จาก State (Moore Property) (หน้า 11)

| State | unlocked (LEDG[0]) | progress (LEDG[2:1]) | alarm (LEDR[9:0]) |
|---|---|---|---|
| IDLE | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| D1_OK | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| D2_OK | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| UNLOCKED | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| WRONG | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| LOCKOUT | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

### 12B. ตารางทำนาย State Transition (หน้า 11)

| ข้อ | Current State | Input / Condition | Next State ที่คาดหวัง |
|---|---|---|---|
| 1 | IDLE | enter=1, digit=3 (ถูก) | <span style="color: #0056b3; font-w |
| 2 | IDLE | enter=1, digit=5 (ผิด) | <span style="color: #0056b3; font-w |

| | | | |
|---|---|---|---|
| 3 | IDLE | enter=0 | <span style="color: #0056b3; font-w |
| 4 | D1_OK | enter=1, digit=7 (ถูก) | <span style="color: #0056b3; font-w |
| 5 | D1_OK | enter=1, digit=9 (ผิด) | <span style="color: #0056b3; font-w |
| 6 | D2_OK | enter=1, digit=1 (ถูก) | <span style="color: #0056b3; font-w |
| 7 | D2_OK | enter=1, digit=0 (ผิด) | <span style="color: #0056b3; font-w |
| 8 | UNLOCKED | tick=1, unlock_timer == 5 | <span style="color: #0056b3; font-w |
| 9 | WRONG | tick=1, wrong_timer=1, attempt_cnt= | <span style="color: #0056b3; font-w |
| 10 | WRONG | tick=1, wrong_timer=1, attempt_cnt= | <span style="color: #0056b3; font-w |
| 11 | LOCKOUT | lock_timer == 0 | <span style="color: #0056b3; font-w |
| 12 | LOCKOUT | lock_timer == 5 (ยังนับอยู่) | <span style="color: #0056b3; font-w |

**ผลการทดสอบ Checklist บนบอร์ดจริง (หน้า 11-12)**

| ลำดับการทดสอบ | การกระทำ | ผลที่คาดหวัง (เอกสาร) | ผลการทดสอบจริง | ผ่าน |
|---|---|---|---|---|
| 1 | กด KEY[0] (reset) | IDLE: LEDG=000, LEDR=off, HEX0=0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 2 | ตั้ง SW=3, กด KEY[1] | D1_OK: LEDG=010 (progress=01) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 3 | ตั้ง SW=7, กด KEY[1] | D2_OK: LEDG=100 (progress=10) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 4 | ตั้ง SW=1, กด KEY[1] | UNLOCKED: LEDG=111, unlocked=1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 5 | รอ 5 วินาทีไม่กดอะไร | Auto-lock: กลับ IDLE, LEDG=000 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 6 | ป้อนผิด (เช่น SW=5, กด) | WRONG $\rightarrow$ 1s $\rightarrow$ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 7 | ป้อนผิดอีก 2 ครั้ง (รวม 3) | LOCKOUT: LEDR ติดหมด, HEX0 นับ $\ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 8 | รอ 10 วินาทีใน LOCKOUT | กลับ IDLE: LEDR ดับ, HEX0=0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 9 | ป้อน 3 $\rightarrow$ 7 $\rightarrow$ | ปลดล็อกได้ตามปกติ (attempt รีเซ็ต) | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 10 | ป้อนถูกครึ่งทาง แล้วผิด | D1_OK $\rightarrow$ WRONG $\rightarrow$ | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

## 4. ข้อควรระวังและข้อผิดพลาดที่พบบ่อย (ตามเอกสาร)

1. Inferred Latch ใน Next-State Logic: การไม่ใส่ default value หรือเขียน case ไม่ครบทุกสถานะ จะทำให้ compiler สร้าง latch ซึ่งส่งผลให้วงจรไม่เสถียร

2. ละเมิดกฎของ Moore FSM: การนำสัญญาณ input จากภายนอกมาคำนวณ output โดยตรงใน Moore FSM จะทำให้ output เกิด glitch ได้ง่าย

3. Missing Sensitivity List: การเขียน always @(state) แทน always @() ทำให้วงจร combinational ไม่ประเมินผลเมื่อสัญญาณอินพุตอื่นเปลี่ยนแปลง

## 5. ตารางสรุปความเชื่อมโยง Lecture ↔ Lab 7

| แนวคิดจาก Lecture | นำมาใช้ในส่วนใดของ Lab 7 |
|---|---|
| Moore FSM 3-Always-Block Style | บล็อก TODO 1 (State Register), TODO |
| Binary State Encoding | กำหนดสถานะ 6 สถานะ (`3'd0` ถึง `3'd |
| Edge Detection (1-Pulse Generation) | โมดูล `edge_detector.v` ใช้ D Flip- |
| Clock Division / Frequency Division | โมดูล `tick_gen.v` นับ 0 ถึง 49,999 |
| Inferred Latch Prevention | การใส่ `next_state = state;` และ `d |

## 6. ส่วนที่เอกสารไม่ได้ระบุ / ความขัดแย้งในเอกสาร

1. ข้อขัดแย้งในเอกสาร Lab 7 หน้า 3: หัวข้อระบุ "แบบฝึกหัดข้อ 4-7 - หาบัคในโค้ด (4 ข้อ ข้อละ 5 คะแนน) 20 pts" แต่ในเอกสารพิมพ์โค้ดมาเพียง 3 ข้อ คือ โค้ดที่ 1, โค้ดที่ 2 และโค้ดที่ 4 โดยไม่มีโค้ดที่ 3 ปรากฏในเอกสาร

2. ไม่ระบุโค้ด 7-segment decoder ภายใน Lab 7: เอกสารอ้างอิงการแสดงผล HEX0 สำหรับนับเวลาถอยหลังและจำนวนครั้งที่ผิด แต่ไม่ได้แนบ source code ของโมดูล 7-segment ในแลปนี้ (เป็นการเชื่อมต่อจากโมดูลเดิมในระดับ Top-level)

ไฟล์ใบงาน Lab 7 ที่เติมคำตอบและเฉลยครบถ้วนสมบูรณ์ถูกบันทึกไว้ที่:

☐ [Lab7_Answered.pdf](file:///d:/vhdl/lab/Lab7_Answered.pdf)

ต้องการให้ดำเนินการต่อใน Week 8 (Lecture & Lab 8) เลยหรือไม่ครับ?

# [WEEK] Week 8: Testbench Design & Functional Verification

## [LECTURE] ส่วนที่ 1: สรุปเนื้อหา LECTURE

# สรุปการเรียนการสอน Week 8: Lecture & Lab 8

## [WEEK] Week 8 - Lecture: Testbench Design & Functional Verification

แหล่งอ้างอิง: [Week8.pdf](file:///d:/vhdl/lec/Week8.pdf) (Slide 1-33)

(หมายเหตุ: เอกสารต้นฉบับระบุหัวสไลด์ว่า "WEEK 9 · บทที่ 8" โดยมีเนื้อหาครอบคลุมบทที่ 8: Testbench & Verification)

### 1. หัวข้อและวัตถุประสงค์การเรียนรู้ (ตามเอกสาร Slide 3)

1. เข้าใจโครงสร้างพื้นฐานของ Testbench: สามารถระบุและเขียน 4 องค์ประกอบหลัก (DUT, Stimulus Generator, Clock/Reset Generator, Checker/Scoreboard) ได้ด้วยตนเอง

2. การพัฒนา Self-Checking Testbench: เขียนชุดทดสอบที่เปรียบเทียบค่าที่ได้จริงจาก DUT กับค่าที่คาดหวัง และตัดสินผล [PASS] / [FAIL] อัตโนมัติด้วยคำสั่ง $display และตัวแปรนับข้อผิดพลาด (errors) แทนการไล่ดูรูปคลื่นสัญญาณ (waveform) ด้วยตา

3. การบันทึกและตรวจสอบ Waveform: ใช้งาน $dumpfile และ $dumpvars เพื่อสร้างไฟล์ VCD (Value Change Dump) สำหรับเปิดดู waveform ใน ModelSim-Altera เพื่อการดีบักเชิงลึก

4. ทำความเข้าใจระดับการจำลอง (Simulation Levels): เข้าใจความแตกต่างระหว่าง RTL Simulation, Post-Synthesis Simulation, และ Post-Implementation (Timing) Simulation

5. แนวคิด Functional Coverage: เข้าใจความแตกต่างระหว่าง Code Coverage กับ Functional Coverage (State Coverage และ Transition Coverage) ในการทดสอบ FSM

### 2. แนวคิดหลัก (Core Concepts)

#### 2.1 โครงสร้าง 4 ส่วนของ Testbench (Slide 5-6)

Testbench คือโมดูล Verilog พิเศษที่ไม่มีพอร์ต I/O ภายนอก (module tb_name; ... endmodule) และไม่ถูกสังเคราะห์ (non-synthesizable) ลงชิปฮาร์ดแวร์จริง ทำหน้าที่เป็นสิ่งแวดล้อมจำลอง (simulation environment) เพื่อห่อหุ้มและตรวจสอบวงจรเป้าหมาย (Design Under Test: DUT):

1. Device Under Test (DUT): วงจรที่เราออกแบบ ถูกนำมาต่อแบบ Named Port Mapping
2. Clock & Reset Generator: สร้างสัญญาณฐานเวลาจำลอง เช่น always #5 clk = ~clk; (คาบ 10 ns = 100 MHz)

3. Stimulus Generator: สร้างชุดสัญญาณอินพุตทดสอบในบล็อก initial begin ... end ควบคุมจังหวะเวลาด้วย #delay, @(posedge clk), และ repeat(N)

4. Checker / Scoreboard: ตรวจสอบค่าเอาต์พุตที่ได้จริงเทียบกับค่าที่คาดหวัง และรายงานผล

#### 2.2 การสร้างกลไก Self-Checking และการใช้ Task (Slide 8-10)

- Inline Self-Check: เปรียบเทียบค่าทันทีหลังการกระตุ้น โดยต้องหน่วงเวลาสั้นๆ #1; หลังขอบขา clock เพื่อให้สัญญาณ non-blocking assignment (<=) อัปเดตเสร็จสมบูรณ์ก่อนอ่านค่า ป้องกัน Race Condition

- Equality Operator: ควรใช้ Case Equality (=== และ !==) แทน Logical Equality (== และ !=) เสมอ เพราะสามารถจับสถานะไม่แน่นอน (X) และ High-impedance (Z) ที่อาจหลุดออกมาจากวงจรได้
- Reusable Checker Task: ยุบกระบวนการเปรียบเทียบเป็น Task เพื่อเรียกใช้ซ้ำได้สะดวก:

```
integer errors = 0;
task check_count(input [3:0] exp, input [127:0] label);
begin
    if (count !== exp) begin
```

```
        $display("[FAIL] %s: expected=%0d got=%0d @%0t", label, exp, count, $time);
        errors = errors + 1;
    end else begin
        $display("[PASS] %s: count=%0d", label, count);
    end
end
endtask
```

### 2.3 ความแตกต่างของ System Tasks ในการพิมพ์ผล (Slide 11-12)

| System Task | เวลาที่พิมพ์ค่า (Execution Region) | ความเหมาะสมในการใช้งาน |
|---|---|---|
| `$display(...)` | พิมพ์ทันทีที่ค่าสั่งถูกเรียก (Active | ข้อความ Checkpoint, Snapshot ข้อมูล |
| `$monitor(...)` | พิมพ์อัตโนมัติเมื่อตัวแปรใน argumen | ติดตามการเปลี่ยนแปลงของสัญญาณตลอด s |
| `$strobe(...)` | พิมพ์ที่ท้ายสุดของ time step (Postp | ตรวจสอบค่าที่นิ่งจริง ป้องกันปัญหา |

### 2.4 ระดับการจำลองการทำงาน (Simulation Levels) (Slide 19-22)

1. RTL Simulation (Functional Simulation):

- จำลองพฤติกรรมโค้ด Verilog โดยตรง เป็นแบบ Zero-delay (ไม่มี gate delay หรือ wire delay)
- รันเร็วที่สุด ใช้ตรวจสอบความถูกต้องเชิงตรรกะเบื้องต้น
- ข้อจำกัด: มองไม่เห็น Glitch, Setup/Hold violation, และปัญหา Inferred Latch

2. Post-Synthesis Simulation (Gate-Level Netlist):

- จำลองจาก Netlist ของ Logic Elements ที่ได้หลังการสังเคราะห์วงจร
- ช่วยตรวจจับปัญหา Latch ที่ไม่ตั้งใจ, ตรวจสอบ logic ที่ถูกตัดทิ้ง (dead logic), และดูพฤติกรรม power-up state

3. Post-Implementation / Post-Fit (Timing) Simulation:

- รวม Gate Delay และ Routing Delay ที่ดึงมาจากชิปจริง (ผ่านไฟล์ SDF - Standard Delay Format)
- จำลองใกล้เคียงฮาร์ดแวร์จริงที่สุด ตรวจพบ Glitch/Hazard และตรวจสอบเงื่อนไขเวลา (Setup/Hold Violation) ร่วมกับเครื่องมือ TimeQuest ก่อนนำไปโปรแกรมลงบอร์ดจริง

### 2.5 แนวคิด Functional Coverage (Slide 29)

- Code Coverage: วัดเชิงปริมาณว่าโค้ดบรรทัดใด หรือ Branch/Condition ใดถูก execute ไปแล้วบ้าง
- Functional Coverage: วัดเชิงคุณภาพว่า สถานการณ์ตาม Specification ถูกทดสอบครบถ้วนหรือไม่

- State Coverage: ทุกสถานะใน FSM ถูกเข้าถึงอย่างน้อย 1 ครั้งหรือไม่
- Transition Coverage: ทุกลูกศรการเปลี่ยนสถานะใน State Diagram ถูกกระตุ้นอย่างน้อย 1 ครั้งหรือไม่

### 2.6 Debug Checklist: บักที่พบบ่อยใน Testbench (Slide 30)

1. ใช้ == / != แทน === / !== ทำให้ตรวจไม่เจอค่า X หรือ Z
2. เซ็ตค่าทันทีที @(posedge clk) โดยไม่ใส่ #1; ทำให้เกิด Race Condition อ่านค่าเก่า
3. ลืม deassert สัญญาณ reset ก่อนเริ่มป้อน stimulus
4. ลืมเรียกคำสั่ง $finish; ทำให้ simulation รันค้างไม่สิ้นสุด

## 3. สรุปประเด็นสำคัญของ Lecture (Key Takeaways)

1. Verification กินเวลา 50-70% ของกระบวนการพัฒนาฮาร์ดแวร์ การเขียน Testbench จึงสำคัญเทียบเท่าการออกแบบ RTL

2. Testbench ที่ดีต้องเป็น Self-Checking ตัดสิน PASS/FAIL ด้วยโค้ดโปรแกรม ไม่พึ่งพาการมอง waveform ด้วยตา

3. ต้องใช้ #1; หลังขอบขา clock เสมอเพื่อให้อ่านค่าเอาต์พุตหลังการอัปเดตแบบ non-blocking
4. RTL simulation ผ่าน ไม่ได้แปลว่าวงจรจะทำงานบนบอร์ดจริงได้เสมอ ต้องคำนึงถึงปัญหา Timing และ Glitch

## 4. จุดที่เอกสารไม่ได้ระบุ

- เอกสารไม่ได้ระบุตัวอย่างโค้ด SystemVerilog covergroup / coverpoint แบบละเอียด
  (ระบุเพียงว่าเป็นเครื่องมือมาตรฐานอุตสาหกรรมที่อยู่นอกขอบเขตคอร์ส)

---

## [LAB] ส่วนที่ 2: เฉลยและอธิบายแบบฝึกหัดปฏิบัติการ LAB 8

## [LAB] Week 8 - Lab 8: Testbench Design & Functional Verification

แหล่งอ้างอิง: [Lab8.pdf](file:///d:/vhdl/lab/Lab8.pdf) (หน้าที่ 1-16)

ไฟล์ที่ทำเสร็จสมบูรณ์: [Lab8_Answered.pdf](file:///d:/vhdl/lab/Lab8_Answered.pdf)

### 1. โจทย์และวัตถุประสงค์ของ Lab (ตามเอกสาร หน้า 1-2)

1. ฝึกทักษะการตรวจสอบและหาบักใน Testbench (Debugging Case Studies)

2. ฝึกเขียน Testbench ตั้งแต่เริ่มต้นสำหรับวงจร Combinational (Full Adder, 2-to-1 MUX, 2-to-4 Decoder) พร้อมบันทึก Truth Table จากการจำลองใน ModelSim

3. พัฒนา Self-Checking Testbench สำหรับ Mod-10 Up/Down Counter และ Traffic Light Moore FSM

4. เป็น Simulation-only week - ใช้ ModelSim-Altera และ Quartus II เพื่อการจำลองเท่านั้น ไม่มีการโปรแกรมลงบอร์ด DE0

---

### 2. การเชื่อมโยงเนื้อหา Lecture ↔ Lab 8

- DUT Instantiation & Stimulus: นำโครงสร้าง Testbench จาก Slide 5-7 มาเขียนควบคุมวงจร Full Adder, MUX, และ Decoder

- Race Condition Prevention: นำกฎการหน่วงเวลา #1; จาก Slide 8, 30 มาเติมใน Testbench ของ Counter และ Traffic Light FSM
  เพื่อให้อ่านค่าได้ถูกต้อง
- Case Equality (!==): นำมาใช้ใน Task checker สำหรับตรวจจับค่าที่ไม่ตรงกับ spec รวมถึงค่า X และ Z

- Simulation Control: ใช้ $dumpfile, $dumpvars, และ $finish ตามมาตรฐานที่อธิบายใน Slide 13-15

---

### 3. เฉลยแบบฝึกหัดในเอกสาร Lab 8 (แสดงคำตอบเป็นสีน้ำเงิน)

#### แบบฝึกหัด 3.1: Debugging Case Studies (หน้า 4-5)

##### Case Study 1: Shift Register (SIPO) (หน้า 4)

| บักที่ | บรรทัด | อาการ / สาเหตุ | วิธีแก้ |
|---|---|---|---|
| 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 2 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 3 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

##### Case Study 2: Priority Encoder (หน้า 5)

| บักที่ | บรรทัด | อาการ / สาเหตุ | วิธีแก้ |
|---|---|---|---|
| 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 2 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w | valid !== exp_valid` |

#### แบบฝึกหัด 3.2: ถูก / ผิด (True / False) (หน้า 5)

1. Self-checking testbench ต้องอาศัยคนนั่งดู waveform เปรียบเทียบค่าด้วยตาเสมอ จึงจะตัดสิน PASS/FAIL ได้ $\rightarrow$ ผิด (Self-checking ตัดสินผลอัตโนมัติด้วยโปรแกรมเปรียบเทียบค่าจริงกับค่าที่คาดหวัง)

2. ควรใช้ === และ !== แทน == และ != ในการเปรียบเทียบค่าใน testbench เพื่อรองรับค่า X/Z ได้ถูกต้อง $\rightarrow$ ถูก (Case equality ตรวจสอบค่าบิต 0, 1, X, Z ได้ตรงเป๊ะทุกบิต)

3. $dumpfile และ $dumpvars ใช้สำหรับสร้างไฟล์ .vcd เพื่อเปิดดู waveform ใน ModelSim $\rightarrow$ ถูก

4. ไฟล์ testbench (tb_.v) จะถูก synthesize ลงบอร์ด DE0 เช่นเดียวกับ DUT $\rightarrow$ ผิด (Testbench ใช้สำหรับการจำลองเท่านั้น ไม่ถูกสังเคราะห์ลงฮาร์ดแวร์)

5. การลบ #1 ทีอยู่หลัง @(posedge clk) ก่อนเรียก task ตรวจสอบผล จะไม่มีผลใดๆ ต่อผลการทดสอบ $\rightarrow$ ผิด (จะเกิด Race Condition ทำให้อ่านค่าเก่าก่อนทีสัญญาณ non-blocking จะอัปเดต)

6. $finish ต้องถูกเรียกเพือจบการจำลอง มิฉะนัน simulation อาจรันค้างไม่จบ $\rightarrow$ ถูก

7. Post-Implementation (Timing) Simulation ใช้ delay model ทีใกล้เคียงวงจรจริงทีสุดในบรรดาการซิมทังสามระดับ $\rightarrow$ ถูก (จำลองทัง Gate Delay และ Wire Routing Delay จากชิปจริง)

8. ถ้า testbench ผ่าน RTL simulation แล้ว รับประกันได้ว่าวงจรจะทำงานถูกต้องบนบอร์ดจริงเสมอ $\rightarrow$ ผิด (RTL simulation เป็นแบบ zero-delay ไม่เห็นปัญหา Glitch และ Timing Violations)

9. State coverage หมายถึงการตรวจสอบว่า testbench ได้ทดสอบครบทุก "การเปลียนสถานะ (transition)" ของ FSM แล้ว $\rightarrow$ ผิด (การตรวจครบทุกการเปลียนสถานะเรียกว่า Transition Coverage ส่วน State Coverage ตรวจว่าเข้าครบทุกสถานะหรือไม่)

10. ลำดับคำสังทีถูกต้องในการรัน ModelSim-Altera คือ vlib → vlog → vsim → run -all $\rightarrow$ ถูก

---

### 4. แบบฝึกหัดอุ่นเครือง - โค้ด Testbench และตารางบันทึกผล (หน้า 6-10)

##### 4.1 วงจรที 1: Full Adder (หน้า 6-7)

```
`timescale 1ns/1ps
module tb_full_adder;
  reg a, b, cin;
  wire sum, cout;

  full_adder DUT (.a(a), .b(b), .cin(cin), .sum(sum), .cout(cout));

  initial begin
    $display("Time\ta b cin | sum cout");
    $monitor("%0t\t%b %b  %b  |  %b    %b", $time, a, b, cin, sum, cout);
    a=0; b=0; cin=0; #10;
    a=0; b=0; cin=1; #10;
    a=0; b=1; cin=0; #10;
    a=0; b=1; cin=1; #10;
    a=1; b=0; cin=0; #10;
    a=1; b=0; cin=1; #10;
    a=1; b=1; cin=0; #10;
    a=1; b=1; cin=1; #10;
    $finish;
  end
endmodule
```

ตารางบันทึกผลลัพธ์ Full Adder (หน้า 7):

| # | a | b | cin | sum (จากการซิม) | cout (จากการซิม) |
|---|---|---|---|---|---|
| :-: | :-: | :-: | :-: | :-: | :-: |
| 1 | 0 | 0 | 0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 2 | 0 | 0 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 3 | 0 | 1 | 0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 4 | 0 | 1 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 5 | 1 | 0 | 0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 6 | 1 | 0 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 7 | 1 | 1 | 0 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |
| 8 | 1 | 1 | 1 | <span style="color: #0056b3; font-w | <span style="color: #0056b3; font-w |

- คำถามวิเคราะห์ (หน้า 7): ตรงกับสมการตรรกะทุกประการทัง 8 กรณี วงจร Full Adder ทำงานถูกต้องสมบูรณ์ตามทฤษฎี ไม่มีข้อผิดพลาด โค้ด DUT และ Testbench สอดคล้องกันอย่างสมบูรณ์

---

##### 4.2 วงจรที 2: 2-to-1 Multiplexer (หน้า 8-9)

```
`timescale 1ns/1ps
module tb_mux2to1;
  reg sel, i0, i1;
  wire y;

  mux2to1 DUT (.sel(sel), .i0(i0), .i1(i1), .y(y));

  initial begin
    $monitor("%0t\tsel=%b i0=%b i1=%b -> y=%b", $time, sel, i0, i1, y);
    sel=0; i0=0; i1=0; #10;
    sel=0; i0=0; i1=1; #10;
    sel=0; i0=1; i1=0; #10;
    sel=0; i0=1; i1=1; #10;
    sel=1; i0=0; i1=0; #10;
    sel=1; i0=0; i1=1; #10;
    sel=1; i0=1; i1=0; #10;
    sel=1; i0=1; i1=1; #10;
    $finish;
  end
endmodule
```

ตารางบันทึกผลลัพธ์ 2-to-1 MUX (หน้า 8-9):

| # | sel | i0 | i1 | y (จากการซิม) |
|---|---|---|---|---|
| :-: | :-: | :-: | :-: | :-: |
| 1 | 0 | 0 | 0 | <span style="color: #0056b3; font-w |
| 2 | 0 | 0 | 1 | <span style="color: #0056b3; font-w |
| 3 | 0 | 1 | 0 | <span style="color: #0056b3; font-w |
| 4 | 0 | 1 | 1 | <span style="color: #0056b3; font-w |
| 5 | 1 | 0 | 0 | <span style="color: #0056b3; font-w |
| 6 | 1 | 0 | 1 | <span style="color: #0056b3; font-w |
| 7 | 1 | 1 | 0 | <span style="color: #0056b3; font-w |
| 8 | 1 | 1 | 1 | <span style="color: #0056b3; font-w |

- คำถามวิเคราะห์ (หน้า 9): ตรงกับพฤติกรรมทุกประการ: เมื่อ sel=0 ค่า y=i0 เสมอ (ไม่ขึ้นกับ i1) และเมื่อ sel=1 ค่า y=i1 เสมอ (ไม่ขึ้นกับ i0) วงจร Multiplexer ทำงานคัดเลือกข้อมูลได้ถูกต้อง

##### 4.3 วงจรที่ 3: 2-to-4 Decoder with Enable (หน้า 9-10)

```
`timescale 1ns/1ps
module tb_decoder2to4;
  reg en;
  reg [1:0] sel;
  wire [3:0] y;

  decoder2to4 DUT (.en(en), .sel(sel), .y(y));

  initial begin
    $monitor("%0t\ten=%b sel=%b -> y=%b", $time, en, sel, y);
    en=0; sel=2'b00; #10;
    en=0; sel=2'b01; #10;
    en=0; sel=2'b10; #10;
    en=0; sel=2'b11; #10;
    en=1; sel=2'b00; #10;
    en=1; sel=2'b01; #10;
    en=1; sel=2'b10; #10;
    en=1; sel=2'b11; #10;
    $finish;
  end
endmodule
```

ตารางบันทึกผลลัพธ์ 2-to-4 Decoder (หน้า 10):

| # | en | sel[1:0] | y[3:0] (จากการซิม) |
|---|---|---|---|

| :-: | :-: | :-: | :-: |
|---|---|---|---|
| 1 | 0 | 2'b00 | <span style="color: #0056b3; font-w |
| 2 | 0 | 2'b01 | <span style="color: #0056b3; font-w |
| 3 | 0 | 2'b10 | <span style="color: #0056b3; font-w |
| 4 | 0 | 2'b11 | <span style="color: #0056b3; font-w |
| 5 | 1 | 2'b00 | <span style="color: #0056b3; font-w |
| 6 | 1 | 2'b01 | <span style="color: #0056b3; font-w |
| 7 | 1 | 2'b10 | <span style="color: #0056b3; font-w |
| 8 | 1 | 2'b11 | <span style="color: #0056b3; font-w |

- คำถามวิเคราะห์ (หน้า 10): ตรงตามเงื่อนไขทุกประการ: เมื่อ en=0 ค่า y เป็น 4'b0000 ทุกกรณี และเมื่อ en=1 ค่า y เป็นรูปแบบ one-hot มีบิตเป็น '1' เพียงบิตเดียวตรงตามรหัส sel เสมอ ไม่มีบิตใดเป็น 1 พร้อมกันเกินหนึ่งบิต

### 5. ส่วนที่ 1: Self-Checking TB สำหรับ Mod-10 Up/Down Counter (หน้า 11-13)

##### เฉลยช่องว่างในโค้ด tb_updown_counter.v และตารางคำตอบ (หน้า 12)

| ช่องว่าง | คำตอบของนักศึกษา | คำอธิบายทางเทคนิค |
|---|---|---|
| (1) | <span style="color: #0056b3; font-w | กลับค่าสัญญาณนาฬิกาทุก 5 หน่วยเวลา |
| (2) | <span style="color: #0056b3; font-w | ใช้ Case Inequality เพื่อตรวจสอบค่า |
| (3) | <span style="color: #0056b3; font-w | หน่วงเวลาสั้นๆ หลัง posedge clk ป้อ |
| (4) | <span style="color: #0056b3; font-w | คำสั่งวนซ้ำ `repeat (10) @(posedge |
| (5) | <span style="color: #0056b3; font-w | ค่าที่คาดหวังเมื่อนับถอยหลังจาก 0 ใ |
| (6) | <span style="color: #0056b3; font-w | คำสั่งยุติการจำลองการทำงาน |

##### ผลลัพธ์ Transcript ใน ModelSim (หน้า 13)

```
[PASS] after reset: count=0
[PASS] wrap 9->0 after 10 counts: count=0
[PASS] wrap 0->9 counting down: count=9
TESTBENCH PASSED
```

### 6. ส่วนที่ 2: Self-Checking TB สำหรับ Traffic Light FSM (Moore) (หน้า 14-16)

##### เฉลยช่องว่างในโค้ด tb_traffic_light.v และตารางคำตอบ (หน้า 15-16)

| ช่องว่าง | คำตอบของนักศึกษา | คำอธิบายทางเทคนิค |
|---|---|---|
| (1) | <span style="color: #0056b3; font-w | คืนค่า tick กลับเป็น 0 หลังผ่านไป 1 |
| (2) | <span style="color: #0056b3; font-w | ตรวจสอบไฟสีแดง (`RED = 3'b100`) ทัน |
| (3) | <span style="color: #0056b3; font-w | ป้อนพัลส์ tick จำนวน 4 ครั้งตาม spe |
| (4) | <span style="color: #0056b3; font-w | ตรวจสอบไฟสีเหลือง (`YELLOW = 3'b010 |
| (5) | <span style="color: #0056b3; font-w | สถานะ YELLOW ต้องอยู่ 2 ticks ก่อน |
| (6) | <span style="color: #0056b3; font-w | คำสั่งยุติการจำลองการทำงาน |

## 4. ข้อควรระวังและข้อผิดพลาดที่พบบ่อย (ตามเอกสาร)

1. Race Condition ใน Testbench: หากเช็คค่าเอาต์พุตทันทีที @(posedge clk) โดยไม่หน่วงเวลา #1; โปรแกรมจะอ่านค่าเก่าก่อนที่ non-blocking assignment (<=) ใน DUT จะอัปเดตเสร็จ ทำให้ผลการทดสอบผิดพลาด

2. การลืมคำสั่ง $finish;: ทำให้โปรแกรม ModelSim รัน simulation ค้างไม่สิ้นสุด เพราะบล็อกสร้างสัญญาณนาฬิกา (always #5 clk = ~clk;) ทำงานวนลูปตลอดเวลา

3. การใช้ตัวดำเนินการเปรียบเทียบผิดประเภท: การใช้ == แทน === ทำให้ไม่สามารถตรวจจับค่า X (Unknown) หรือ Z (High-Impedance) ที่เกิดจากการลืม reset วงจรได้

## 5. ตารางสรุปความเชื่อมโยง Lecture ↔ Lab 8

| แนวคิดจาก Lecture | นำมาใช้ในส่วนใดของ Lab 8 |
|---|---|

| | |
|---|---|
| Testbench Skeleton (DUT + Stimulus | โครงสร้างโมดูลใน `tb_full_adder.v`, |
| Self-Checking Pattern (`!==` & Erro | Task `check_count` ในส่วนที 1 และ |
| Race Condition Mitigation (`#1` del | การใส่ `#1;` หลัง `@(posedge clk)` |
| VCD Waveform Generation | คำสั่ง `$dumpfile("tb_counter.vcd") |
| State & Transition Coverage | แผนการทดสอบของ Traffic Light FSM ที |

### 6. ส่วนที่เอกสารไม่ได้ระบุ

- ในเอกสารไม่ได้ระบุการเขียน Testbench แบบสุ่มอินพุต (Constrained-Random Verification) หรือการทำ Assertion-Based Verification (SVA)

ไฟล์ใบงาน Lab 8 ที่เติมคำตอบและเฉลยครบถ้วนสมบูรณ์ถูกบันทึกไว้ที่:
☐ [Lab8_Answered.pdf](file:///d:/vhdl/lab/Lab8_Answered.pdf)

## [WEEK] Lab 9 (Special): การออกแบบวงจร AES-128 Encryption ด้วย Generative AI

แหล่งอ้างอิง: [lab9_Lab_AES128_Encryption 1.pdf](file:///d:/vhdl/lab/lab9_Lab_AES128_Encryption%201.pdf) (หน้าที 1-12)

### 1. วัตถุประสงค์และสเปกมาตรฐาน (หน้าที 1-3)

- วงจรรับ Plaintext ขนาด 128 บิต (16 ไบต์) และ Cipher Key ขนาด 128 บิต เข้าคำนวณ Ciphertext ขนาด 128 บิต ตามมาตรฐาน NIST FIPS-197
- วงจรทำหน้าที่เข้ารหัส (Encryption) เพียงอย่างเดียว ทำงานแบบ Synchronous Sequential

### 2. ลำดับขั้นตอนการทำงาน 10 Rounds ของ AES-128 (หน้าที 3)

1. Round 0: state = plaintext XOR round_key[0] (ทำเฉพาะ AddRoundKey เท่านั้น)
2. Round 1 ถึง 9: ทำครบ 4 ขั้นตอน:
- SubBytes: การแทนที่ไบต์แบบ Non-linear ผ่านตาราง S-Box
- ShiftRows: การเลื่อนไบต์ในแต่ละแถวของ State Matrix
- MixColumns: การคูณเชิงเมทริกซ์ในฟิลด์จำกัด Galois Field GF(2^8) เพื่อกระจายบิต (Diffusion)
- AddRoundKey: นำ State มา XOR กับกุญแจย่อยประจำรอบ round_key[r]
3. Round 10 (รอบสุดท้าย):
- state = AddRoundKey( ShiftRows( SubBytes(state) ), round_key[10] )
- ข้อควรระวังสำคัญ: ใน Round 10 จะไม่มีขั้นตอน MixColumns โดยเด็ดขาดตามมาตรฐานสากล

### 3. หลักการคณิตศาสตร์ของ S-Box (หน้าที 4)

- S-Box ไม่ใช่การสุ่มค่า แต่สร้างขึ้นจาก GF(2^8) Multiplicative Inverse ร่วมกับ Affine Transformation เหนือ $GF(2)$ เพื่อสร้างคุณสมบัติ Confusion ที่สูงสุด

## [WEEK] Lab 11 (Special): ตัวนับขึ้น/ลงบน 7-Segment ด้วย RV32I SoC

แหล่งอ้างอิง: [Lab11_SoC_7seg_Counter.pdf](file:///d:/vhdl/lab/Lab11_SoC_7seg_Counter.pdf) (หน้าที 1-4)

### 1. โครงสร้างหน่วยประมวลผลและการจัดสรรหน่วยความจำ (Memory Map) (หน้าที 2)

- ระบบใช้ซอฟต์คอร์ซีพียู RISC-V สถาปัตยกรรม RV32I Single-Cycle
- แผนที่หน่วยความจำ (Address Decoder addr_decoder.v จากบิต addr[31:28]):
- 0x0000_0000: Instruction ROM (imem.v) โหลดจากไฟล์ไบนารี main.hex
- 0x1000_0000: Data RAM (dmem.v) สำหรับตัวแปรและ Stack
- 0x2000_0000: Memory-Mapped I/O Registers สำหรับควบคุมจอ 7-Segment และปุ่มกด

### 2. การแบ่งหน้าที่ระหว่างฮาร์ดแวร์และซอฟต์แวร์ (Hardware/Software Co-Design) (หน้าที 3-4)

- ฮาร์ดแวร์ (hex7seg.v): ทำหน้าที่ถอดรหัสเลขฐานสอง 4 บิต (0-F) ออกเป็นสัญญาณดิดดับของแต่ละขีด 7 ส่วน (Common Anode Active-Low)

- ซอฟต์แวร์ (ภาษา C บน RV32I): โปรแกรมภาษา C มีหน้าที่เพียงเขียนค่าตัวเลขลงใน Memory-Mapped Register เช่น HEX0 = count; โดยไม่ต้องถอดรหัสบิตของ 7-segment เองในโค้ด C
- การตรวจจับปุ่มกดในระดับซอฟต์แวร์ (Software Edge Detection):

- ความเร็ว Loop ของซีพียูเร็วกว่าการกดปุ่มของมนุษย์มาก

- ในภาษา C ต้องเก็บค่าปุ่มของรอบก่อนหน้า (KEY_prev) เพื่อนำมาเปรียบเทียบกับรอบปัจจุบัน (KEY_curr) โดยนับเฉพาะจังหวะที่ปุ่มเปลี่ยนสถานะจากไม่กดเป็นกดเท่านั้น เพื่อป้องกันการนับซ้ำหลายรอบในการกดปุ่มค้าง

## ☐ ตารางสรุปภาพรวมเนื้อหาการเรียนรู้ตลอดหลักสูตร (Week 2 - Week 8)

ตามข้อกำหนดของหลักสูตร สรุปความเชื่อมโยงระหว่างการบรรยาย (Lecture) และการปฏิบัติการ (Lab) ตั้งแต่ Week 2 ถึง Week 8 ดังนี้:

| สัปดาห์ | หัวข้อ Lecture | หัวข้อ Lab | แนวคิดหลักที่เชื่อมโยงกัน |
|---|---|---|---|
| Week 2 | Quartus II Software Installation & | Practice Lab Sheet 2: Project Creat | การแปลงแนวคิดวงจรดิจิทัลสู่ฮาร์ดแวร |
| Week 3 | Foundations & Verilog Language Basi | Lab 3: Basic Logic Gates & Structur | ไวยากรณ์พื้นฐานของ Verilog HDL: Mod |
| Week 4 | Combinational Logic Design | Lab 4: Multiplexer, Decoder, ALU & | วงจรที่ไม่มีหน่วยความจำ: การออกแบบ |
| Week 5 | Sequential Logic Design | Lab 5: Latches, Flip-Flops, Registe | วงจรที่มีหน่วยความจำตามสัญญาณนาฬิกา |
| Week 6 | Counters and Shift Registers | Lab 6: Mod-N Counters, Clock Divide | วงจรนับเชิงเวลาและการเลื่อนข้อมูล: |
| Week 7 | Finite State Machine (FSM) Design | Lab 7: Combination Lock System (Moo | การควบคุมระบบเชิงสถานะ: โครงสร้าง M |
| Week 8 | Testbench Design & Functional Verif | Lab 8: Self-Checking Testbench for | การตรวจสอบความถูกต้องเชิงฟังก์ชันอั |

ดำเนินการสรุปเนื้อหาและจัดทำไฟล์แลปเฉลย Week 2 ถึง Week 8 ครบถ้วนสมบูรณ์ตามข้อกำหนดทุกประการครับ